Technical Article

Author : *Yves Puelings, Pieter Dewulf* Date : 10 May 2004

LMS TEST.LAB™ AUTOMATION

Updated for revision 5.A

1. INTRODUCTION

A standard software application usually aims at offering and integrating the overall majority of its target industry's common practice. Some even succeed in re-styling the users' way of working in order to boost his performance and provide fit-for-purpose solutions. The LMS Test.Lab™ product family is definitely such a trendsetter. It's a group of applications, which result from years of field experience, passed on from CADA-X, and a refreshing vision aimed at solving the customers' problems more quickly.

Next to using LMS Test.Lab™ applications as an integrated, streamlined set of measurement and analysis functions, there are a few other ways in which users might want to address its capabilities. Let's take a look at some different viewpoints.

1.1 Directing Test.Lab™ applications.

In a number of cases Test.Lab™ applications need to be driven according to an externally defined workflow scheme. Controlling the sequence in which specific functionality is to be executed, is just one of the typical examples. Conditioning of measurement and process parameters according to company standards is also a frequently occurring request.

1.2 Monitoring Test.Lab™ applications

Dedicated user scenarios most often require the observation and reading of Test.Lab™ results. This is for instance the case in doing remote test follow-up or automatic broadcasting of measurement read-outs.

1.3 Analysis extensions

Even though there's an extensive list of processing and analysis functions in Test.Lab™, a lot of users require being able to perform their own proprietary routines on measured or intermediate Test.Lab™ data. Measurement validation, break-squeal analysis, comfort analysis are only a few of examples to illustrate this requirement.

1.4 Integration Third-party

Merging Test.Lab™ applications with other third-party technologies has proven to be the solution for rationalizing and improving efficiency in the users' measurement process. One-man-operation or remote operation, for instance, becomes feasible when bridging between Test.Lab™ and roller-bench or climate chamber control, automatic SMS or e-mail messaging, speech recognition, ... etc.

1.5 Test.Lab™ toolbox services

From the customers' viewpoint, interested in doing special development on top of Test.Lab™ applications, it 's a no-brainer that he would like to re-use some of the

Test.Lab™ internal components to improve consistency and avoid redundancy. This might come down to accessing block, waterfall or geometry operations, or even display or data navigation functionality.

Only few add-ons can be associated with only one single viewpoint. In almost every case it's a mixture of more than one of previous categories.

In order to meet all of these user requirements, we have provided controlled access to Test.Lab™. When selecting the appropriate mechanism to do so, a number of different software technologies were evaluated thoroughly. We'll comment on a few of them briefly.

First of all, there was the option of writing our own development environment, as with UPA in CADA-X. Since Test.Lab™ is fundamentally different from CADA-X, the environment cannot be ported, nor does it meet all the functionality one nowadays expects from a native windows development environment. It would also take a couple of man-years to get this to a mature level in terms of functionality coverage, support and documentation for instance.

Another possibility was to make Test.Lab™ accessible by means of API functions. This however, requires a much higher level of development knowledge, thus increasing the initial threshold and decreasing usability.

A common problem to both alternatives is that they lack the 'look-and-feel' reference. Each will have its own set of functions structured, unrelated to the way other typical MS Windows applications can be accessed.

These were only a few of the considerations, which led to having Test.Lab™ support 'Automation'.

2. WHAT IS AUTOMATION?

Automation is a technology that allows software packages to expose their features to development tools and other applications. It allows to:

- Create applications and programming tools that expose objects and their functionality. The application, which exposes, is being called the 'server'.
- Create and manipulate objects exposed in one application from another application. An application, which uses exported functionality, is called a 'client'.

In other words, the Test.Lab™ applications can expose some of their functionality by means of automation. Therefore it has been enhanced with the necessary server capabilities.

2.1 Key concepts.

In order to support automation, a server application needs to describe which objects it will export, and what kind of actions can be performed on those objects. This language independent description provides information to the automation services of how client applications can interface with the server. Supplying such a description for automation is like applying for a telephone number. You'll need to supply your personal co-ordinates on a standard form, deliver it to the phone company and they'll make sure you get assigned a phone or fax number which they add to the online or offline phone pages.

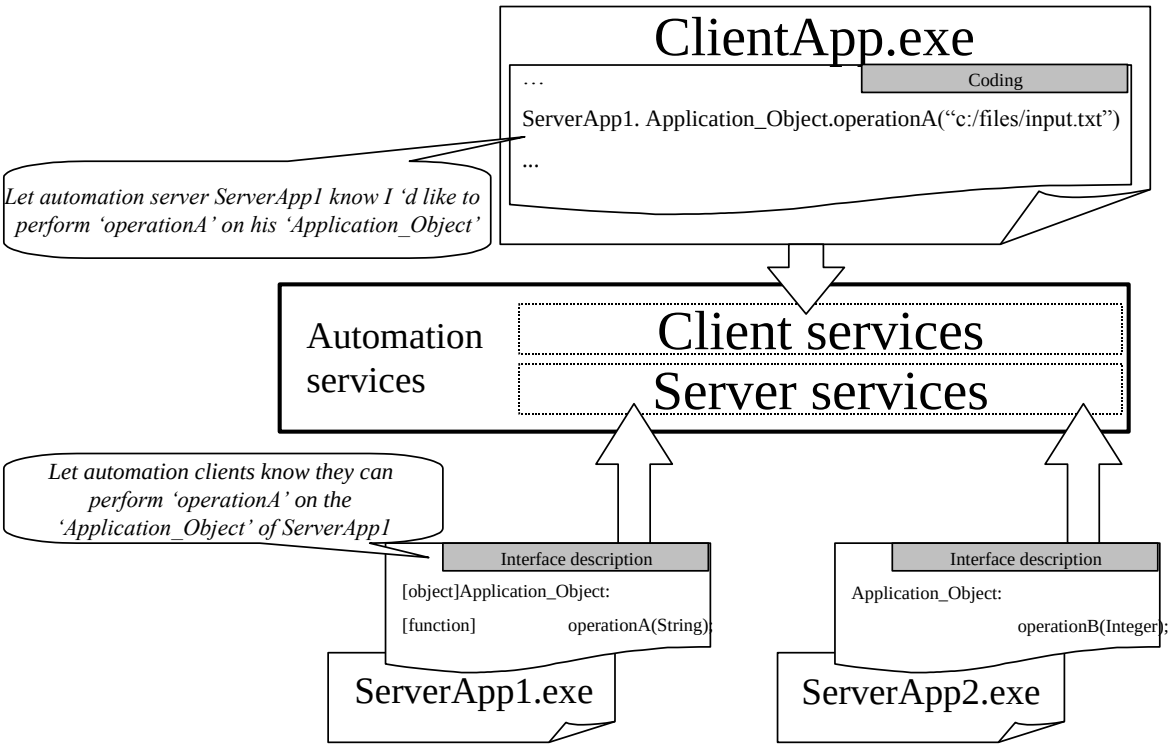


Figure 1

The previous figure shows how both server and client applications can call upon the common services to communicate. The automation layer provides language and Windows platform independence. Although Automation is specially aimed at supporting interpreted languages, like Visual Basic and MATlab, it just as well can be used from compiled languages like Visual C++, J++, Java, C#.

An additional strong point in the binding between client and server application is that they both can run as separate processes. Such a configuration will improve stability. When the client application should crash, the server application will continue running without further disadvantage.

2.2 Object model.

An object model, associated with an application, is a description of the set of objects, their properties and methods, and the relations between these objects.

Applications, which support automation, supply a kind of roadmap, which you can use to navigate from one object to another. Once you reached the desired object, you can query for some characteristics or perform some actions, defined on it.

Following is an extract of the object model defined by MS Excel. MS Word, PowerPoint, Outlook, Visio and Publisher are only a few other MS applications, which define analog models.

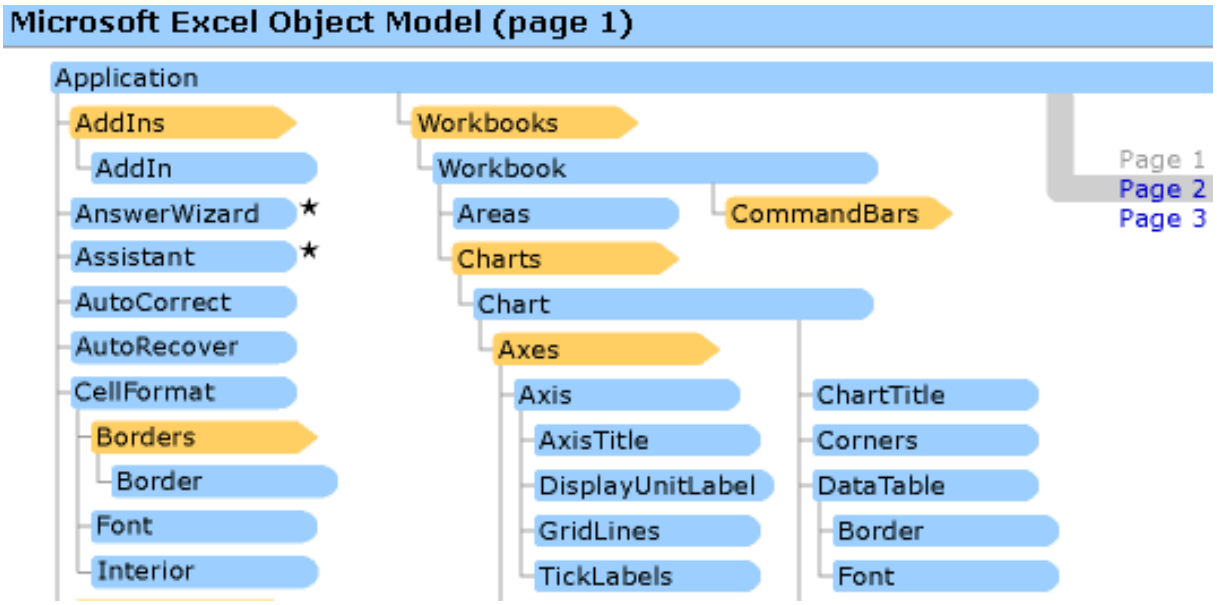


Figure 2

Using this Excel object model, we can, for instance, change the AxisTitle of a chart in an Excel workbook. As a convention the main object you can get access to is called 'Application' and represents the complete application. In the case of Excel this object can be requested to return a list of all 'Workbooks'. This object is a collection, i.e. a list, of zero or more individual books opened. A name or sequence number, to return an individual item, can index such a collection. Once you've got the target workbook, the client can navigate charts, resulting in a single chart. From there it goes deeper into the axes structure of the chart itself, down to the AxisTitle object.

2.2.1 Interfaces

An interface is a group of semantically related functions that provide access to an automation object. Each interface defines a contract that allows objects to interact according to the given Component Object Model (COM). For instance in the case of the Excel Application, following VB commands would return a reference to the main Excel application object.

```
Dim ExcelApp As Excel.Application
Set ExcelApp = CreateObject("Excel.Application")
```

The ExcelApp returned is a reference to an object, which is managed by MS Excel, and which has agreed to implement all functionality, defined by the 'Application' interface. So once you have such an object, you are safe to perform any operation defined in the interface.

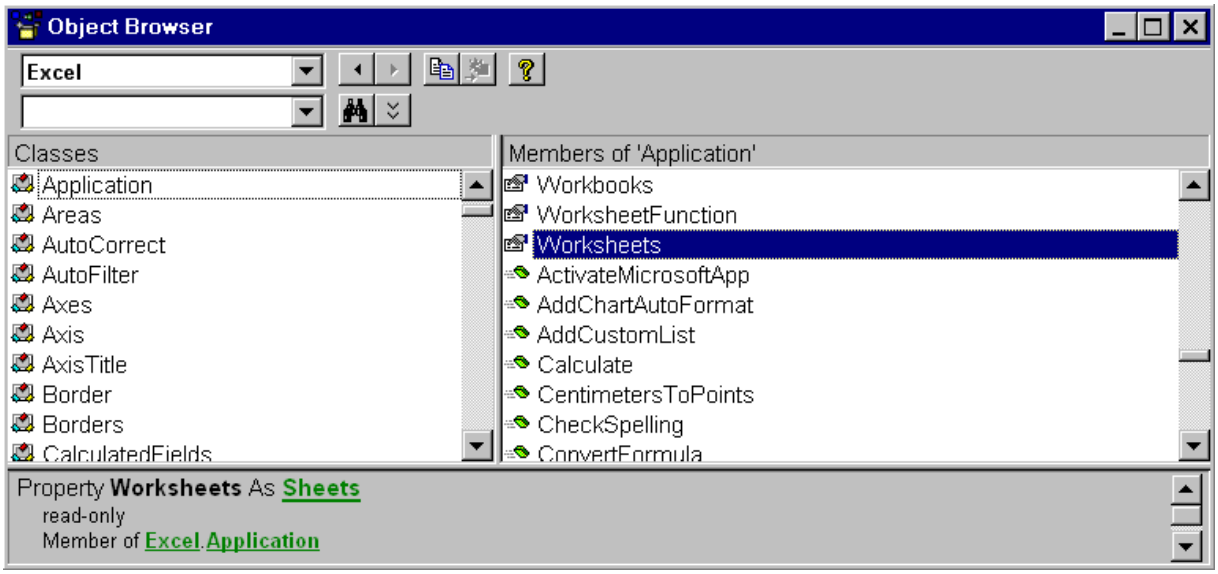


Figure 3

Previous figure shows a subset of the list of 'things' you can do with such an Application object. The highlighted member will return an object, which has committed to provide all functionality on the Sheets interface. This is what the VB code would look like.

```
Dim ExcelSheets As Excel.Sheets
Set ExcelSheets = ExcelApp.Sheets
```

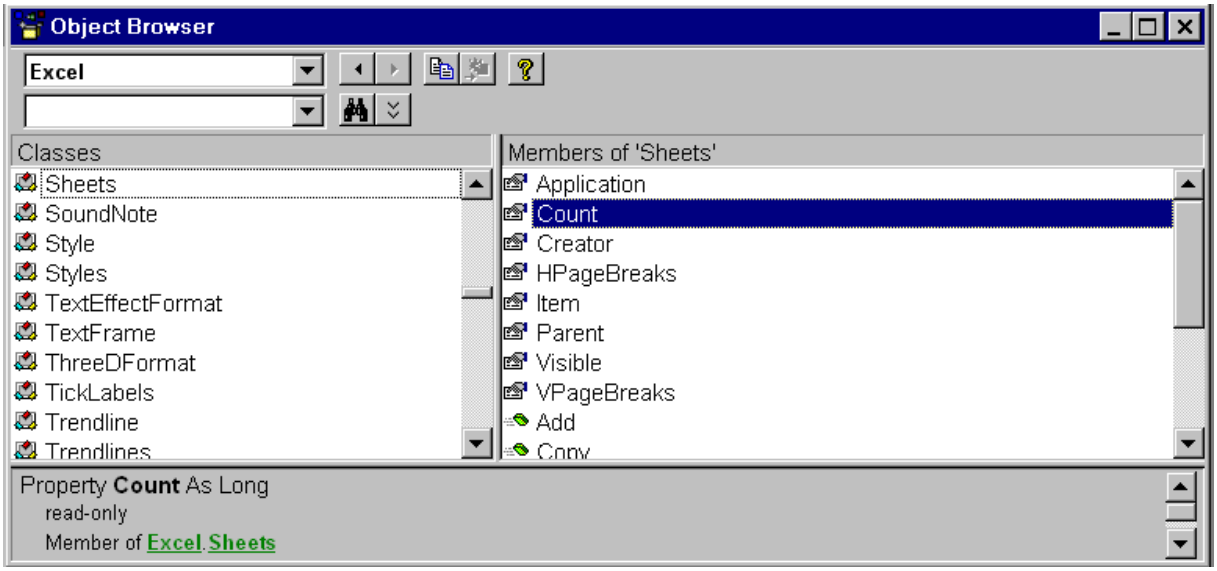


Figure 4

As you can see in figure 4, any object supporting the 'Sheets' interface will need to implement the 'Count' function, which will return the number of sheets this application has.

In fact there are only three types of members, which an interface can provide.

- Methods

- Properties
- Events

2.2.2 Properties

An interface property returns discrete state information that is associated with an object. It merely reflects the objects' condition. The number of sheets in a workbook, the width of a window, the number of columns in a grid, the title of a paragraph, the visibility or sensitivity of a sheet, the X and Y values of a data block are only a few examples of properties.

2.2.3 Methods

A method actually makes the object perform a specific action. This can be opening a project or document, bringing a specific sheet to the front, saving data, ... etc.

2.2.4 Events

'Events' are notifications from the server application to the clients. These notifications are asynchronous. This means that a client's call-back routine can be triggered without it having to do a busy-wait for this event. So instead of performing an action which will keep pending until an event occurs on the server side, you just need to specify a routine which will be called, should the situation occur Some interfaces supply events, which will report the client when a project is opened, when a cell is changed, when a measurement is launched, ... etc.

2.3 Type library.

Previously mentioned information regarding different interfaces, their methods, properties and events, associated parameters, parameter types and short descriptions are stored in a separate file, called a type library. The type library file is a binary file, usually has a .TLB filename extension and can be used by other applications to get information about the automation server. Next to any written documentation, this is probably the most important source of information regarding the object model, which is exposed by a server application. There are some viewing tools that allow inspecting a type library's contents.

- Object browser. When using the Visual Basic Development environment, selecting 'View>Object Browser (F2)' will launch the Object browser to view the Type Lib contents. Figure 3 shows an instance of such a browser. Before you can view a specific type library, it needs to be added to the 'Project>Reference' list in the VB IDE.
- OLE viewer. This is an 'Object Linking and Embedding' viewing tool. Its 'File>View TypeLib...' entry will allow you to select a .tlb file and opens it in an 'ITypeLib viewer' window. This OLE viewer comes with the Visual C++ development environment. This viewer shows the interface type information in a way, which is closer to the raw description, provided by the server application. Next is an

example of this view viewing the information on the Excel Application object.

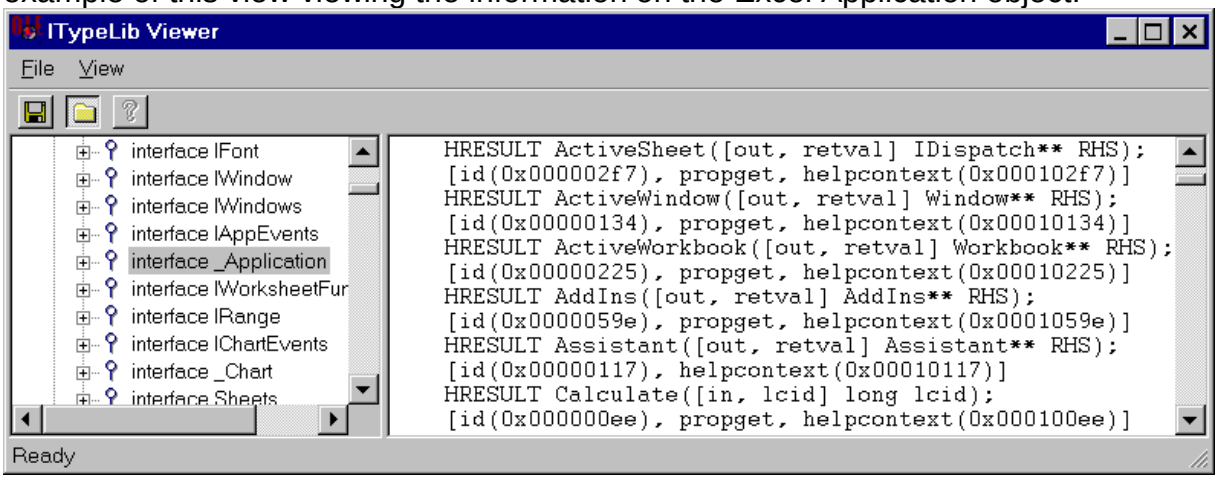


Figure 5

- Some freeware tools. There are some stand-alone type-library-viewing tools available on the Internet.

3. TEST.LAB™ WORKBOOK LOADER AUTOMATION INTERFACES.

Any windows system, even with few applications installed, easily has a couple of hundred type libraries available for client applications to use. This shows that it is common practice for an application to export functionality this way. So anyone, acquainted with inspecting one type library, can reuse his experience to navigate any object model, described in any type library.

Test.Lab™ also has it own set of interfaces defined. The description of the Test.Lab™ object model can be found in its type library, called LMSTestLabAutomation.tlb. It is copied to the <installation>/Central/Configuration directory during Test.Lab™ installation.

If you add a reference to 'LMS Test.Lab™ Automation Type Library', the object browser will show all interfaces supported by the Test.Lab™ applications.

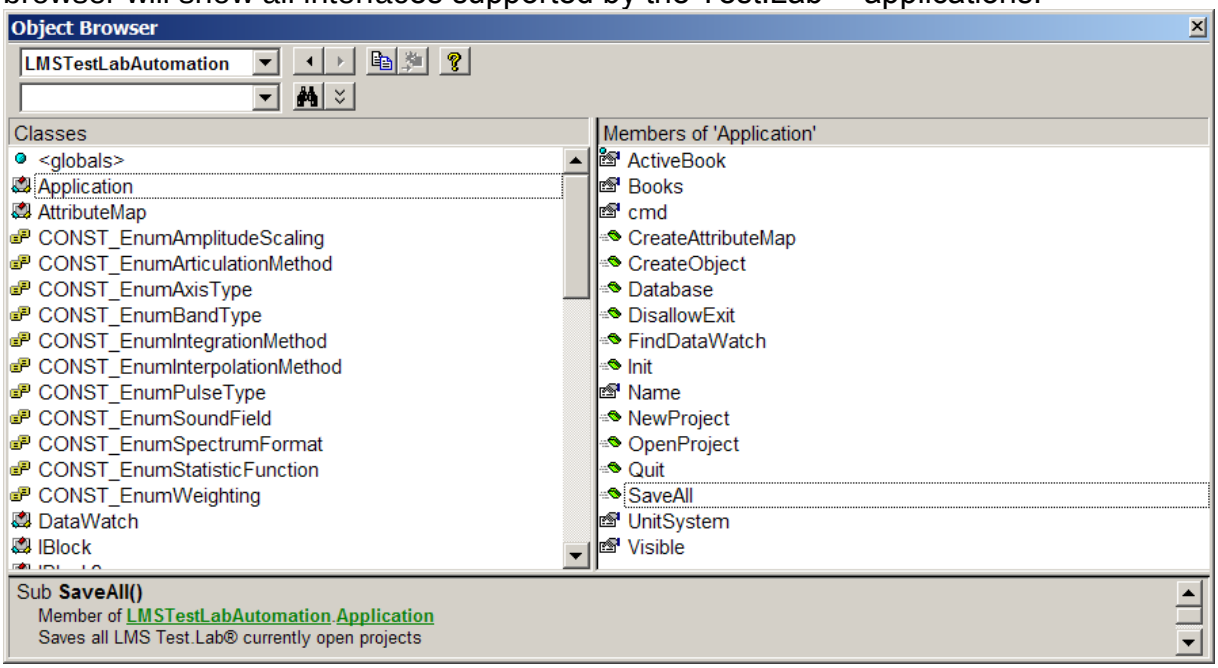


Figure 6

We'll elaborate one each of these interfaces individually.

3.1 Application.

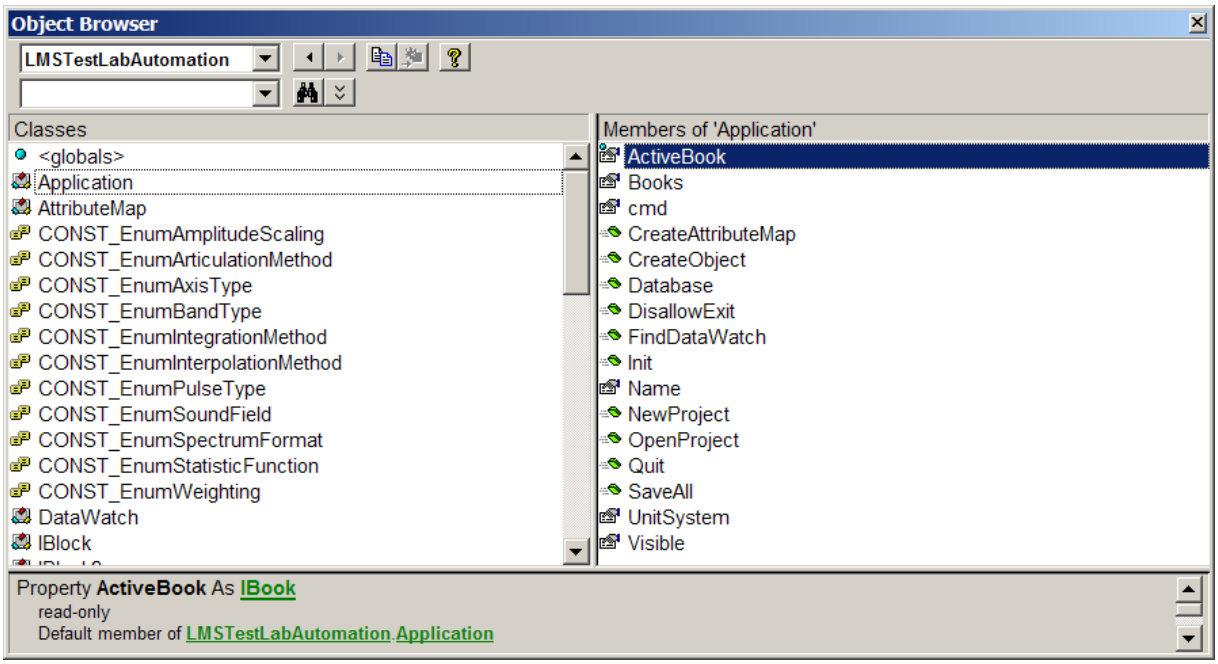


Figure 7

In automation, by convention, an object called 'Application' represents the entire server application. If you would go looking on your system to find a SignatureFS executable, or a SpectralAcquisition executable, you 'd end up empty handed. That is

because any Test.Lab™ application is one and the same executable (LmsHq_ActiveCompVC_LmsLoader_v5.exe) being instructed to load different descriptions for running different types of applications. It can be compared to a CD player, which only contains some general services, while the actual songs are on the individual CD discs. As long as a CD disc complies with the protocol the CD player supports, the player is able to play the music, different kinds on different discs. The Test.Lab™ workbook loader is the main executable that offers some application independent services like for instance unit system manipulation. The specific application aspects are contained in a workbook, an .awb file. So this one executable can load different workbooks resulting in different workbook functionality, e.g. SignatureFS, SpectralAcquisition, Environmental testing, ... etc. .

The next few lines of VB code show how to create of an automated Test.Lab™.

```
Dim TestLab As LMSTestLabAutomation.Application
Set TestLab = CreateObject("LMSTestLabAutomation.Application")
```

The execution of the VB CreateObject instruction can have two results.

- If a Test.Lab™ application was already running, it kind of hooks up to this running application, and automatically uses the workbook that is loaded.
- If no Test.Lab™ application is running, it launches the workbook loader executable, without loading any workbook. You can instruct the loader to load a workbook using the 'Init' method in the Application interface. E.g.

```
TestLab.Init("-w ModalImpactStandard")
```

3.1.1 Method: Init.

Signature:

```
Init(ApplicationStartupArguments As String)
```

Description:

'Init' instructs the TestLab object to initialize with the passed StartupArguments. You can supply a number of options as arguments:

- -w <workbook name>

  Specifies the name of the workbook to be loaded. All workbook descriptions are in the <installation>/Central/Workbooks directory.

- -a <architecture>

  Specifies the architecture to be used. This specifies, for instance, options that are shared between different workbooks. By default, just specify the workbook name without the 'Standard' or 'Basic' extension

- -t <template name>

  Specifies the template to be used at startup.

- -b

  Indicates to start the workbook without automatically opening a project.

- -c <configuration file>

Specifies the configuration file to be used. This configuration file can contain some personalisation settings, like colors, styles, layouts, etc. .

- <project name>

  Specifies the project name to use at startup.

Example:

```
TestLab.Init ("-w ModalImpactStandard -a ModalImpact C:/MyProjects/PrjDB.lms")
```

Initializes the Test.Lab™ workbook loader application with the ModalImpactStandard workbook, using the ModalImpact architecture and creating one instance, given the C:/MyProject/PrjDB.lms as its project.

3.1.2 Property: Books

Type: Read-only

Description:

This property returns the collection of workbook instances, currently active in the Test.Lab™ application. Figure 8 shows the DesktopStandard workbook, twice instantiated. One based on project1 and a second time with project2. Such a list of instantiated workbooks can be retrieved by the 'Books' property.

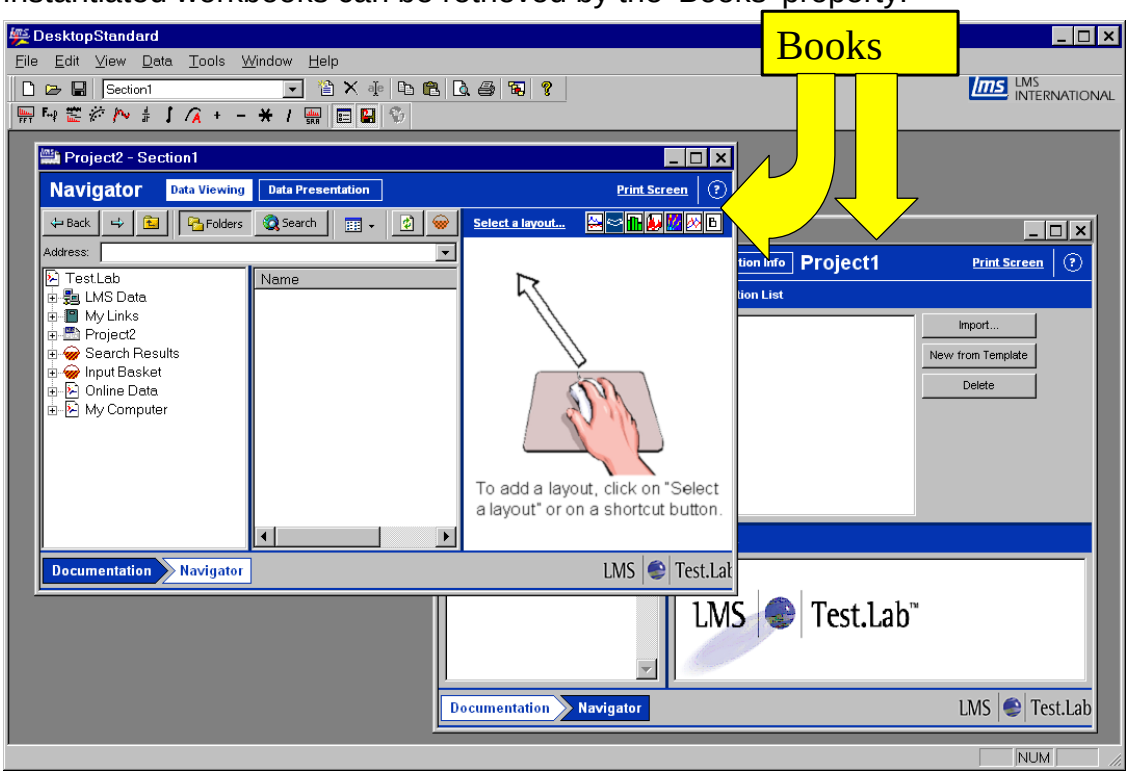


Figure 8

Example:

```
Dim List_of_opened_books As LMSTestLabAutomation.IBooks

Set List_of_opened_books = TestLab.Books
```

3.1.3 Property: ActiveBook

Type: Read-only

Description:

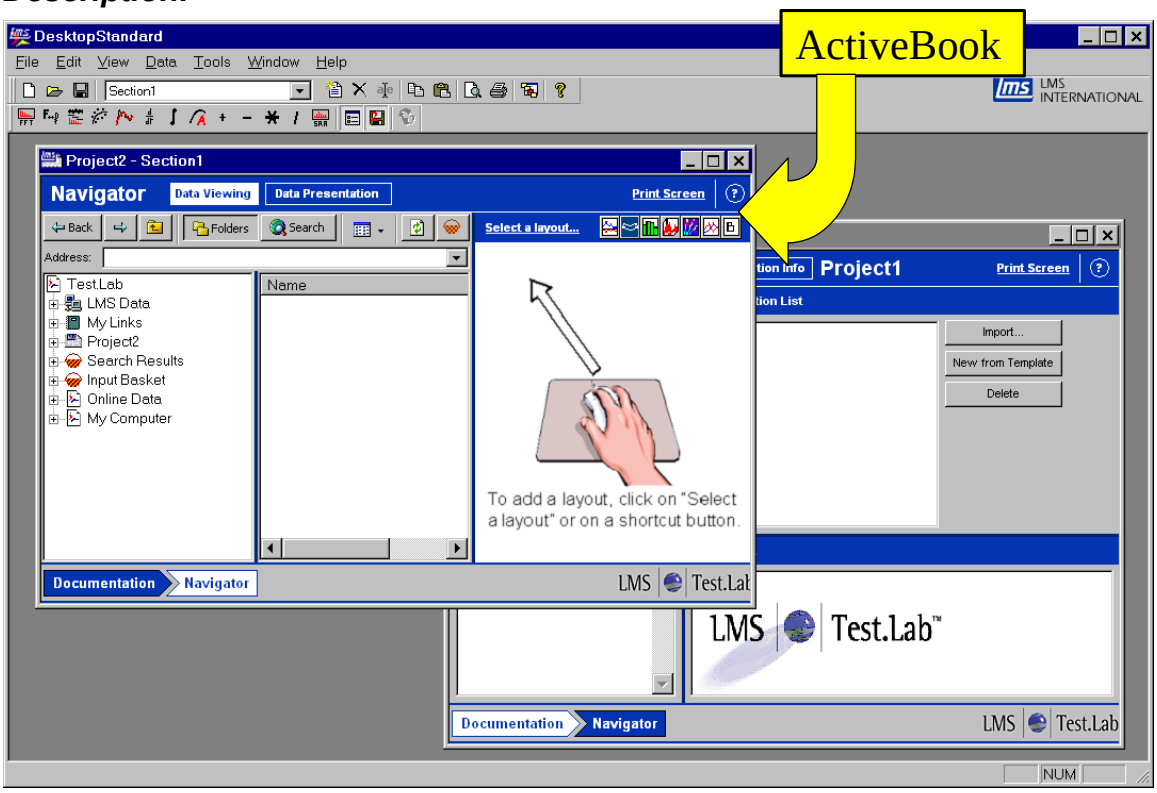


Figure 9

This property allows an automation client to get a reference to whichever book has been selected as the active one. The active MDI window has a highlighted title bar.

It will return a different book when the user selects another MDI window, that way making the other book the active one. The active book is one of the books in the list returned by the 'Books' property.

Example:

```
Dim SelectedBook As LMSTestLabAutomation.IBook

Set SelectedBook = TestLab.ActiveBook
```

3.1.4 Property:UnitSystem

Type: Read-only

Description:

The UnitSystem property, returns a reference to an object in Test.Lab™ that is responsible for handling unit system operations. Further interaction with this unit-system-managing object can be done by means of the members defined on the IUnitSystem interface. (Ref. IUnitsystem.)

Example:

```
Dim TestLab_UnitSys as LMSTestLabAutomation.IUnitSystem

Set TestLab_UnitSys = TestLab.UnitSystem
```

This assigns a reference to the TestLab unit system to a local variable, TestLab_UnitSys. This variable is of type LMSTestLabAutomation.IUnitSystem and so it supports all member functions and properties in the IUnitSystem interface, defined in LMSTestLabAutomation.

3.1.5 Property: Visible

Type: Read-Write

Description:

This functionality could be used to show or hide the Test.Lab™ User Interface. This however is not yet supported.

This visibility property is not related to the fact whether the application window is minimized or not. So you can continue to use Test.Lab™ automation, and have full response and control even when the Test.Lab™ GUI is iconified.

Example:

```
If  TestLab.Visibility = true then ...
```

Previous code lets the user test the visibility of the UI.

```
TestLab.Visibility = false
```

Causes Test.Lab™ to be made invisible.

3.1.6 Method: NewProject

Signature:

```
NewProject() as String
```

Description:

This method opens a new instance of a workbook description, passed by means of the 'Init' method, and pops-up as a new window in the Test.Lab™ application. It doesn't take any parameters, so it will just create a temporary project and use that to create the book. Upon saving, of course, a 'save as' project name will be required. As shown in the VB signature above, the method returns a string. This string holds a unique id associated with the created workbook instance. You can use this id to select a specific book out of the list of books.

Example:

```
TestLab.NewProject ()
```

This will create a new instance, but doesn't captures the returned new book id.

3.1.7 Method: OpenProject

Signature:

```
OpenProject(ProjectPathName As String) As String
```

Description:

This allows you to create a book, supplying the absolute pathname to the project it will have to use. Again the return value is a unique id of the book created as result of calling this method.

Example:

```
Bookid = TestLab.OpenProject("c:/MyProject/PRJ1234.lms")
```

3.1.8 Method: Quit

Signature:

```
Quit()
```

Description:

Quits the complete Test.Lab™ application. This method doesn't include saving any projects before shutting down, nor does it present a message box to ask the user whether he wants to save his data or not. This can be done in the client application depending on the desired automation scenario.

Example:

```
TestLab.Quit
```

3.1.9 Method: SaveAll

Signature:

```
SaveAll()
```

Description:

As mentioned earlier, a single Test.Lab™ application can have several instances of a workbook running, each related to a project. The 'SaveAll' method instructs all books to save their associated project. It's shorthand for iterating all the open books and save each in turn.

Example:

```
TestLab.SaveAll
```

3.1.10 Method: FindDataWatch

Signature:

```
FindDataWatch(<General Port ID> [,MaxUpdateFrequency ] [,IsAutoBuffered])
```

General Port ID: ID of the port.

MaxUpdateFrequency: the interval, specified in milliseconds, with which the system will check the contents of the port and notify the client application if the value on the port has changed since the previous check. Default is 1000

IsAutoBuffered: a boolean value to indicate that the value returned from the datawatch needs to be returned as a buffered, normalised type.

Description:

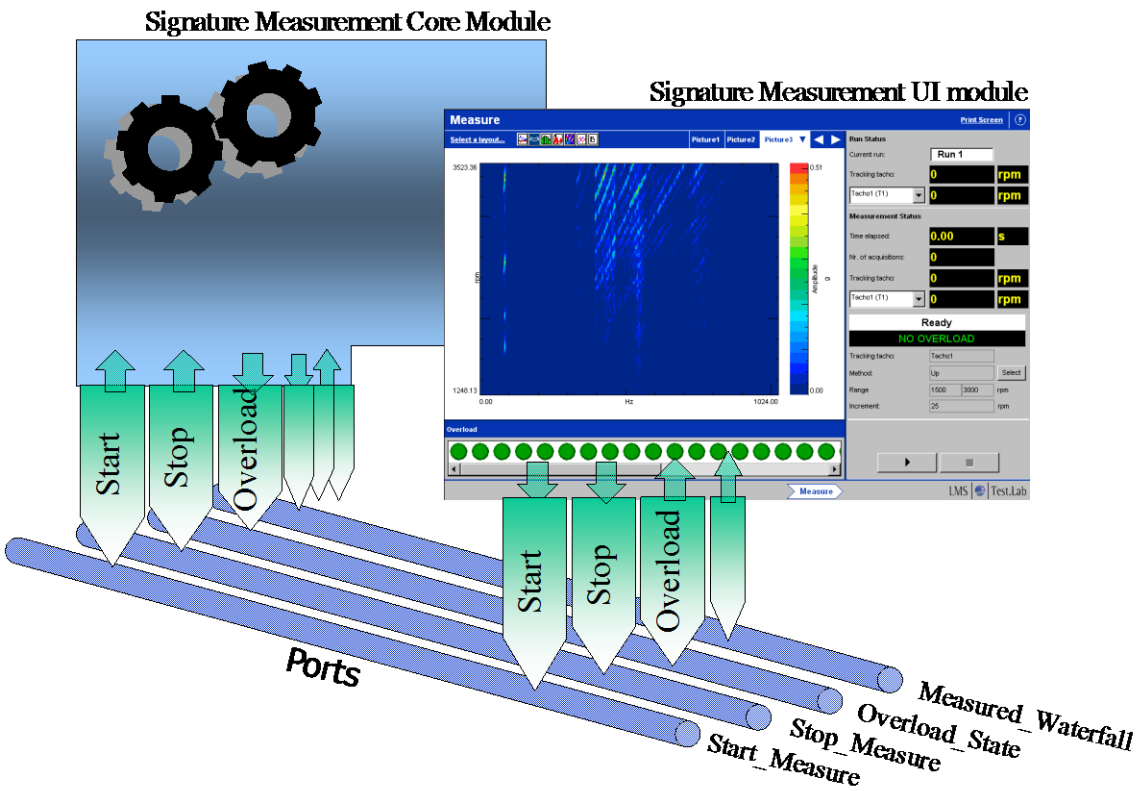
In order to explain the powerful functionality of this method, it is important to get some insight in the truly Component Based architecture of the Test.Lab™ application.

As we've seen earlier (ref Application.) all the actual application specific know-how is in the 'Application Workbook' files with the extension .awb. Each of these workbooks describes the set of modules they need to offer the total packaged functionality. Each module contributes part of the solution, but isn't conditioned to serve only one single application. Most of the internal modules can be reused in different application. The 'Documentation' and 'Navigation' modules are probably some of the most frequently reused modules that serve different applications. Also more specialized modules can serve different end-applications. The 'Run Data Averaging & Comparison Organizer' or the 'Psycho-Acoustic Metrics' module, for instance, can be used in a SignatureFixedSampling workbook or even in a SpectralAcquisistion workbook.

A Test.Lab™ module doesn't necessarily need to provide any user interface. In fact some modules merely provide the core processing, while others don't do any processing of themselves.

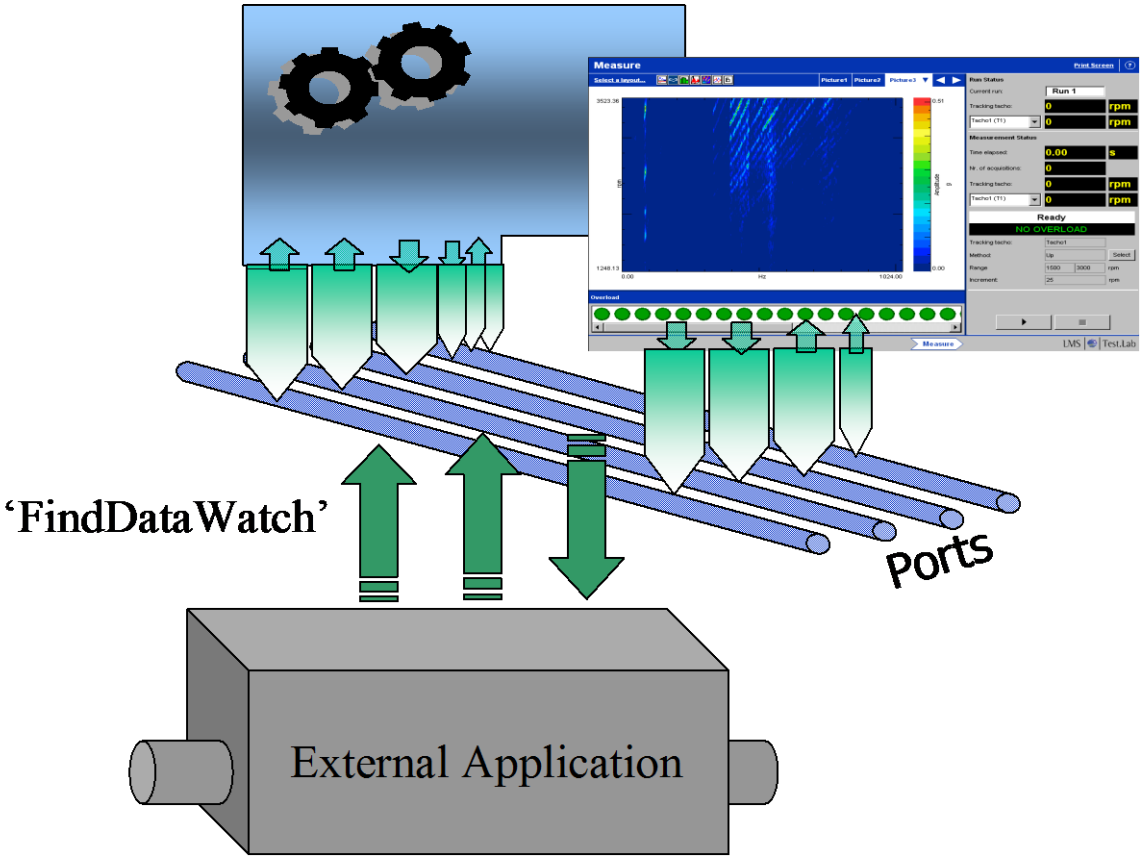
You can think of them as devices, where the user can interact by means of a front-panel to control the internal services of the device. The front-panel merely allows a user to get access to some of the internals of the device. Replacing the front-panel with a more reduced set of controls doesn't require the internal system to be changed if you 'd want it to have some fixed settings.

Next to their high-tech functionality, the seamless integration of the different modules into fit-for-purpose applications is a major asset to the Test.Lab™ applications. But in order to do this, the modules need to be able to pass information. To be able to do so, they connect to dedicated ports. These ports are a kind of data-busses, communicating data packages. The next figure schematically illustrates the internal architecture and the different ports as communication channels.



This of course is just a subset of the great number of modules, present in an actual Test.Lab™ workbook, but it indicates that the different independent modules connect to the same port if they wish to communicate on detailed information with a specific functional character, such as 'start the measurement', 'last measured data block', etc.

This 'FindDataWatch' method offers access to each of these ports.



Most of the ports are replicated for each running book. (Ref IBook Method: FindDataWatch). The 'FindDataWatch' method on the Application object level, will allow to get connected to the ports which are general to all instantiated books, so independent from the project opened.

There are, for instance, global ports holding default information about DisplayStyle properties or Navigator Sorting Type.

The method will return a DataWatch object that encapsulates a connection to one port and some additional information on the frequency with which to access it and the data returned. More information on this when elaborating on the DataWatch object. (Ref DataWatch.)

Example:

```
Dim Loaded_AddIns_Port as LMSTestLabAutomation.DataWatch

Set Loaded_AddIns_Port  = TestLab.ActiveBook.FindDataWatch("Loaded_AddIns")
```

3.1.11 Method: CreateObject

Signature:

```
CreateObject(DesiredType, Attributes)
```

Description:

This method allows a client application to create native Test.Lab™ objects such as commands, weighting types, data blocks, calculations, etc. To do so it needs a string to indicate the type of data to create, and an attribute list, a kind of parameter list, as input for creation. Please read through the description of the next interfaces and by

the time you get to the AttributeMap interface, the function of this method should be clear.

3.1.12 Method: Database

Signature:

Database(ProjectPathName : String) : IDataBase

Description:

This method allows a client application to get hold of the automation object called IDatabase for databases solely stored on disk. This interface holds all the functionality for interacting with the project database. Please refer to the description of the IDatabase interface to get a clear idea on the offered project database functionality.

Example:

Dim TL_DB As LMSTestLabAutomation.IDatabase

Set TL_DB = TestLab.Database(e:\local\data\Archive\PRJ990604.lms“”)

3.1.13 Method: DisallowExit

Signature:

DisallowExit(ac_bDisallowExit : Boolean)

Description:

This method enables a client application to get control over allowing /disallowing the interactively exiting the Test.lab application.

3.1.14 Method: CreateAttributeMap

Signature:

CreateAttributeMap()

Description:

This method creates an Attributemap object in Test.lab and returns a reference to it. Such an ‘Attributemap’ is a general implementation of a list or a map and is used for passing lists of arguments to and from several Test.lab automation commands. The dataobject itself will be discussed in 3.9

3.1.15 Property: Name

Type: Read-only

Signature:

Name() : String

Description:

This method returns the name of the opened workbook of the attached Test.lab application.

3.1.16 Property: cmd

Type: Read-only

Signature:

```
cmd() : ICmd
```

Description:

This method returns a handle to the command interpreter.

3.2 IBooks.

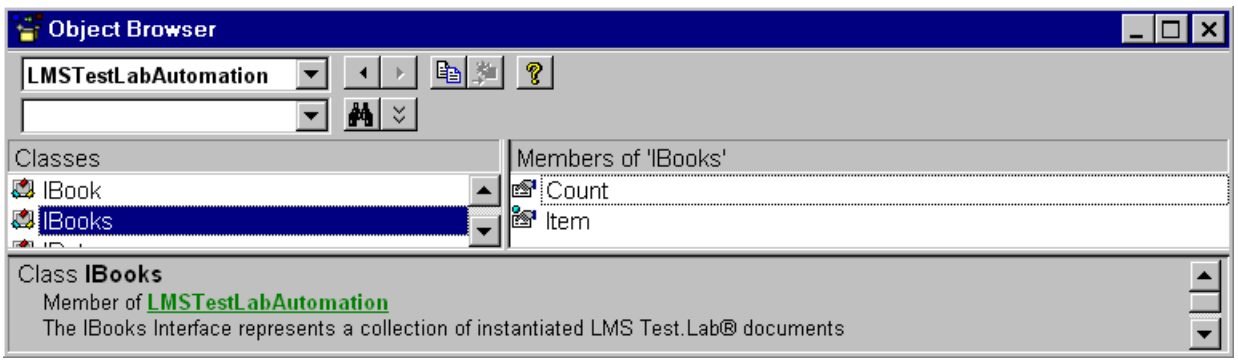


Figure 10

As indicated by Figure 8 this interface offers some functions on the list of running workbook instances. The 'IBooks' interface meets all requirements, set forth by automation, for being a collection. This means that the VB 'ForEach' construction can be used on this Test.Lab™ object as shown in next example.

```
For Each OpenedBook In TestLab.Books
  ' Do some operation on a single book
Next
```

3.2.1 Property: Count

Type: Read-only

Description:

Returns the number of books currently running in the application. This allows you to iterate all running instances.

Example:

```
Dim List_of_opened_books As LMSTestLabAutomation.IBooks
number_of_books = List_of_opened_books.Count
For loopcounter = 1 To number_of_books ...
```

3.2.2 Property: Item

Type: Read-only

Signature:

Item(Index)

Description:

Using the Item property you can retrieve a specific book. The desired book can be indicated by passing an sequence number *n* to the 'Item' property, saying give me a reference to the *n*-th book.

Another way of retrieving a book from the list of books, is by means of its unique id. Such an id is returned when creating a book. (See Method: NewProject Method: OpenProject)

Example:

```
Dim SingleBook As LMSTestLabAutomation.IBook

Set SingleBook = List_of_opened_books.Item(1)

Set SingleBook = List_of_opened_books.Item(Bookid)

Set SingleBook = List_of_opened_books.Item[1]

Set SingleBook = List_of_opened_books.[Bookid]
```

All previous examples retrieve a book from the available books' list. The last two are VB short-hands.

3.3 IBook.

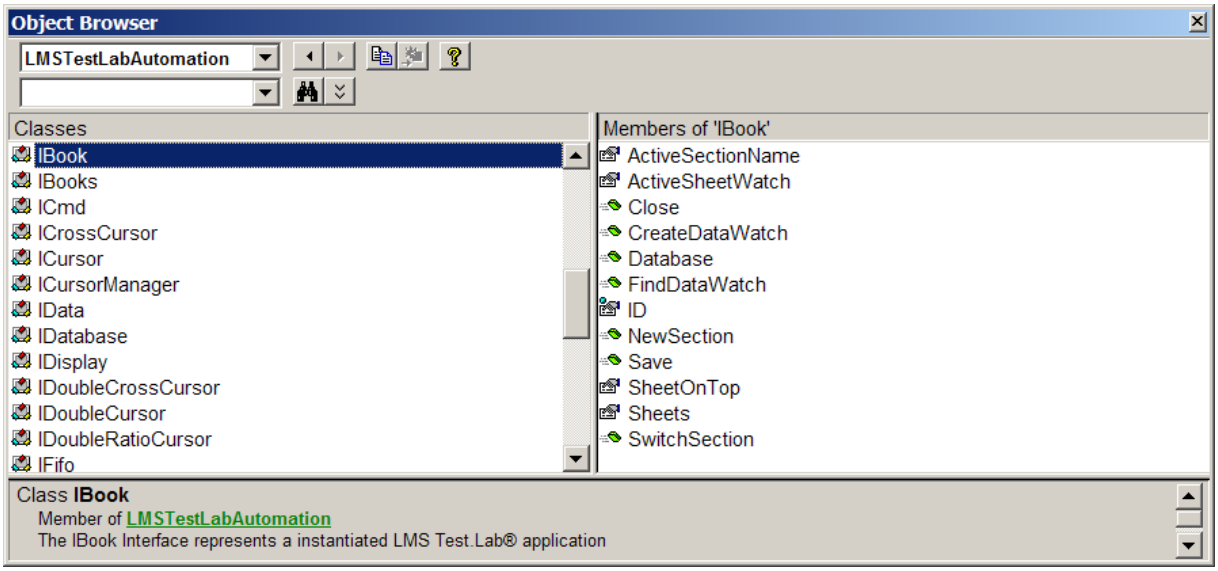


Figure 11

3.3.1 Property: ActiveSectionName

Type: Read-only

Description:

This property will hold the currently active section name for the book of which you retrieve this property.

Example:

```
Dim Selected_book As LMSTestLabAutomation.IBook
```

```
Set Selected_book  = TestLab.Books(0)

SectionName = Selected_book.ActiveSectionName
```

3.3.2 Property: ActiveSheetWatch

Type: Read-only

Description:

This will hold a Watch object that allows access to the active sheet name. The active sheet is the sheet in the active book that is on top. We'll elaborate more on 'Watch' and 'DataWatch' later on (cfr 3.6 & 3.7). For now you can think of it as a way to peek and modify the sheet that is activated.

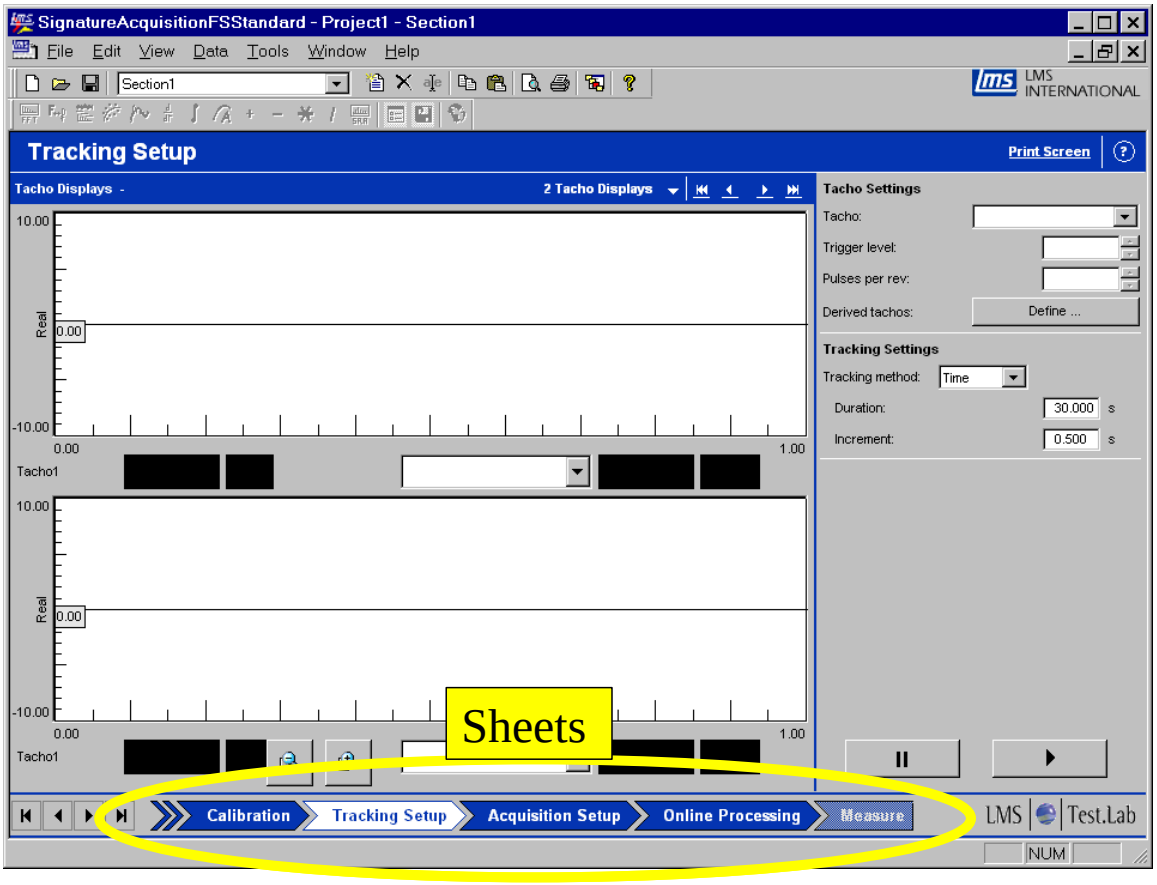


Figure 12

Example:

```
Dim ActiveSheet as LMSTestLabAutomation.Watch

Set ActiveSheet = TestLab.ActiveBook.ActiveSheetWatch

Currentsheetname = ActiveSheet.Data        'reads the currently active sheetname

ActiveSheet.Data  = "Measure"              'sets the Measure sheet on top
```

3.3.3 Property: ID

Type: Read-only

Description:

This property is a transient unique id that has been assigned to a book upon creation. Such a unique ID is never the same, so it cannot be used across different process runs. The ID of a valid running book can be used in book related operations, such as the Item method on the IBooks interface.

Example:

```
Dim Selected_book As LMSTestLabAutomation.IBook

Set Selected_book  = TestLab.ActiveBook

ActivebookID = Selected_book.ID
```

3.3.4 Property: SheetOnTop

Type: Read

Description:

From previous explanation (Method: FindDataWatch) we know that a book presents a number of sheets to the user to offer some process guidance, sequencing support, run-time add-in functionality, ... etc. Once a book is instantiated, there's always one single sheet selected and drawn as the active one. The UI content of the active sheet is the one that gets shown. The property 'SheetOnTop' can be used to retrieve the localized name of the active sheet. Setting this property however instructs the application to switch to the indicated sheet. It can be set to the localized name of the sheet or to its index. Note that the index to be used is the sheet sequence number, as it occurs in the initial list of sheets. So if a sheet was moved forward or backwards, by means of 'Tools>Workbook Configuration ...' its index still stays the same.

This method is just a subset of the functionality offered by ActiveSheetWatch, introduced in 5A.

Example:

```
Active_sheet_name  = TestLab.ActiveBook.SheetOnTop
```

Retrieves the localized name of the active sheet.

```
TestLab.ActiveBook.SheetOnTop = "Measure"
```

Brings up the sheet showing "Measure"

```
TestLab.ActiveBook.SheetOnTop = 3
```

Brings up the sheet which is in the third one in the original worbookconfiguration.

3.3.5 Property: Sheets

Type: Read-only

Description:

Just as with the collection of books, to which you got access by means of the 'Books' method on the Application object, you can get a hold of all available sheets in a single book. In other words, this 'Sheets' property will return an interface to a list of sheets.

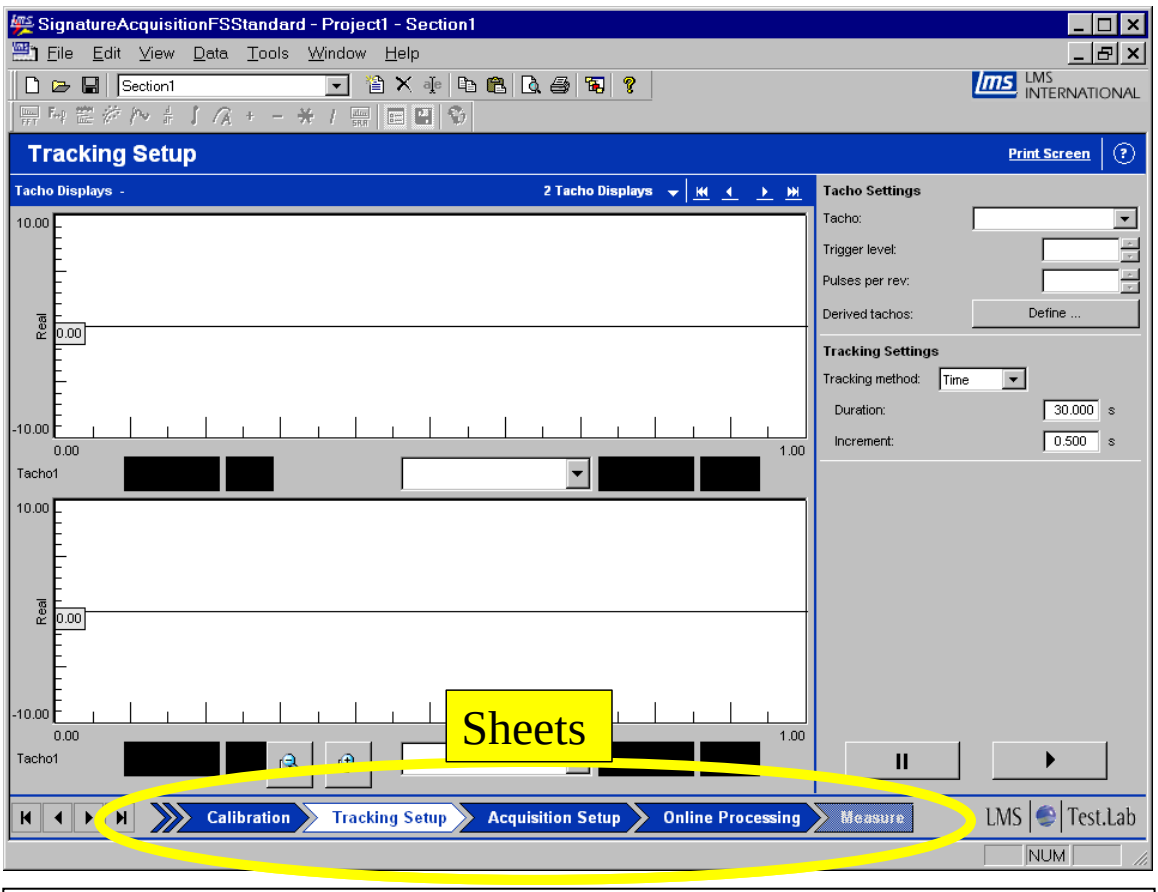


Figure 13

Example:

```
Dim List_of_sheets as LMSTestLabAutomation.ISheets

List_of_sheets = TestLab.ActiveBook.Sheets
```

3.3.6 Method: NewSection

Signature:

```
NewSection(<sectionname>)
```

Description:

Creates a new section on the project associated with this book object. This method has the same effect as if a user would go to 'Data>New Section...'.

An error status will be returned when the supplied name is the name of an existing section.

Example:

```
TestLab.OpenProject("c:/MyProject/PRJ1234.lms")
Open_book = TestLab.ActiveBook
Open_book.NewSection("MyAutomationSession")
```

Creates a new section in the open project.

3.3.7 Method: SwitchSection

Signature:

```
SwitchSection(<sectionname>)
```

Description:

Switches all settings data of the book to that in the specified section on the project associated with this book object. This method has the same effect as if a user would select a different section name in the toolbar.

An error status will be returned when the supplied name doesn't exist as section.

Example:

```
TestLab.OpenProject("c:/MyProject/PRJ1234.lms")
Open_book = TestLab.ActiveBook
Open_book.SwitchSection("Section2")
```

Switches to 'section2' in the open project.

3.3.8 Method: Save

Signature:

```
Save([filename],[compress])
```

Description:

Saves all application related information to the active project, with which the book was created.

When an explicit filename is provided as parameter, the 'Save' method will act as a 'save as'. If not, it will be saved with the same name as the one it was opened with. Should the book be created on a temporary project, e.g. with the CreateProject method, the 'Save' will return an error.

The optional <compress> parameter indicates whether the database should be compressed upon saving or not. Default value is FALSE.

Example:

```
TestLab.OpenProject("c:/MyProject/PRJ1234.lms")
Open_book = TestLab.ActiveBook
...
Open_book.Save("c:/MyProject/Updated_PRJ1234.lms")
```

3.3.9 Method: Close

Signature:

```
Close()
```

Description:

Closes the book on which the method was issued. This method will remove all runtime data from the Test.Lab™ application without saving any. If saving data is required, use the, previously described, 'Save' method on the same IBook object.

Example:

```
Dim Open_book as LMSTestLabAutomation.IBook

TestLab.OpenProject("c:/MyProject/PRJ1234.lms")

Open_book = TestLab.ActiveBook

...

Open_book.Close()
```

3.3.10 Method: FindDataWatch

Signature:

```
FindDataWatch(<General Port ID> [,MaxUpdateFrequency ] [,IsAutoBuffered])
```

General Port ID: ID of the port.

MaxUpdateFrequency: the interval, specified in milliseconds, with which the system will check the contents of the port and notify the client application if the value on the port has changed since the previous check. Default is 1000

IsAutoBuffered: a boolean value to indicate that the value returned from the datawatch needs to be returned as a buffered, normalised type.

Description:

Each project, which is opened, gets a new book instance associated. A user can have different books running, when, for instance, preparing a set-up in one while launching a measurement in another one. In such scenarios, pushing a start-measure button in one book shouldn't be communicated out of the boundaries of that one book. In order to do this, each book has it own set of ports. So a 'FindDataWatch' call for 'portA' on book1 will return a different DataWatch object then the same call on book2. The internal wiring of a Test.Lab™ Signature application, such as the one in Figure 13, is schematically drawn up in Figure 14.

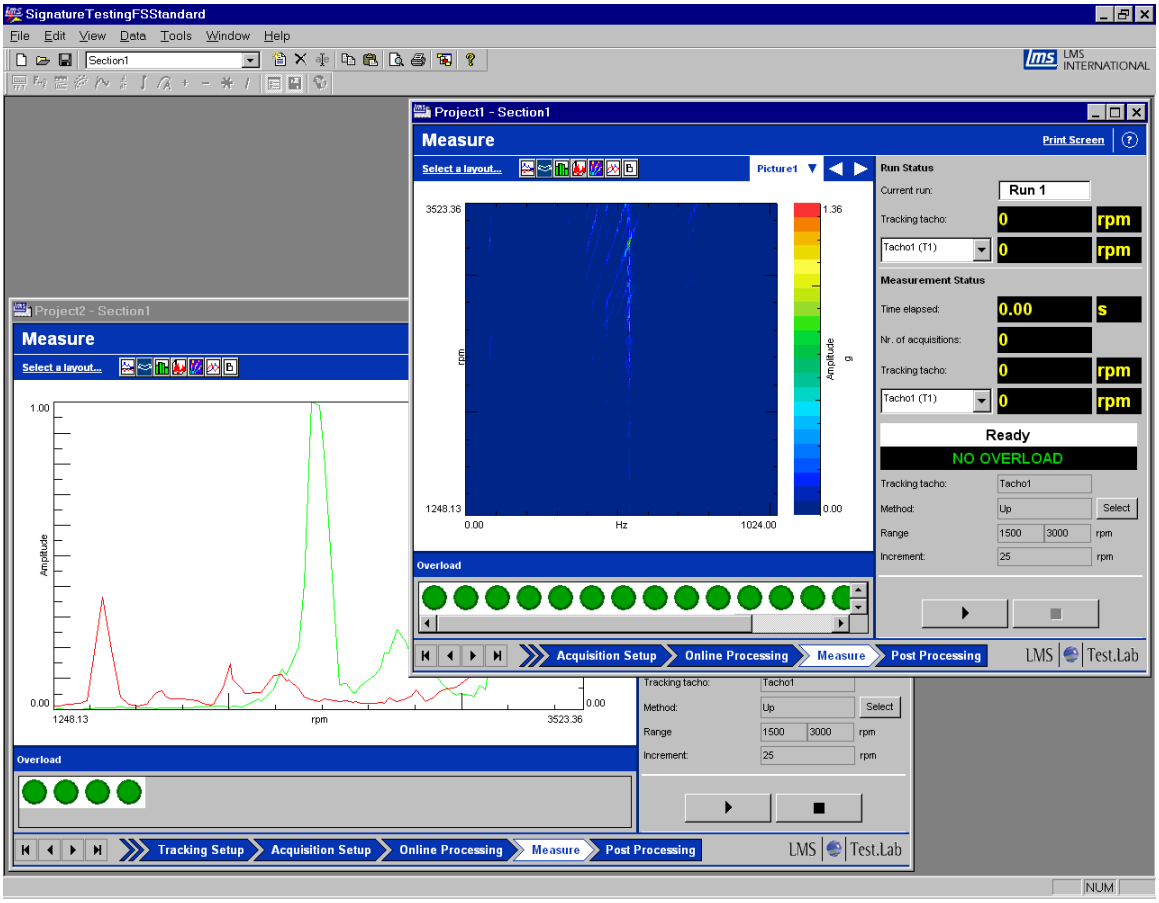


Figure 14

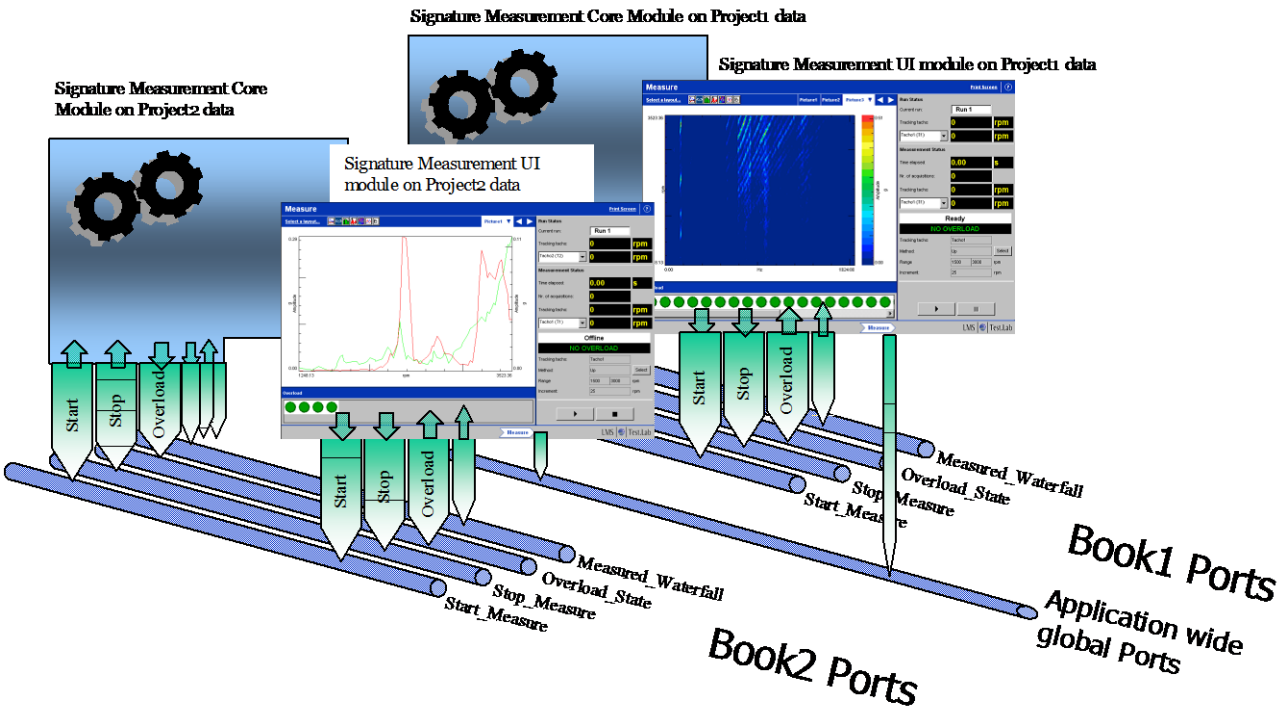


Figure 15

This last figure also indicates the difference between the ports a user can get access to on the Application level and those on the Book level. In practice there are only a few relevant global ones while there are a couple of hundred interesting ports on the IBook object. So it depends on the object you apply the 'FindDataWatch' method to. 'Application.FindDataWatch' will look for a port with the specified ID in the global ports, while 'IBook.FindDataWatch' will check the list of book-specific ports for one with the passed ID.

Example:

```
Dim Spectral_Start_Measure_Port as LMSTestLabAutomation.DataWatch

Set Spectral_Start_Measure_Port  = TestLab.ActiveBook.FindDataWatch("Spectral//Measure")
```

3.3.11 Method: CreateDataWatch

Signature:

CreateDataWatch (<General Port ID>, <DataType>,<IsPersistent> [,MaxUpdateFrequency] [,IsAutoBuffered] [,IsVisisble])

General Port ID: ID of the port.

DataType: Type of data the port can hold.

IsPersistent: Boolean value to specify if the value on the port has be saved upon saving the project.

MaxUpdateFrequency: the interval, specified in milliseconds, with which the system will check the contents of the port and notify the client application if the value on the port has changed since the previous check. Default is 1000

IsAutoBuffered: a boolean value to indicate that the value returned from the datawatch needs to be returned as a buffered, normalised type.

IsVisible: a boolean value to indicate whether the port should be visible in the 'online-data' of the Test.Lab™ GUI.

Description:

Modules communicate by means of ports. All Test.Lab™ applications implicitly define a set of ports to allow their modules to communicate. Next to these predefined ports, a user can create their own ports. Such externally created ports can be used to store data to the active section, or to drag and drop into a Test.Lab™ display. When the user creates a port at runtime, it can be found in the 'Online-Data' section in the navigator sheet. Once a port is created is treated no different from any other predefined one and can be dragged and dropped in a display for instance. Once it's dropped the display will get notified whenever the data on de port changes. Independent whether the data comes from another Test.Lab™ module or an external application.

Example:

```
Dim MyOwn_Port as LMSTestLabAutomation.DataWatch

Set MyOwn_Port  = TestLab.ActiveBook.CreateDataWatch("BreakSqual_Analysis_Result")
```

3.3.12 Method: Database

Signature:

Database() : IDataBase

Description:

This method allows a client application to get hold of the automation object called IDatabase for an active project database which is associated with an open book. This interface IDatabase encapsulates all the functionality regarding the project database. Please read through the description of the IDatabase interface to get a clear idea on the offered project database functionality.

Example:

Dim TL_DB As LMSTestLabAutomation.IDatabase

Set TL_DB = TestLab.ActiveBook.Database

3.4 **ISheets.**

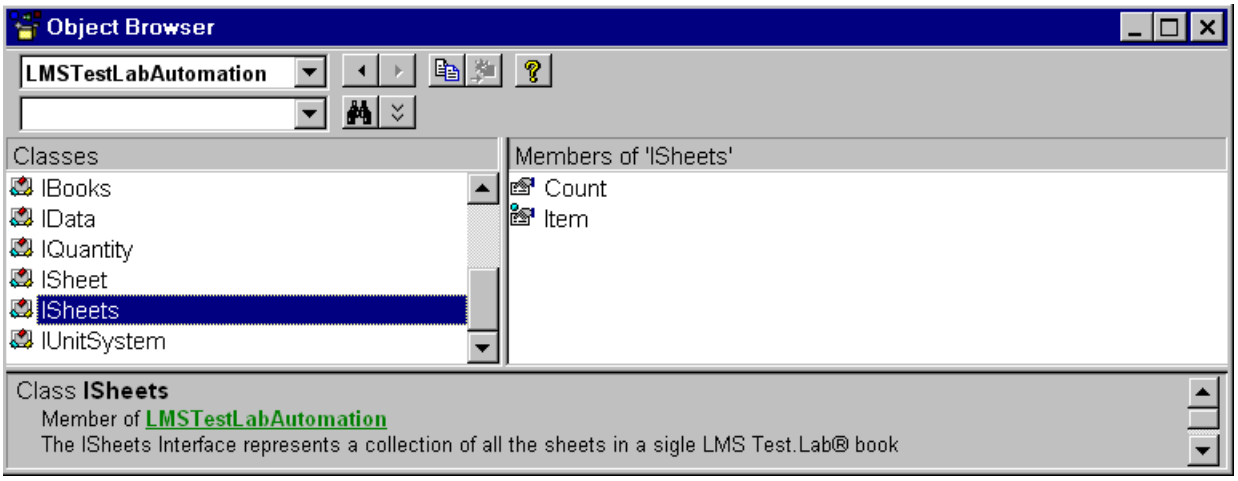


Figure 16

This interface allows to navigate the list of sheets. This list can be retrieved by means of the ISheets property on a book object. (Ref. Property: Sheets)

3.4.1 Property: Count

Type: Read-only

Description:

Returns the number of sheets in the associated book.

Example:

```
Dim List_of_sheets As LMSTestLabAutomation.ISheets
...
number_of_sheets = List_of_sheets.Count
For loopcounter = 1 To number_of_sheets ...
```

3.4.2 Property: Item

Type: Read-only

Signature:

```
Item(Index)
```

Description:

Using the Item property you can retrieve a specific sheet. The desired sheet can be indicated by passing an sequence number *n* to the 'Item' property, saying 'give me a reference to the *n*-th sheet'. The same remark has to be made as for 'SheetOnTop'. The sequence number to use is that of the sheet in it's original setting. That is without sheet order customization.

Another way of retrieving a sheet from the list of sheets, is by means of its label. This label is the one, which is visualized on the tab below. Since these labels can be localized, it's the localized string to use in the 'Item' call.

Example:

```
Dim NavigationSheet As LMSTestLabAutomation.ISheet

Set NavigationSheet = List_of_sheets.Item ("Navigator")

Set NavigationSheet = List_of_sheets.Item [1]
```

3.5 ISheet.

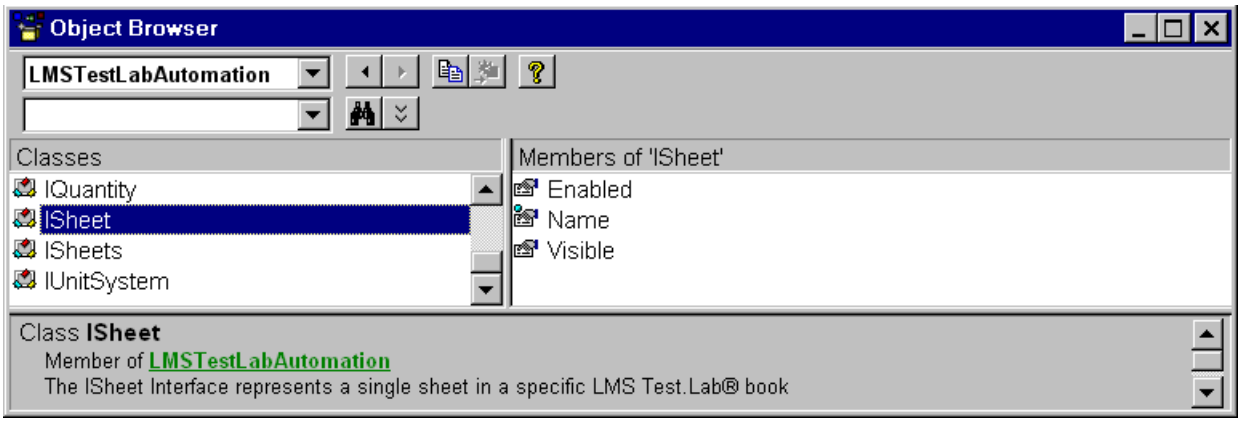


Figure 17

This interface holds a number of members to query and manipulate sheet properties.

3.5.1 Property: Enabled

Type: Read-Write

Description:

Gets or sets the 'enabled' status of a sheet. A sheet, which is disabled is greyed out and cannot be accessed interactively.

Example:

```
IsSheetEnabled  = TestLab.ActiveBook.Sheets.Item("Measure").Enabled
```

Retrieves the enabled status of the 'Measure' sheet.

```
TestLab.ActiveBook Sheets.Item("Navigator").Enabled = False
```

Disables the 'Navigator' sheet.

3.5.2 Property: Name

Type: Read-only

Description:

Gets the name of the sheet. This is the localized string you can find on the sheet tab.

Example:

```
Sheetname = TestLab.ActiveBook Sheets.Item(2).Name
```

3.5.3 Property: Visible

Type: Read-Write

Description:

Gets or sets the visibility status of a sheet. A sheet, which is set invisible, is removed from the list of visual sheets.

Example:

```
IsSheetVisible  = TestLab.ActiveBook.Sheets.Item("Measure").Visible
```

Retrieves the visibility status of the 'Measure' sheet.

```
TestLab.ActiveBook Sheets.Item("Navigator").Visible = False
```

Makes the 'Navigator' sheet invisible.

3.6 DataWatch.

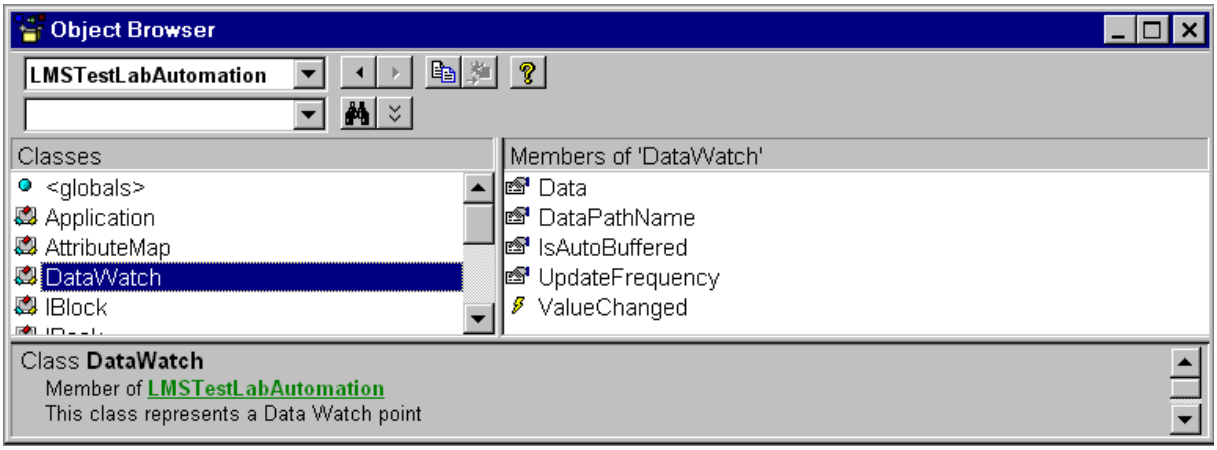


Figure 18

Any object of this DataWatch type, represents a reference to a specific port in Test.Lab™. Next to that, it holds some settings in the way you can access that port. The ‘FindDataWatch’ methods on Application and IBook allow the user to hook up to one of the application defined ports.

3.6.1 Property: Data

Type: Read-Write

Description:

Ports are the carriers that are being used to communicate specific data packages between subscribers. This property is the means to either get or set such a data package.

Getting data means the latest data package is returned to the client.

Putting data will instruct Test.Lab™ to broadcast the supplied data to other subscribers. Since each port is designed for a specific purpose, it is only able to transport data of a specific type. It's the users' responsibility to provide correctly typed data packages. There are a number of different ways in which the user can get a hold of data to put on ports.

- Data objects, gotten from another port. The types of both ports have to be compatible.
- A Test.Lab™ data object, altered by means of the 'Edit' method in the IData interface. (Ref Method: Edit)
- A Test.Lab™ data object, created with the 'CreateObject' method in the Application interface. (Ref Method: CreateObject)
- A non Test.Lab™ native data object, which fully supports the automation interfaces as laid out in the Test.Lab™ object model. E.g. VB class supporting the IBlock interface.

Example:

```
Public port_to_override As LMSTestLabAutomation.DataWatch

Set port_to_override = testlab.ActiveBook.FindDataWatch("Post Processing//Weighting")

Dim NewWeightingValue As LMSTestLabAutomation.IData

Dim attributes As LMSTestLabAutomation.AttributeMap

Set attributes = CreateObject("LMSTestLabAutomation.AttributeMap")

attributes.Add "EnumWeighting", 5

Set NewWeightingValue = testlab.CreateObject("LmsHq::DataModelI::DataAttributes::CBufferIEnumWeighting", attributes)

port_to_override.Data = NewWeightingValue
```

Previous example shows how to override the weighting setting.

3.6.2 Property: DataPathName

Type: Read-Only

Description:

Returns the ID of the port the DataWatch represents. This is the ID which was specified in the 'FindDataWatch' method that returned the DataWatch object in the first place.

Example:

```
PortID = port_to_override.DataPathName
```

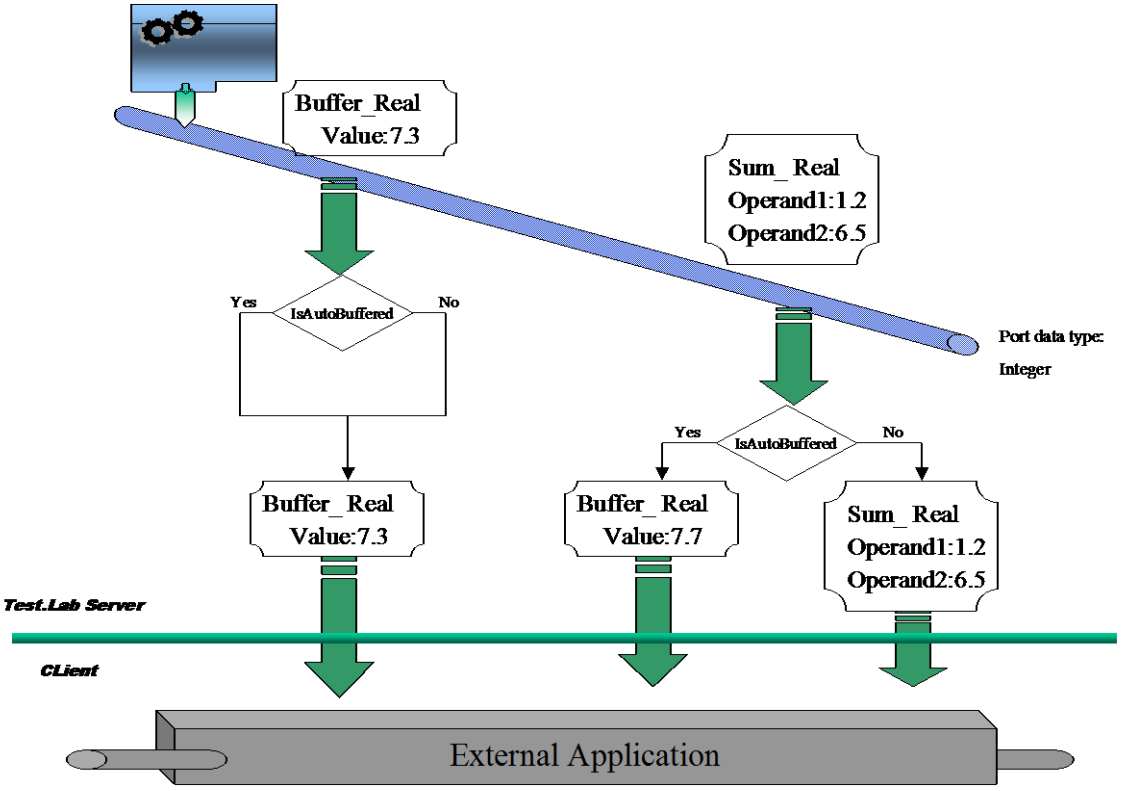
3.6.3 Property: IsAutoBuffered

Type: Read-Write

Description:

Returns a boolean value to indicate whether the the data on the port needs to be converted into a standardized data type, before handing it over to the client.

Suppose we have a port, which can transfer real values. Such a data object could be of type 'TestLab_Real', holding the value 7.3. Not always do such a port communicate calculated, self-contained values such as 7.3, but in some cases it will hold recipes, resulting in a real value, but only after evaluating the recipe. Lets say this recipe would be of type 'TestLab_RealAdd', and hold two other values e.g. 1.2 and 6.5. Evaluating the 'TestLab_RealAdd' would result in the sum of both values, being 7.7, also a real value. So even if the port can only hold real values, the actual data types passed, can differ, depending on the recipe they represent. In order to avoid having to check for all kinds of different 'recipes' passing on a single port, the user can request to have the data converted to a predefined buffer-type before handing it over to the client application. The next figure points out the meaning of the 'IsBuffered' flag, which can be passed in the 'FindDataWatch' method. The default setting value for this parameter is to return the buffered-type data.



In the previous scenario, the application holds a port of type real. Such a Test.Lab™ real can hold a calculated value, as in the case of Buffer_Real, or it can hold a recipe to calculate an real value, e.g. Sum_Real. If you turn 'IsAutoBuffered' off, you get the raw object types as they pass on the port. If it is turned on, the recipe is evaluated and turned into a buffered form, which contains the result. That way the client application doesn't have to take all different possible Test.Lab™ object types into account.

Example:

```
Set Spectral_Start_Measure_Port  = TestLab.ActiveBook.FindDataWatch("Spectral//Measure",5000,True)

...

Does_this_datawatch_have_autobuffering_on = Spectral_Start_Measure_Port.IsAutoBuffered
```

3.6.4 Event: ValueChanged

Signature:

```
Private Sub <DataWatchName>_ValueChanged(ByVal NewValue As Variant)
```

Description:

Whereas properties and methods define a synchronous communication between a client and server application, the automation 'Event' mechanism provides a way to let the server notify a client application of a specific happening.

The 'ValueChanged' event, defined on the DataWatch object, allows a client to register a call-back routine. This routine, on the client side, will be called whenever data changes on the specified port. The frequency, with which to check for changes, can be configured by means of the 'UpdateFrequency' parameter in the 'FindDataWatch' method. A 'Data' property on a datawatch object returns the instantanious data on the associated port, whereas an event routine will be called whenever the data on the port changes.

Example:

```
Private Sub SignatureFS_Measurement_Ended_Port_ValueChanged(ByVal NewValue As Variant)

If NewValue = cMeasurementEnded Then

   Response = MsgBox("Positioning Laser. Continue?", vbYesNo, "Measurement Automation")

    If Response = vbYes Then

         Call Relaunch_Measurement

     End If

End If

End Sub
```

3.6.5 Property: UpdateFrequency

Type: Read-Only

Description:

Returns the time interval used for event notification. If no event routine is registered this value is irrelevant. If event notification is required, this update frequency in milliseconds will specify the rate at which Test.Lab™ will check for new values on the port, identified by the datawatch object. This 'UpdateFrequency' is the worst case latency before getting notified in the client application. The smaller the value, the more frequent Test.Lab™ will check for changes, thus consuming more the server CPU time, but communicating the changes quicker to the client application.

Example:

```
Set rpm_value_datawatch  = TestLab.ActiveBook.FindDataWatch("Signature//measure//rpmvalue ",250,True)

Rpm_value_update_freq = rpm_value_datawatch.UpdateFrequency
```

Have Test.Lab™ check the 'rpm_value' port every quarter of a second for new values and call the "rpm_value_datawatch_ValueChanged" event routine if anything has changed.

```
Set measurement_ended_datawatch  = TestLab.ActiveBook.FindDataWatch("Spectral//Measurement_state",2000,True)

measurement_ended_ update_freq = measurement_ended_datawatch.UpdateFrequency
```

Have Test.Lab™ check the 'rpm_value' port every quarter of a second for new values and call the "rpm_value_datawatch_ValueChanged" event routine if anything has changed.

3.7 Watch

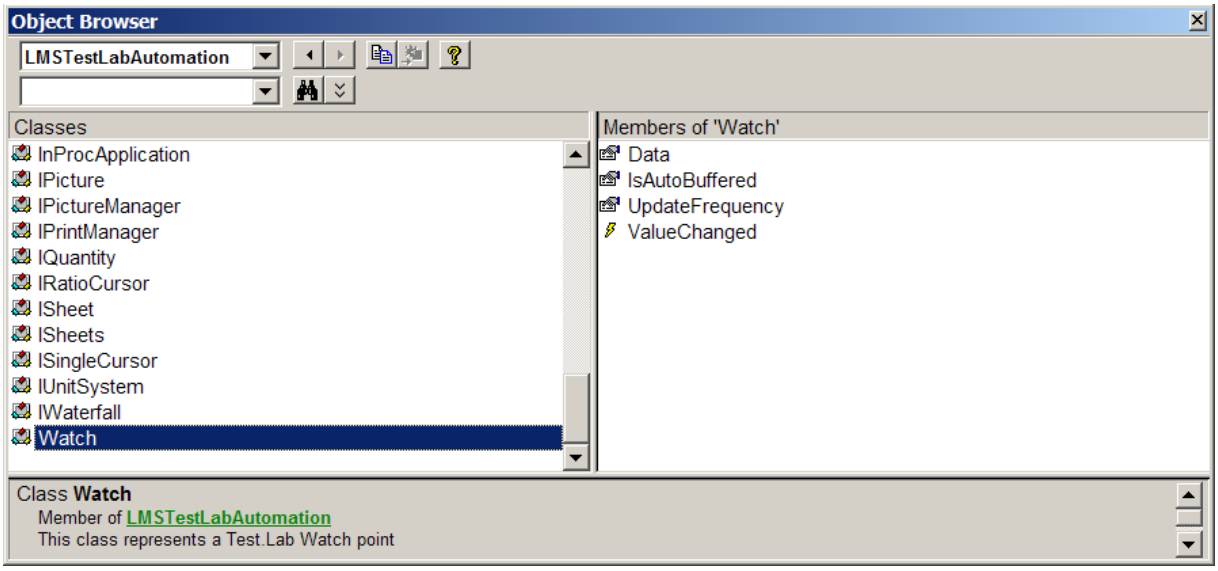


Figure 18

Whereas a DataWatch is a handle to a communication port between modules, a Watch object is a handle to an internal port. You can explicitly query and retrieve a DataWatch with a specific name, but not a Watch. A watch object can only be provided by some dedicated function, allowing you to intervene with internal ports. The ‘ActiveSheetWatch’ method on IBook, for instance, hands a reference to an internal port that controls the active sheet for that book instance. The only difference between both objects is the ‘Pathname’ property. Since it is not a published port, you will not be able to retrieve its pathname, by which it can be located.

All other methods and properties are identical to that of ‘DataWatch’ (cfr 3.6)

3.8 IData.

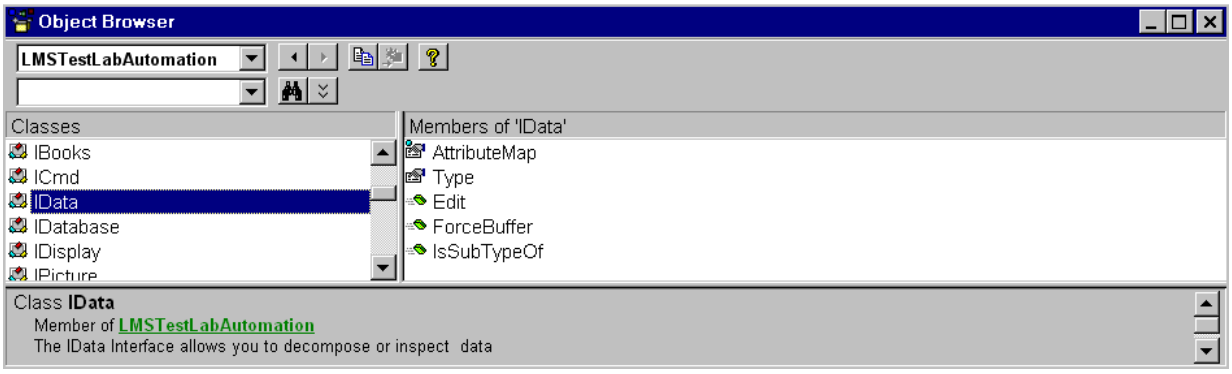


Figure 19

As described earlier, there are a number of Interfaces, methods, properties and events that can give a client application access to Test.Lab™ internal data structures. There are numerous different data types, ranging from simple types such as integers, strings, real values to complex composite structures, such as measured data blocks, waterfalls or geometries. Any complex Test.Lab™ data structure is returned and wrapped in an IData object.

3.8.1 Property: Type

Type: Read-Only

Description:

This property returns a string value, indicating the type of the data in the IData object. Client applications can check this type string to determine the way the data object can be queried.

Example:

```
Set display_style_port  = TestLab.ActiveBook.FindDataWatch("Signature//measure//Colormap//Style" )

Dim style as LMSTestLabAutomation.IData

Set style = display_style_port.Data

If  style.Type = "LmsHq::DataModelI::Display2D::CBufferIDislayStyle"

...
```

Gets the instantaneous data from the " Signature//measure//Colormap//Style" port and checks the type.

3.8.2 Method: IsSubtypeOf

Signature:

```
IsSubTypeOf(PresumedBaseType)
```

Description:

This method can be used to check the relation between different native Test.Lab™ types. All types have their place within one single hierarchical tree structure. So all recipes, which will evaluate into an integer, will be subtypes of the Test.Lab™ Integer type. All recipes, which will evaluate to a data block (e.g. weighting, averaging, ...), wil be subtypes of the Test.Lab™ Block type, and so on. A user can use the 'IsSubtypeOf' method for error handling or to verify the compatibility of different data objects.

Example:

```
Set port1_data = datawatch1.Data

If  port1_data.IsSubTypeOf("LmsHq::DataModelI::Expression::IBlock") = True

Set MyOwn_BlockPort  = TestLab.ActiveBook.CreateDataWatch("LmsHq::DataModelI::Expression::IBlock ")

Set MyOwn_BlockPort.Data = port1_data
```

The type of port1_data is checked to be a subtype of the Test.Lab™ IBlock, before putting that data object on a port of type IBlock.

3.8.3 Property: AttributeMap

Type: Read-Only

Description:

An IData object, no matter how simple or complex the contained data might be, can be decomposed into its constructing parts. This can be used recursively to allow navigation and access to individual values in an object.

Lets compare this to decomposing a car into more detailed objects.

Figure 20

Even though you don't have immediate access to every valve of the car, it is possible to get there by means of decomposition. One of the main attributes in the car is the engine. In turn this engine has a number of valves. This way you can get to the third one, for instance, and check upon some of its characteristics.

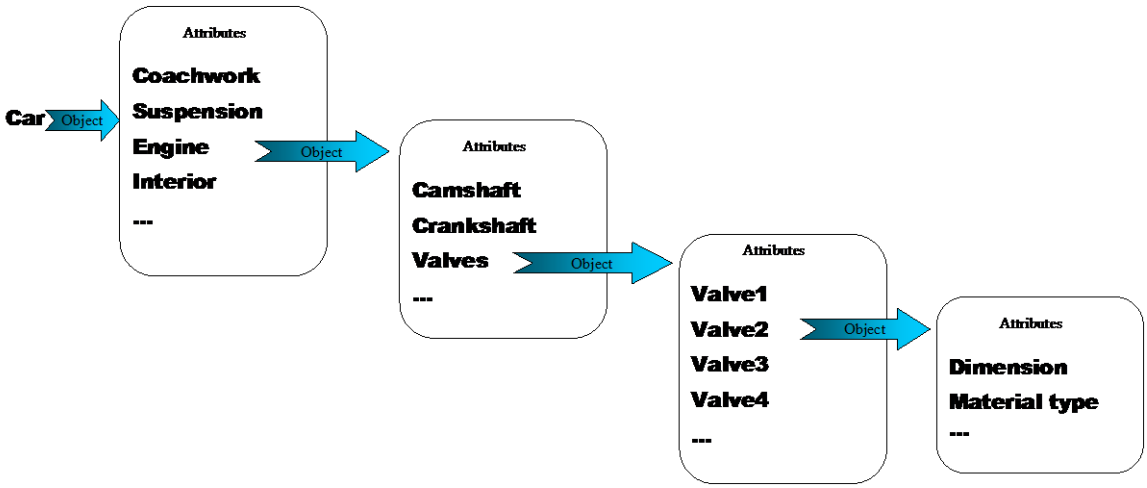


Figure 21

A client application can use the same methodology to navigate a Test.Lab™ data structure. A 'CBufferIDisplayStyle' object, for instance, could be navigated like this to get to a color setting.

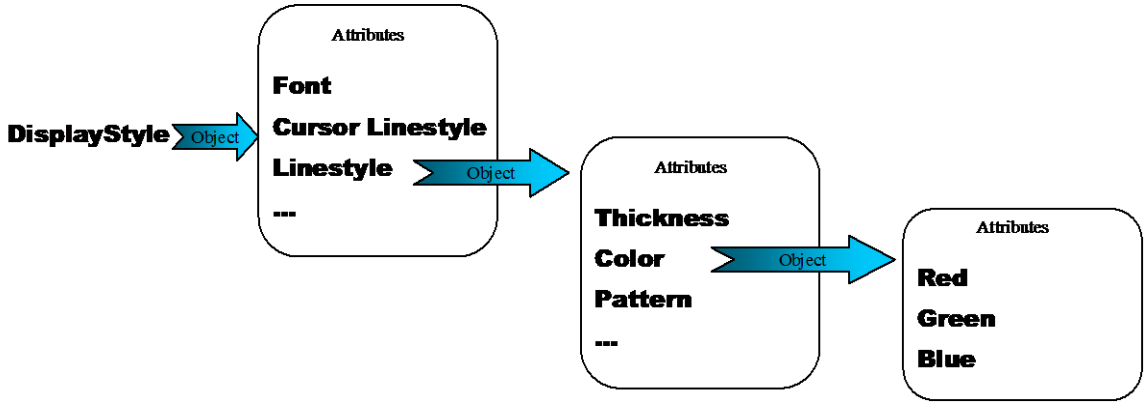


Figure 22

Such a single level of attributes is returned in an AttributeMap object. The 'AttributeMap' property returns a reference to such an object.

Example:

```
Dim attribute_list as LMSTestLabAutomation.AttributeMap

Set attribute_list = display_style_data.AttributeMap
```

Returns an attributemap, holding all the different attributes the 'display_style_data' is made up of.

3.8.4 Method: Edit

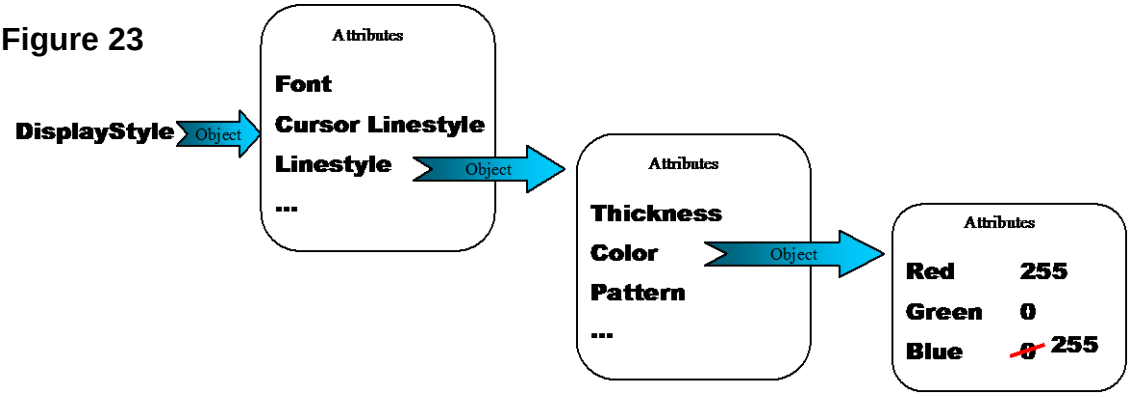
Signature:

```
Edit(PathWithinObject, NewAttribute)
```

Description:

In order to prevent corrupting internal Test.Lab™ structures, it is prohibited to modify existing objects. So how can you alter an object or make some changes to a Test.Lab™ object, before re-entering it into Test.Lab™? You can make a new one, re-using most of the attributes of the original object. This 'Edit' method helps doing so. If we take our DisplayStyle object, of previous example, we could use the method to replace only a single value in the hierarchy of attributes. All the 'Edit' method needs is a valid path to navigate the attributes and the value you want it to replace at the end, in order to return a newly constructed object of the same type.

Figure 23



Example:

```
Set purple_lined_style = display_style_data.Edit ("Linestyle/Color/Blue",255)
```

Creates a new DisplayStyle type object based on the previous one but with only one attribute being replaced.

3.8.5 Method: ForceBuffer

Signature:

```
ForceBuffer()
```

Description:

As mentioned in one of the 'DataWatch' properties (cfr 3.6.3), there are a few ways in which you can get a reference to 'raw', unprocessed Test.Lab data. This means that the data is still in its 'recipe' form, holding all the components it needs to do a calculation. If we think of the Sum_Real Test.Lab data object mentioned in the explanation 3.6.3, we could get this from a port or even create the object by means of the 'CreateObject' method in the Application object (cfr 3.1.11). The ForceBuffer method will force the object to be evaluated and converted to its normalized buffered form, containing the result. So this method will allow to force the execution of calculations.

Example:

```
Dim SymbolicCalc as LMSTestLabAutomation.IData

Dim ArgList as LMSTestLabAutomation.AttributeMap

ArgList.Add ("Operand1",1.2)

ArgList.Add("Operatnd2",6.5)

Set SymbolicCalc = TestLab.CreateObject("LmsHq::DataModelI::Base::Real_Sum",ArgList)

Set Result = SymbolicCalc.ForceBuffer()
```

Creates a fictive recipe Real_Sum and forces the calculation to be executed.

3.9 AttributeMap.

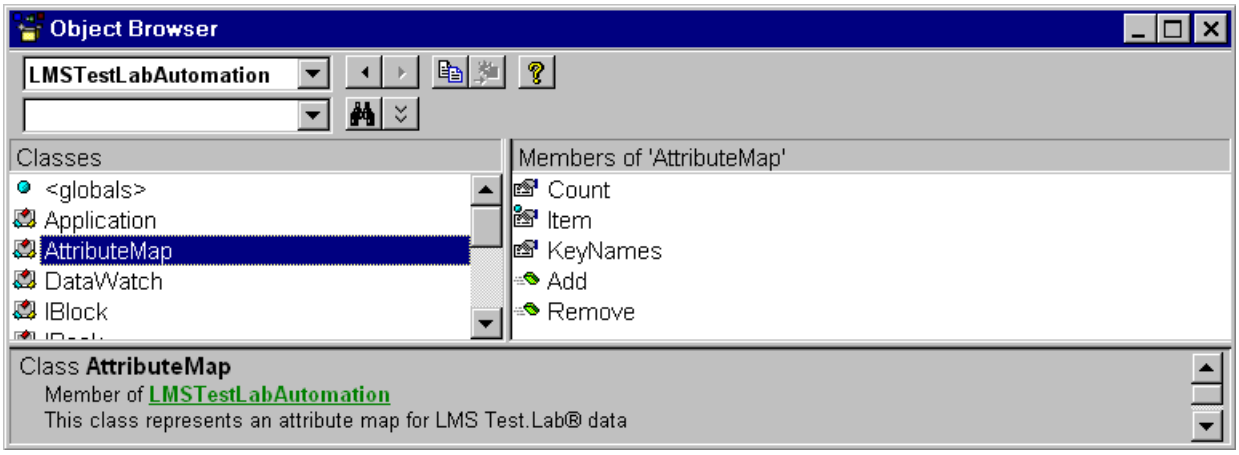


Figure 24

An AttributeMap object is a list of name-value pairs. The names are strings while the values associated can be any Test.Lab™ IData object.

These attributemaps are being used for decomposition and composition. In case of decomposition, it is an attribute list representing an existing object, while in the case of composition it is used as input to create a new object.

The 'IData.Edit' method, discussed in previous section, can return a new IData object, based on an existing one. It is also possible to create a Test.Lab™ IData object from scratch by means of the 'CreateObject' method on the Application object. In order to do so, a list of attributes has to be passed, along with a string indicating the required type of object.

3.9.1 Property: Count

Type: Read-Only

Description:

Returns the number of attributes in the map object.

Example:

```
Number_of_attributes_in_object = attribute_map.Count
```

3.9.2 Property: Item

Type: Read-Only

Signature:

```
Add(Index)
```

Description:

Returns the value associated with a specific attribute in the map. The index specified can be an zero-based sequence number or a string, in which case it'll return the value associated with the matching key.

Example:

```
Set attribute_value = attribute_map.Item (1)

Set attribute_value = attribute_map.Item ("LineStyle")
```

3.9.3 Property: KeyNames

Type: Read-Only

Description:

Returns an AttributeMap object, which only holds the names of the original attribute map, but this time as values. This way the list can be iterated by means of the Count and Item property to consult all available key names.

Example:

```
'should the attribute_map contain following pairs:

'        keyname: "foreground"        value:<value_object_1

'        keyname: "background"        value:<value_object_2

'        keyname: "font"              value:<value_object_3

Set keyname_list = attribute_map.KeyNames ()
```

```
'keyname_list contains following pairs:
'       sequencenr: 1           value:"foreground"
'       sequencenr: 2           value:"background"
'       sequencenr: 3           value:"font"
For Each keyname In keyname_list
        'keyname will contain "foreground", "background" and "font" in turn.
Next
```

3.9.4 Method: Add

Signature:

```
Add(Index, ac_Item)
```

Description:

Adds an attribute to the attributemap. This attribute is added with a keyname 'Index' and value 'ac_Item'.

Example:

```
Dim new_attributes as LMSTestLabAutomation.AttributeMap
Set new_attributes = CreateObject("LMSTestLabAutomation.AttributeMap")
new_attributes.Add("LineStyle", local_line_style)
```

Creates an empty attributemap and adds an attribute value under the name of "LineStyle".

3.9.5 Property: Remove

Signature:

```
Remove (Index)
```

Description:

Removes an attribute from the AttributeMap. If Index is a string, it will remove the associated entry from the map. If Index is an integer value *n*, it will remove the *n*-th occurrence.

Example:

```
attributes.Remove("LineStyle")
attributes.Remove(1)
```

3.10 IDatabase.

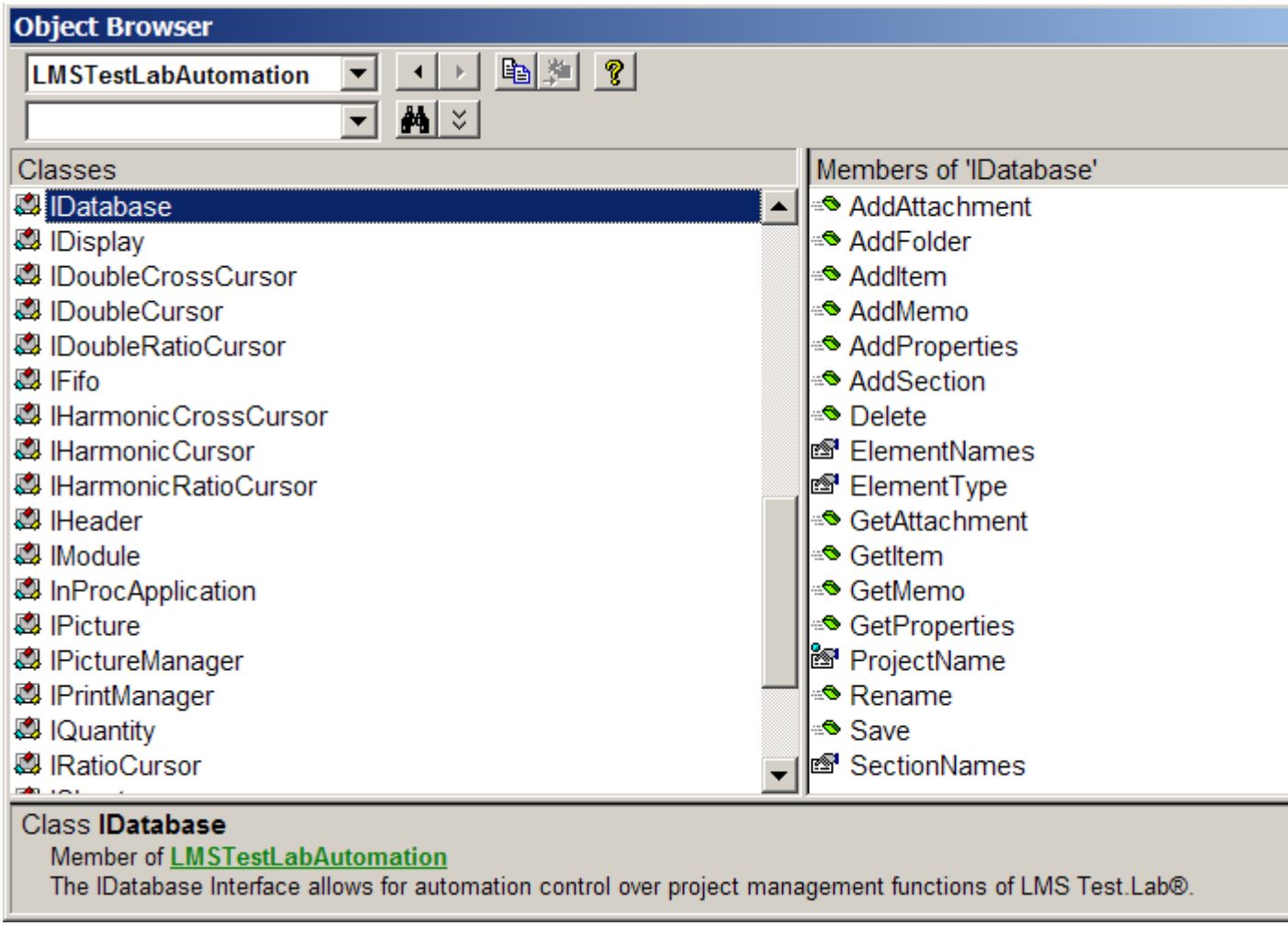


Figure 25

The automation object called IDatabase encapsulates all the functionality regarding the project database. This interface allows you to interact with 'offline' databases stored on disk and those, which are 'online' that is open in a workbook. There are two ways of getting a hold of such an object.

- Explicitily refer a database on the file system.

  Dim TL_DB As LMSTestLabAutomation.IDatabase

  Set TL_DB = TestLab.Database(e:\local\data\Archive\PRJ990604.lms“”)

- Refer to an active project associated with an open book.

  Dim TL_DB As LMSTestLabAutomation.IDatabase

  Set TL_DB = TestLab.ActiveBook.Database

A Test.Lab Project Database is a hierarchical structure, which contains two kinds of hierarchical elements, being 'folders' and 'items'. An item is a hierarchical endpoint, which contains binary system or user information, while a folder is a hierarchical

structure, which in turn can contain hierarchical elements. This can be compared with the representation of any normal filesystem, thus consisting of directories or folders and files or items.

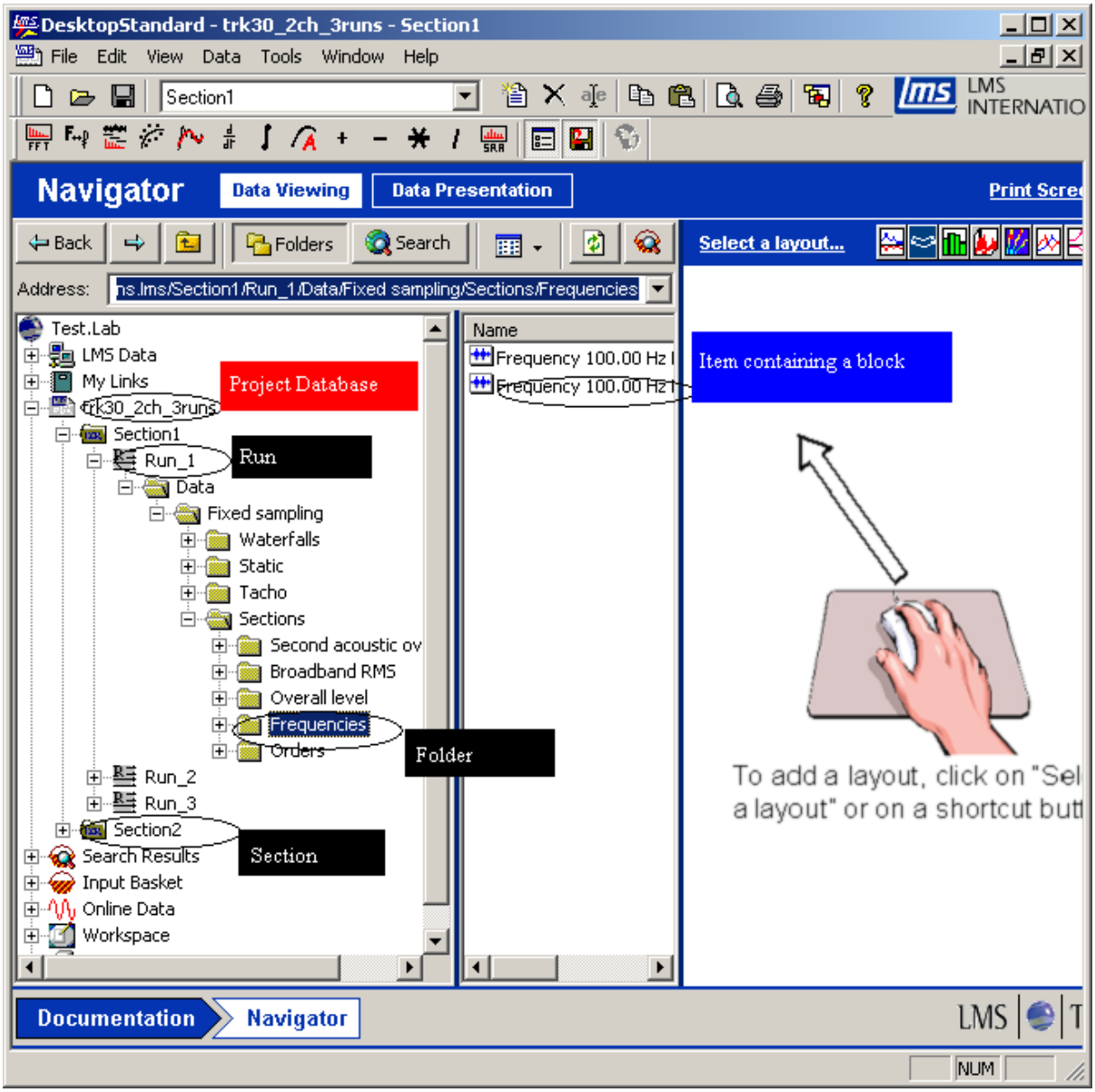


Figure 26

Let's take the tree view on the 'Navigator' sheet in a Test.Lab application to inspect the Project Database. (cfr. Figure 26) The red text box points out to the root of the hierarchical structure of the project database associated with this book. The black text boxes refer to some of the different folders. Next to general folders such as the “frequencies” there are two predefined type of folders. First of all there's the folder that holds the section information (e.g. “Section2” in the example above) and secondly there's the 'Run' folder, which can hold some snapshot information associated with one single run.

The blue text box represents an item, containing any type of data.

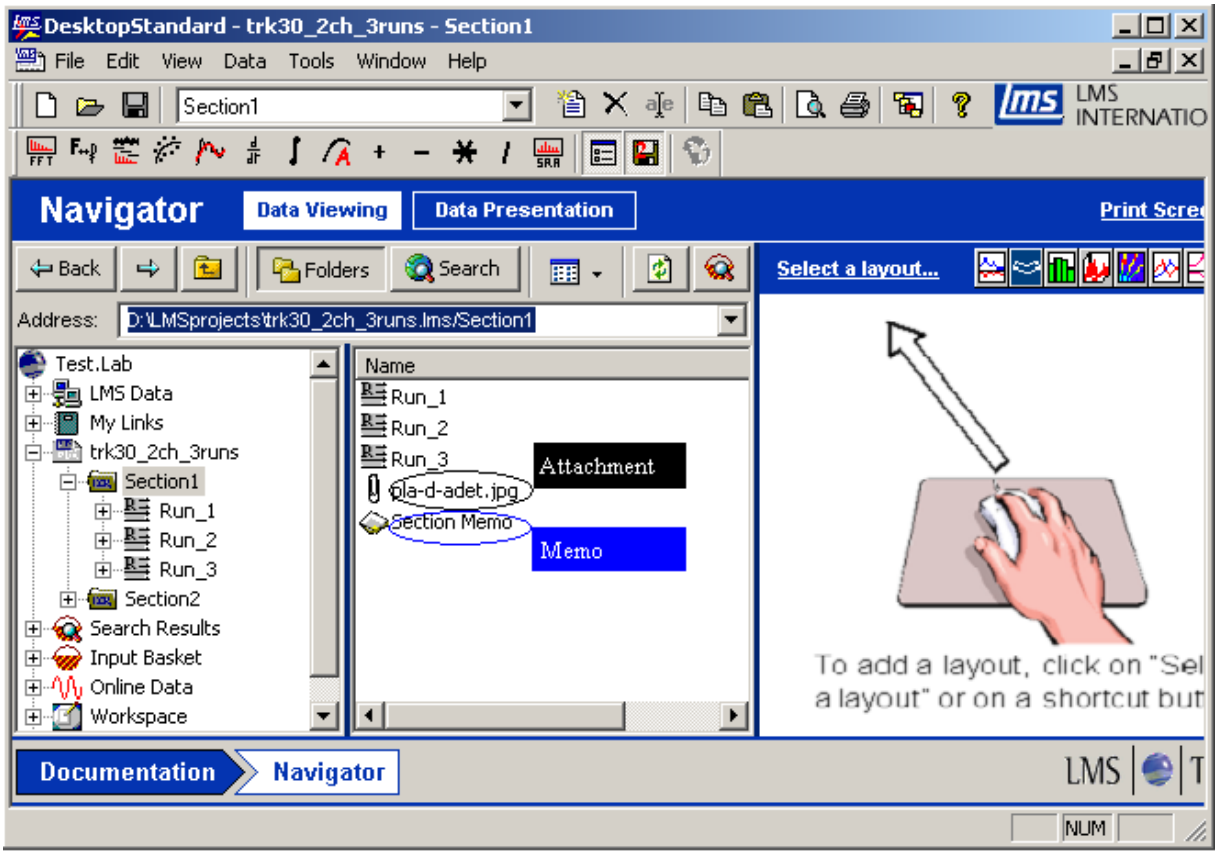


Figure 27

The Test.Lab project database can also embed user informational items like an attachment or a memo. The above figure shows the visual representation of an attachment and a memo.

3.10.1 Property: ProjectName

Type: ReadOnly

Signature:

ProjectName : String

Description:

Returns the full path name, including the name of the project itself.

3.10.2 Property: ElementNames

Type: ReadOnly

Signature:

ElementNames(PathToTop : String) : AttributeMap

Description:

Returns the list of element names contained in a folder, pointed to by the PathToTop, in the database hierarchical structure. An error will be returned if the PathToTop is invalid or if the pointed element is not an hierarchical folder.

3.10.3 Property: SectionNames

Type: ReadOnly

Signature:

```
SectionNames : AttributeMap
```

Description:

Returns the list of section names in the IDatabase.

3.10.4 Property: ElementType

Type: ReadOnly

Signature:

```
ElementType(PathToTop : String) : String
```

Description:

Returns the type of an database element on the hierarchical level as string. An error will be returned if there is no item on the path specified.

“section”, “block”, “waterfall” , “run”, etc are examples of possible database element types.

3.10.5 Method: GetItem

Signature:

```
GetItem(PathToTop : String) : IData
```

Description:

Retrieves the value entity stored in a Test.Lab database on the hierachical path specified by PathToTop. An error will be returned if the PathToTop is invalid or if the pointed element is not an hierarchical endpoint.

3.10.6 Method: GetAttachment

Signature:

```
GetAttachment(PathToTop : String) : String
```

Description:

Retrieves the attachment stored in a Test.Lab database on the hierarchical path specified by PathToTop. This method returns the pathname to a temporary file, where the attachment content has been expanded. An error is returned if the PathToTop is invalid or if the pointed element does not contain an attachment.

3.10.7 Method: GetMemo

Signature:

```
GetMemo(PathToTop : String) : String
```

Description:

Retrieves the memo stored in a Test.Lab database on the hierarchical path specified by PathToTop. This method returns the pathname to a temporary file, where the

memo content has been expanded. An error is returned if the PathToTop is invalid or if the pointed element does not contain an memo.

3.10.8 Method: AddItem

Signature:

AddItem(PathToTop : String, Name : String, Value : IData [,PropertyContainer : AttributeMap] [,Override : Boolean = false])

Description:

Adds a data entity to the database project on the given PathToTop. The default is not to override existing items. In this case an error is returned that there was an attempt to do so. An error is returned if the PathToTop is invalid.

3.10.9 Method: AddFolder

Signature:

AddFolder(PathToTop : String, Name : String [,PropertyContainer : AttributeMap])

Description:

Adds a folder entry to the database project on the given PathToTop. An error is returned if there's already a folder on the specified path.

3.10.10 Method: AddSection

Signature:

AddSection (NewName : String)

Description:

Adds a section to the database in a Test.Lab application independent way, there will be no settings copied to the newly created section. For creating new sections and having them taking over settings of a previous section, you can use 'NewSection' on IBook for any specific workbook, since it is part of the knowledge of each application.

3.10.11 Method: AddAttachment

Signature:

AddAttachement(PathToTop : String, AttachmentName : String, AttachmentFilePath : String)

Description:

Embeds the contents of any file as an attachment item on the hierarchical path in the database. An error is returned if there's already such an item on the specified path.

3.10.12 Method: AddMemo

Signature:

AddMemo(PathToTop : String [, MemoName : String] [, MemoFilePath : String])

Description:

Embeds the contents of any file in an item on the hierarchical path in the database as a memo item. The MemoName and MemoFilePath are optional. In this case the

predefined memo file of the memoinventory is used. An error is returned if there's already such an item on the specified path.

3.10.13 Method: Delete

Signature:

```
Delete(PathToTop : String)
```

Description:

Deletes any type of element (folder or data entity) in the database hierarchy specified by path. An error is returned if there is no item on the path specified.

3.10.14 Method: Rename

Signature:

```
Rename(PathToTop : String, NewName : String)
```

Description:

Renames an element on the hierarchical path to the root of the database and changes the name to NewName. An error is returned if there is no item on the path specified.

3.10.15 Method: Save

Signature:

```
Save()
```

Description:

Saves the Database object and its changes to disc.

3.10.16 Method: GetProperties

Signature:

```
GetProperties(PathToTop : String) :  AttributeMap
```

Description:

Returns the complete propertyset associated with a database element. An error is returned if there is no item on the path specified.

3.10.17 Method: AddProperties

Signature:

```
AddProperties(PathToTop : String, PropertyContainer : AttributeMap [, OverrideIfExists : Boolean = true])
```

Description:

Adds properties to the existing propertyset associated with the database element. Existing properties will be overriden depending on the setting of the OverrideIfExists flag. The default value is true. An error is returned if there is no item on the path specified.

3.11 ICmd.

This interface provides access to the list of mathematical functions, which you can perform on different types of Test.Lab data. Since the list of functions will be extended regularly we suggest you'd refer to the online 'Automation documentation' viewer. Upon installation of the Test.Lab software, this viewer will be copied to your system to Program Files/LMS/Lms Test.Lab <rev>/Automation Documentation. This viewer offers you, amongst other information, an overview of all available calculations. The object browser offers you an alternative view on the list of mathematical commands.

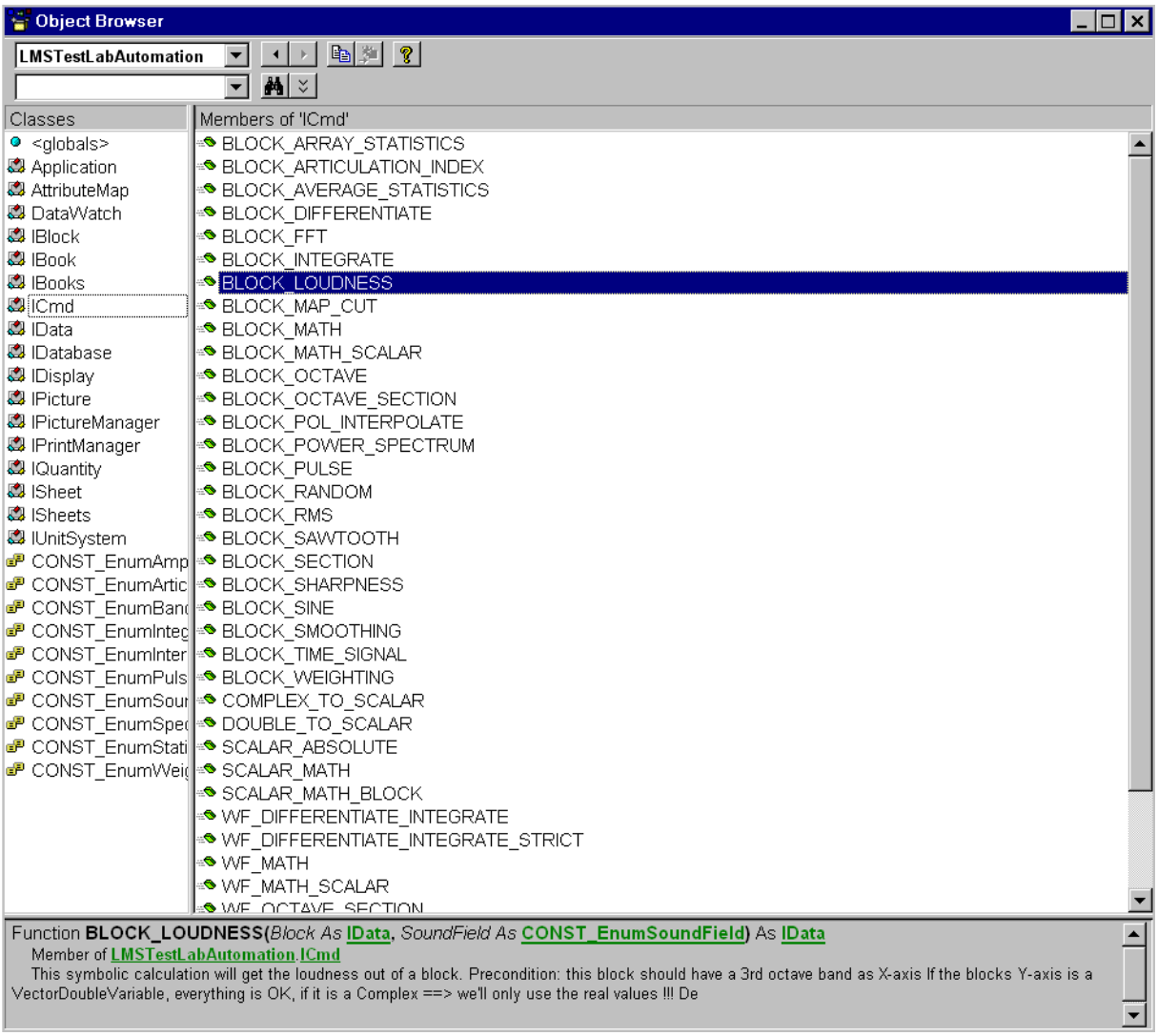


Figure 28

Previous figure shows the list of methods that can be issued on the ICmd interface. Lets take the selected one as an example and see how it can be used.

Method: BLOCK_LOUDNESS

Signature:

Cfr . Object Browser or Automation Viewer

Description:

Cfr . Object Browser or Automation Viewer

Example:

```
Set TestLab = CreateObject("LMSTestLabAutomation.Application.4A")

Dim TestLabCmds As Object

Set TestLabCmds = TestLab.cmd

Dim ResultBlock As LMSTestLabAutotmation.IBlock

Dim SourceBlock As LMSTestLabAutotmation.IBlock

'Retrieve SourceBlock

...

Set ResultBlock = TestLabCmds.BLOCK_LOUDNESS(SourceBlock, SoundFieldDiffuse)
```

Calculates the loudness of the sourceblock according to the specified SoundField. The different Soundfield values are available in the typelibrary as enumerations.

3.12 IUnitsystem.

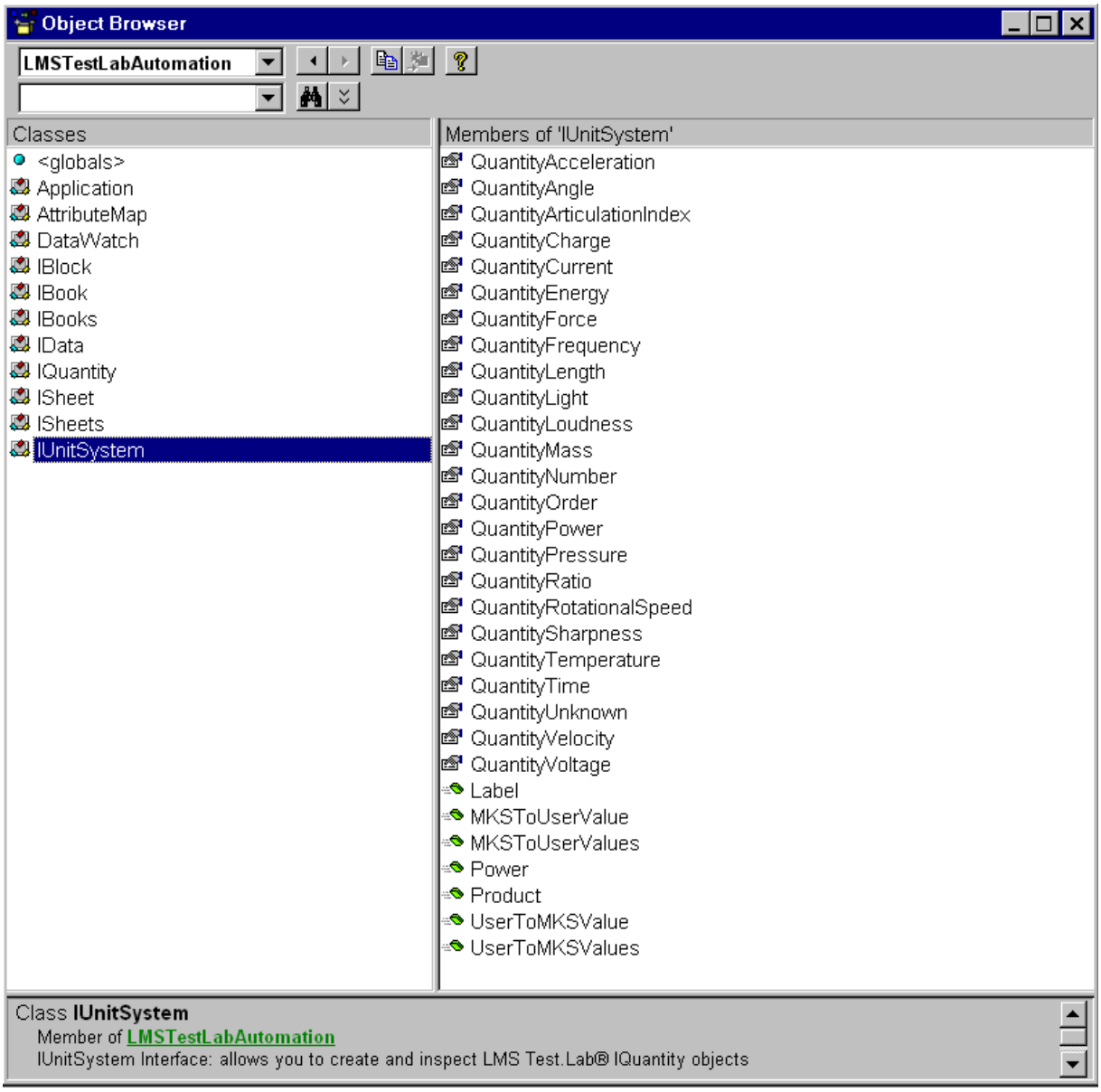


Figure 29

3.12.1 Property: Quantity...

Returns the quantity for 'Acceleration', 'Angle', 'ArticulationIndex',...

3.12.2 Method: Power

Signature:

Power(Quantity, Power)

Description:

Returns a quantity corresponding to Quantity ** Power, whereas Power is an integer value.

3.12.3 Method: Product

Signature:

Product(Quantity1 , Quantity2)

Description:

Returns a quantity that represents the product of Quantity1 and Quantity2.

3.12.4 Method: Label

Signature:

Label(Quantity)

Description:

Returns the label that represents this quantity in the current unit set.

3.12.5 Method: MKSToUserValue

Signature:

MKSToUserValue(Quantity, MKSValue)

Description:

Returns a converted User Value for a given MKS Value in the represented quantity in the current unit set.

3.12.6 Method: MKSToUserValues

Signature:

MKSToUserValues(Quantity, MKSValues)

Description:

Returns the converted User Values for the given array of MKS Values in the represented quantity in the current unit set.

3.12.7 Method: UserToMKSValue

Signature:

UserToMKSValue (Quantity, UserValue)

Description:

Returns a converted MKS Value for a given User Value in the represented quantity in the current unit set.

3.12.8 Method: UserToMKSValues

Signature:

UserToMKSValues (Quantity, UserValues)

Description:

Returns the converted MKS Values for the given array of User Values in the represented quantity in the current unit set.

3.13 IQuantity.

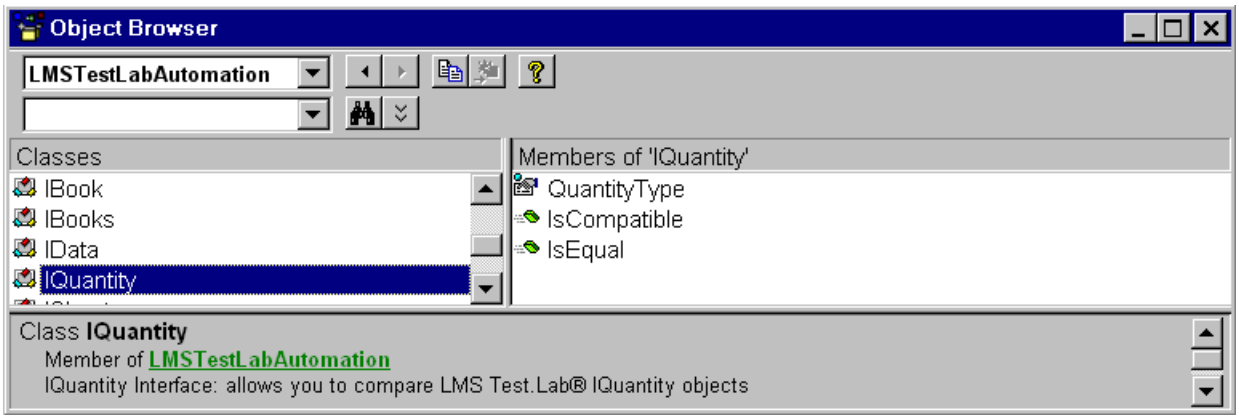


Figure 30

3.13.1 Property: QuantityType

Type: Read-Only

Description:

Returns the Quantity Type as a string value, e.g. "Acceleration", "Pressure", "Voltage".

3.13.2 Method: IsCompatible

Signature:

IsCompatible(OtherQuantity)

Description:

Returns whether a quantity is compatible with another quantity

3.13.3 Method: IsEqual

Signature:

IsEqual(OtherQuantity)

Description:

Returns whether a quantity equals another quantity

3.14 **InProcApplication**

Working with automation as a technology has a lot of benefits.

-Language independency

-Platform independency

-Transparent support for remote communication.

-Easy and flexible interoperability

-…

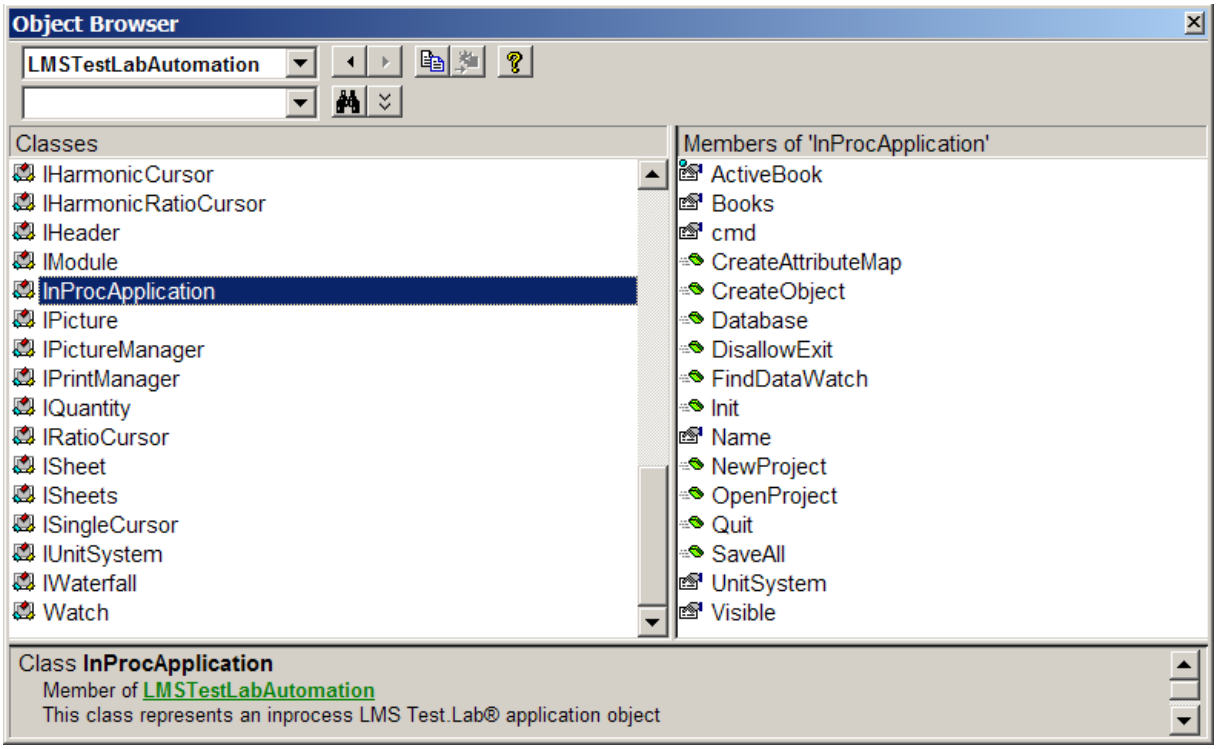
All this support for transparent systems implies that the Windows Automation mechanism has to maintain some cross process information for each reference that is distributed to a client application. However light-weighted this may be, it can have some effects on both performance and data integrity.

In scenarios, in which the client application is run inside Test.Lab, this cross process 'wrapping' of data is some unnecessary overhead. Connecting and using Test.Lab automation functionality in an ActiveX control, running in one of the Test.Lab user sheets, is one of those possible scenarios.

The 'InProcApplication' interface has been set up especially for those specific situations that the client application is running inside the same process as Test.Lab. Creation is identical as for the Application object. The result of the creation of an LMSTestLabAutomation.InProcApplication object can either be a valid object, or an empty object, in case the client code is not executed in the Test.Lab process itself.

Next picture indicates the members of InProcApplication. Mark that these are identical to the ones in the Application object. In most cases switching to the better performing in-process object is usually merely a matter of specifying a different class name.

```
Dim TestLab As LMSTestLabAutomation.InProcApplication
Set TestLab = CreateObject("LMSTestLabAutomation.InProcApplication")
```



Please note that the performance benefits will only be experienced when Test.lab automation functions are being used under stress, such as looping and executing Test.lab methods very frequently.

3.15 Custom Data Interfaces.

Next to the IData object, which represents any Test.Lab™ internal object, specific data objects can have more fit-for-purpose interfaces. These interfaces can only be applied on certain data types, and provide a more convenient way of interacting with these data objects, as compared to the generic 'IData' decomposition.

3.15.1 IBlock (enhanced by IBlock2)

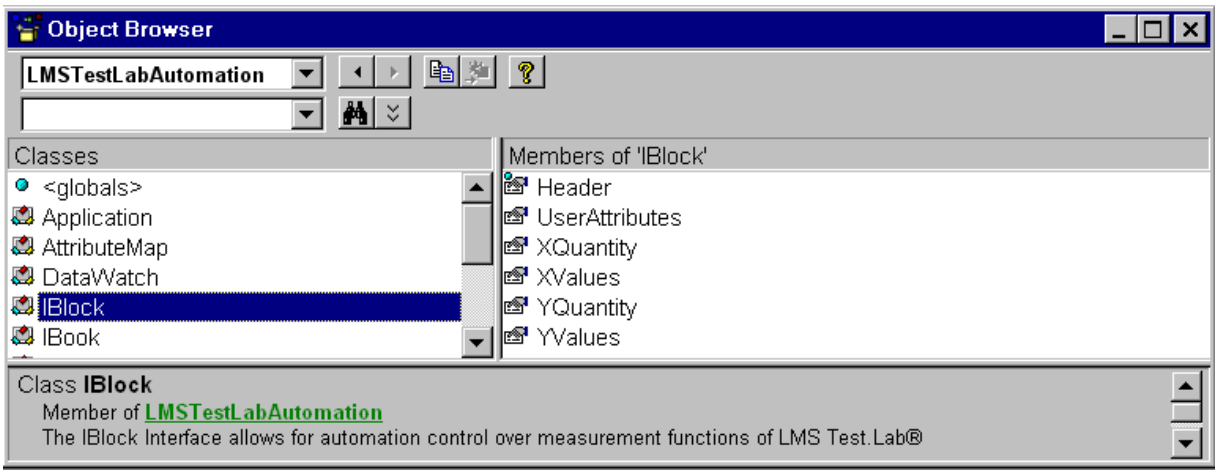


Figure 31

One of the first custom data interfaces added to the Test.Lab™ automation object model is that of a data-block.

3.15.1.1 Property: Header

Type: Read-Only

Description:

Shows the header of this object. The header values are a predefined, application dependent set of name value pairs, returned in an AttributeMap object. This set will differ between Signature and Environmental applications for instance.

Example:

```
' Test the type of the data on the selected port
 If BlockPort.Data.Type = "LmsHq::DataModelI::Expression::CBufferIBlock" Then
    ' Assign the generic IData to a specific Interface for IBlock
    Dim BlockData As LMSTestLabAutomation.IBlock
    Set BlockData = BlockPort.Data
    Dim header_attributes as LMSTestLabAutomation.AttributeMap
    Set header_attributes = BlockData.Header
```

3.15.1.2 Property: UserAttributes

Type: Read-Only

Description:

Shows the user attributes of this object. The attribute values are returned in an AttributeMap. There's no application context limitation on the contents of the 'UserAttributes' list, and can be extended freely to host additional information.

Example:

```
Dim BlockData As LMSTestLabAutomation.IBlock
Set BlockData = BlockPort.Data
Dim header_attributes as LMSTestLabAutomation.AttributeMap
Set header_attributes = BlockData.UserAttributes
```

3.15.1.3 Property: XQuantity

Type: Read-Only

Description:

Gets the X-axis quantity.

Example:

```
Set BlockData = BlockPort.Data
' Do something with Quantities
Dim XUnitLabel As String
XUnitLabel = TestLab.UnitSystem.Label(BlockData.XQuantity)
Dim XUnitLabel As String
XUnitLabel = TestLab.UnitSystem.Label(BlockData.XQuantity)
```

3.15.1.4 Property: XValues

Type: Read-Only

Description:

Returns an array of double values, holding all the values of the X-axis.

Example:

```
If BlockPort.Data.Type = "LmsHq::DataModelI::Expression::CBufferIBlock" Then
  Dim BlockData As LMSTestLabAutomation.IBlock
  Set BlockData = BlockPort.Data
  ' Get the X-values
  Dim xval() As Double
  xval = BlockData.XValues
  length = UBound(xval) - LBound(xval) + 1
```

3.15.1.5 Property: YQuantity

Type: Read-Only

Description:

Gets the Y-axis quantity.

Example:

```
Set BlockData = BlockPort.Data
' Do something with Quantities
Dim XUnitLabel As String
XUnitLabel = TestLab.UnitSystem.Label(BlockData.XQuantity)
Dim YUnitLabel As String
YUnitLabel = TestLab.UnitSystem.Label(BlockData.YQuantity)
```

3.15.1.6 Property: YValues

Type: Read-Only

Description:

Returns a two-dimensional array of double values, holding in its first dimention all real values and in the second dimension all imaginary values of the Y-axis.

Example:

```
' Get the Y-values,' 2 dimensions : real, imag
Dim yval() As Double
yval = BlockData.Yvalues
```

3.15.1.7 Property: Properties

Type: Read-Only

Description:

Shows the properties of this object. The properties are returned in an AttributeMap object.

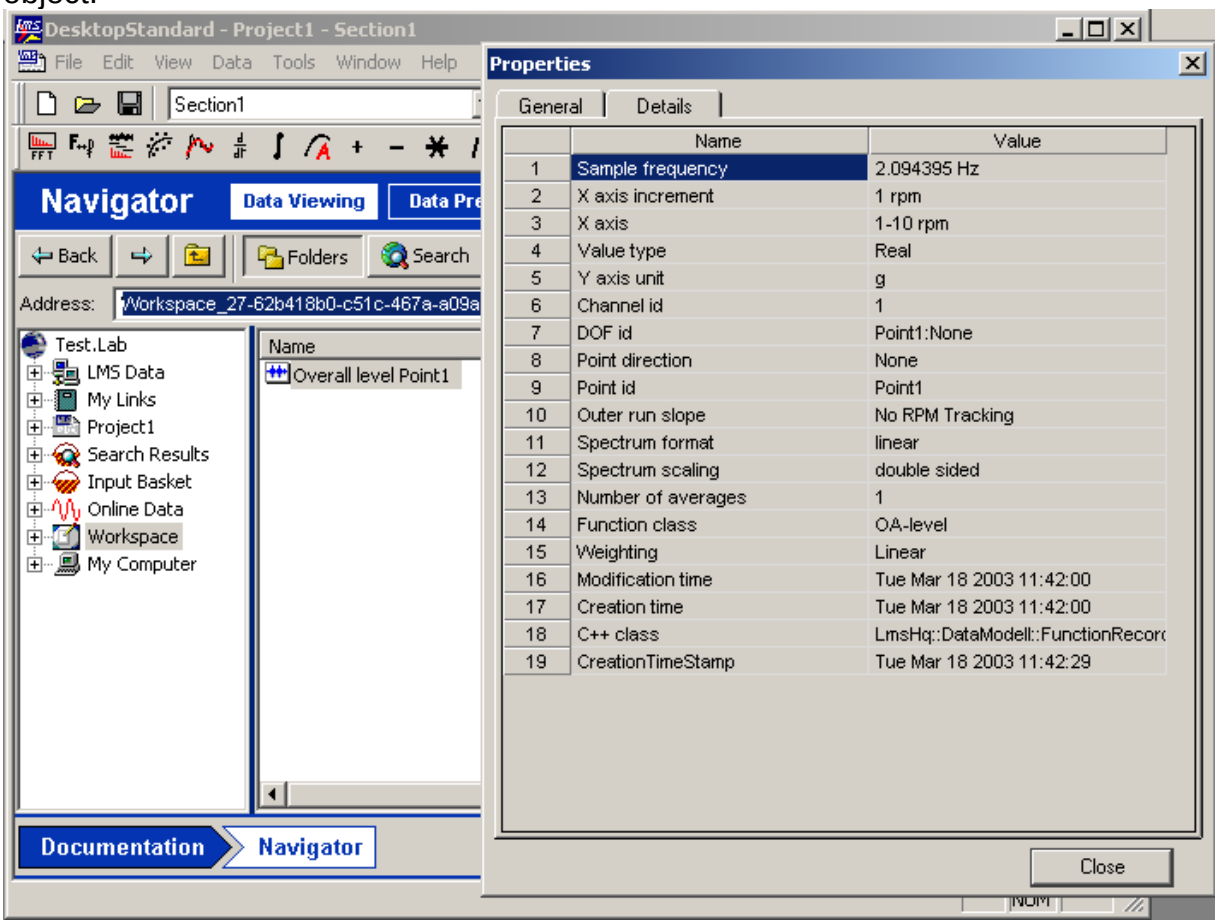
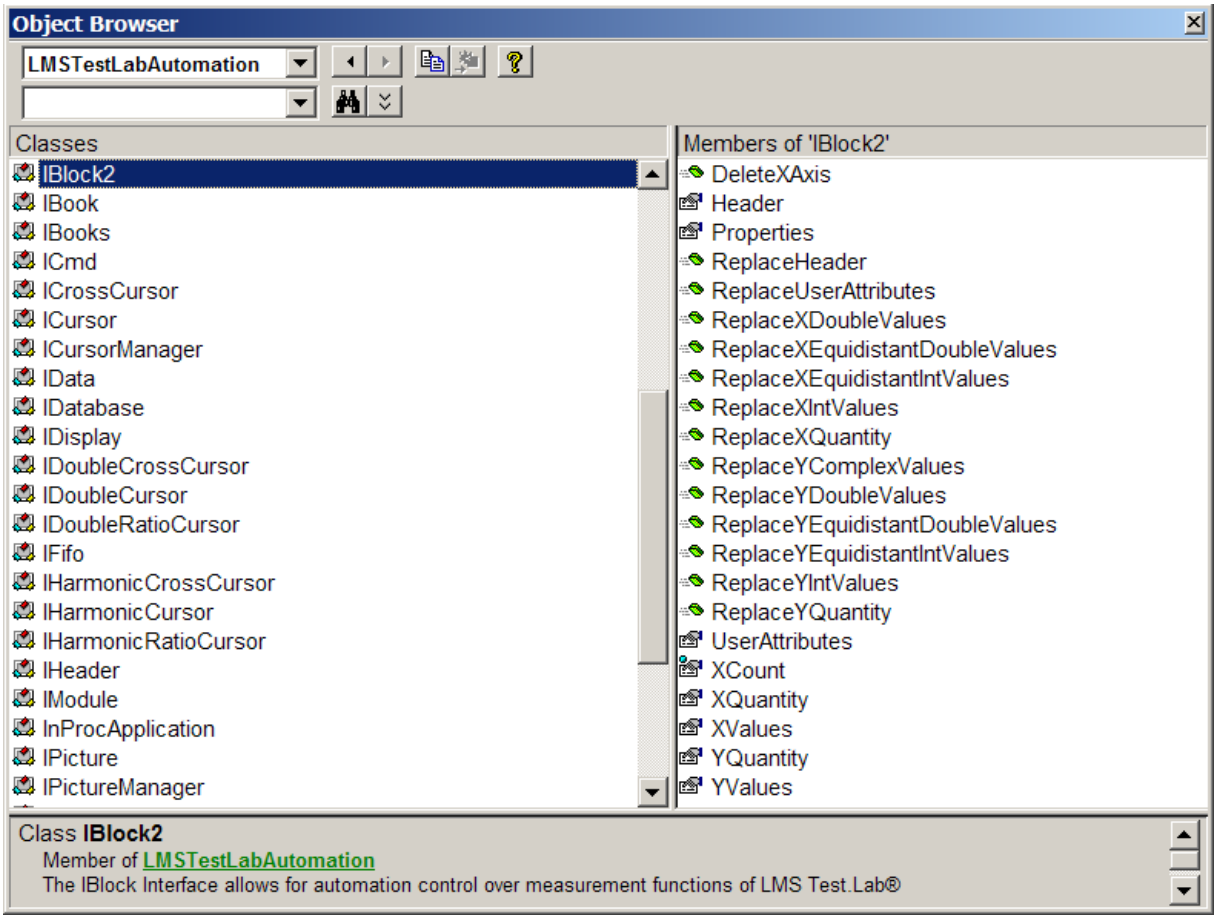


Figure 32

Shows some property values. They are a predefined, application dependent set of name value pairs. The information presented is a summary of a information coming from header, axis and user attributes.

3.15.2 IBlock2



3.15.2.1 Property: XCount

Type: Read-Only

Description:

Returns the number of X-axes in the block-object upon which the property was requested.

Example:

```
Dim number_of_axes as Integer

number_of_axes = BlockData.XCount
```

3.15.2.2 Property: XValues

Type: Read-Only

Description:

Returns an array of double values, holding all the values of the n-th X-axis. If no axis index is specified the values of the first one are returned. Index values are zero-based.

Example:

```
If BlockPort.Data.Type = "LmsHq::DataModelI::Expression::CBufferIBlock" Then
```

```
Dim BlockData As LMSTestLabAutomation.IBlock
Set BlockData = BlockPort.Data
' Get the X-values
Dim xval() As Double
xval = BlockData.Xvalues(1)
length = UBound(xval) - LBound(xval) + 1
```

3.15.2.3 Property: YValues

Type: Read-Only

Description:

Returns a two-dimensional array of double values, holding in its first dimention all real values and in the second dimension all imaginary values of the Y-axis.

Example:

```
' Get the Y-values,' 2 dimensions : real, imag
Dim yval() As Double
yval = BlockData.Yvalues
```

3.15.2.4 Property: XQuantity

Type: Read-Only

Description:

Gets quantity for the n-th X-asis. Default value for n is zero, which is the first one.

Example:

```
Set BlockData = BlockPort.Data
' Do something with Quantities
Dim XUnitLabel As String
XUnitLabel = TestLab.UnitSystem.Label(BlockData.Xquantity(1))
```

3.15.2.5 Property: YQuantity

Type: Read-Only

Description:

Gets the Y-axis quantity.

Example:

```
Set BlockData = BlockPort.Data
' Do something with Quantities
Dim YUnitLabel As String
YUnitLabel = TestLab.UnitSystem.Label(BlockData.YQuantity)
```

3.15.2.6 Property: Header

Type: Read-Only

Description:

Returns the header of the data block. The IHeader interface will be covered in the next section (cfr 3.15.3)

Example:

```
Set BlockData = BlockPort.Data

Dim Blockheader As LmsTestLabAutomation.IHeader

Blockheader = BlockData.Header
```

3.15.2.7 Property: UserAttributes

Type: Read-Only

Description:

Shows the user attributes of this object. The attribute values are returned in an AttributeMap. There's no application context limitation on the contents of the 'UserAttributes' list, and can be extended freely to host additional information. This means that the application does not use the contents of the userattributes to interpret the measurement data, as opposed to the header object. Note for UPA users: since headers in CADA-X were fixed and contained some unused fields, depending on the type of blocks it described, these fields were ‘used’ to store some user annotation. In Test.Lab the header contains only those fields essential for the application to interpret the data, these rarely need to be modified. All user annotation can be safely stored and retrieved in the UserAttributes.

Example:

```
Dim BlockData As LMSTestLabAutomation.IBlock

Set BlockData = BlockPort.Data

Dim user_attributes as LMSTestLabAutomation.AttributeMap

Set user_attributes = BlockData.UserAttributes
```

3.15.2.8 Property: Properties

Type: Read-Only

Description:

Shows the properties of this object. The properties are returned in an AttributeMap object.

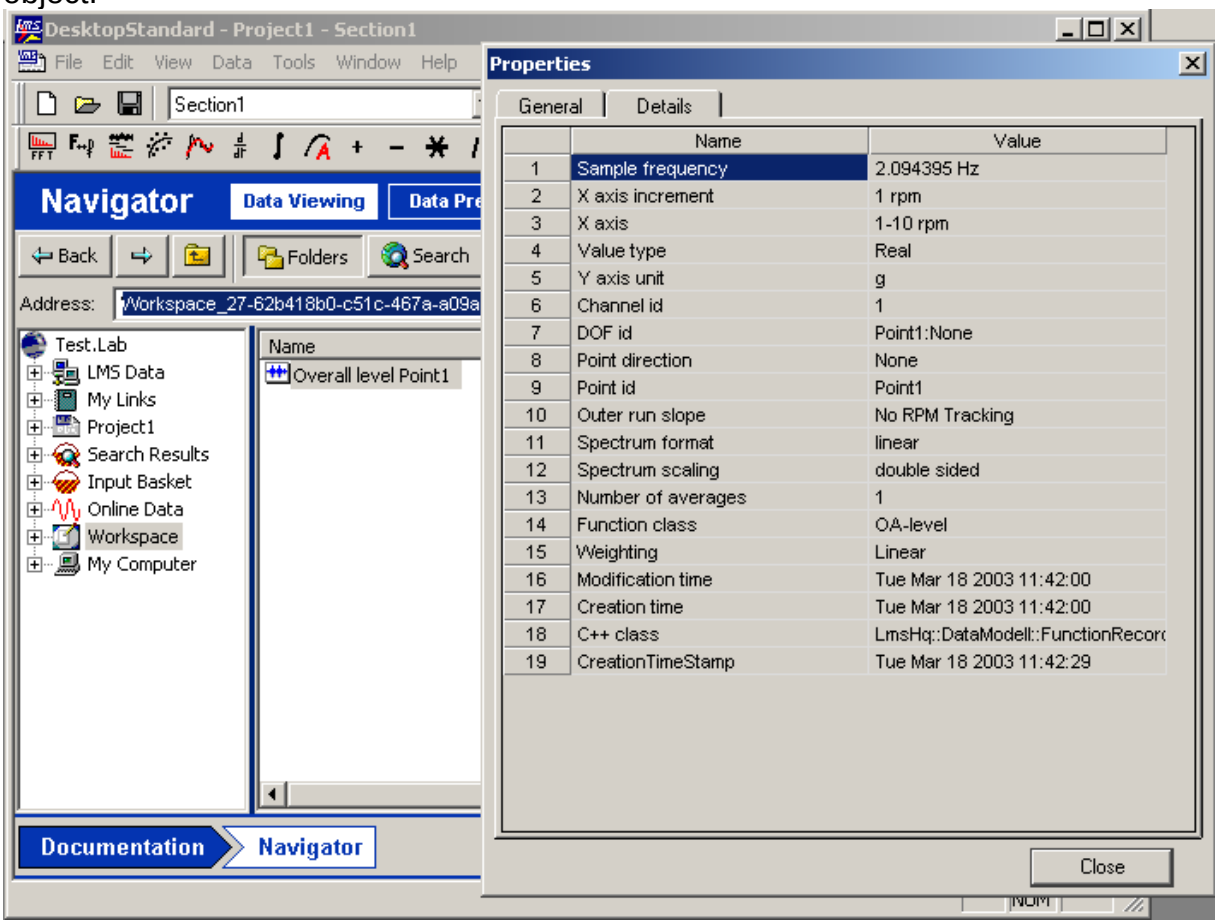


Figure 33

Shows some property values. They are a predefined, application dependent set of name value pairs. The information presented is a summary of information coming from header, axis and user attributes.

3.15.2.9 Method: ReplaceXDoubleValues

Signature:

ReplaceXDoubleValues (NewValues,AxisIndex)

Description:

This method creates a new IBlock2 object based upon the old block, but with the X-MKS-values of the 'axisindex'-th axis replaced by the specified array of doubles. If omitted AxisIndex = 0;

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2

Dim array_of_double_values () as Integer

…

Dim NewBlock as LMSTestLabAutomation.IBlock2

Set NewBlock =  TlBlock. ReplaceXDoubleValues  ( array_of_double_values,2 )
```

3.15.2.10 Method: ReplaceXIntValues

Signature:

ReplaceXIntValues (NewValues,AxisIndex)

Description:

This method creates a new IBlock2 object based upon the old block, but with the X-MKS-values of the 'axisindex'-th axis replaced by the specified array of integers. If omitted AxisIndex = 0;

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
Dim array_of_int_values() as Integer
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceXDoubleValues ( array_of_int_values,2 )
```

3.15.2.11 Method: ReplaceXEquidistantIntValues

Signature:

ReplaceXEquidistantIntValues (NewStartXValue,XaxisIncrement,,AxisIndex)

Description:

Replaces the 'Axisindex'-th X-axis with an equidistant one, starting with the integer NewStartXValue and increment XaxisIncrement. If omitted, AxisIndex = 0.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceXEquidistantIntValues (16,2,2 )
```

3.15.2.12 Method: ReplaceXEquidistantDoubleValues

Signature:

ReplaceXEquidistantDoubleValues (NewStartXValue,XaxisIncrement,,AxisIndex)

Description:

Replaces the 'Axisindex'-th X-axis with an equidistant one, starting with the double NewStartXValue and increment XaxisIncrement. If omitted, AxisIndex = 0.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceXEquidistantIntValues (10.3,2.5,2 )
```

3.15.2.13 Method: ReplaceXQuantity

Signature:

```
ReplaceXQuantity (Quantity, AxisIndex)
```

Description:

Replaces the quantity of the 'Axisindex'-th X-axis with the specified one. If omitted, AxisIndex = 0.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceXQuantity (new_tl_quantity )
```

3.15.2.14 Method: DeleteXAxis

Signature:

```
DeleteXAxis (AxisIndex)
```

Description:

Deletes the specified X-axis. If no index is specified the first one is taken as default.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. DeleteXAxis (1 )
```

3.15.2.15 Method: ReplaceYComplexValues

Signature:

```
ReplaceYComplexValues (NewValues,AxisIndex)
```

Description:

This method creates a new IBlock2 object based upon the old block, but with the Y-MKS-values replaced by the specified 2-dimentional array of doubles. First dimension will contain the real, second the imaginary values.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
Dim array_of_ complex _values (2)() as Integer
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceYComplexValues ( array_of_complex_values)
```

3.15.2.16 Method: ReplaceYDoubleValues

Signature:

ReplaceYDoubleValues (NewValues)

Description:

This method creates a new IBlock2 object based upon the old block, but with the MKS real Y-values replaced by the specified 1-dimentional array of doubles.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
Dim array_of_double_values () as Integer
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceYDoubleValues ( array_of_double_values)
```

3.15.2.17 Method: ReplaceYIntValues

Signature:

ReplaceYIntValues (NewValues)

Description:

Replaces the Y-axis with the values in the passed array of integers and return the newly created IBlock2 object.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
Dim array_of_int_values() as Integer
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceYDoubleValues ( array_of_int_values)
```

3.15.2.18 Method: ReplaceYEquidistantIntValues

Signature:

ReplaceYEquidistantIntValues (NewStartYValue,YaxisIncrement)

Description:

Replaces the Y-axis with an equidistant one, starting with the integer NewStartYValue and increment YaxisIncrement.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceYEquidistantIntValues (16,2)
```

3.15.2.19 Method: ReplaceYEquidistantDoubleValues

Signature:

```
ReplaceYEquidistantDoubleValues (NewStartYValue,YaxisIncrement)
```

Description:

Replaces the Y-axis with an equidistant one, starting with the double NewStartYValue and increment YaxisIncrement.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceYEquidistantIntValues (10.3,2.5)
```

3.15.2.20 Method: ReplaceYQuantity

Signature:

```
ReplaceYQuantity (Quantity)
```

Description:

Replaces the quantity of the Y-axis with the specified one, and returns the newly created block.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
…
Dim NewBlock as LMSTestLabAutomation.IBlock2
Set NewBlock = TlBlock. ReplaceYQuantity (new_tl_quantity )
```

3.15.2.21 Method: ReplaceHeader

Signature:

```
ReplaceHeader (NewHeader)
```

Description:

Creates a new IBlock2 object, based on the original one, with only the header object replaced with the passed NewHeader.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2
Dim NewBlock as LMSTestLabAutomation.IBlock2
Dim ModifiedHeader as LMSTestLabAutomation.IHeader
…
set ModifiedHeader = OrigHeader.Edit(( “HeaderFieldZ”,UpdatedvalueforZ )
Set NewBlock = TlBlock. ReplaceHeader (ModifiedHeader)
```

3.15.2.22 Method: ReplaceUserAttributes

Signature:

ReplaceUserAttributes (NewUserAttributes)

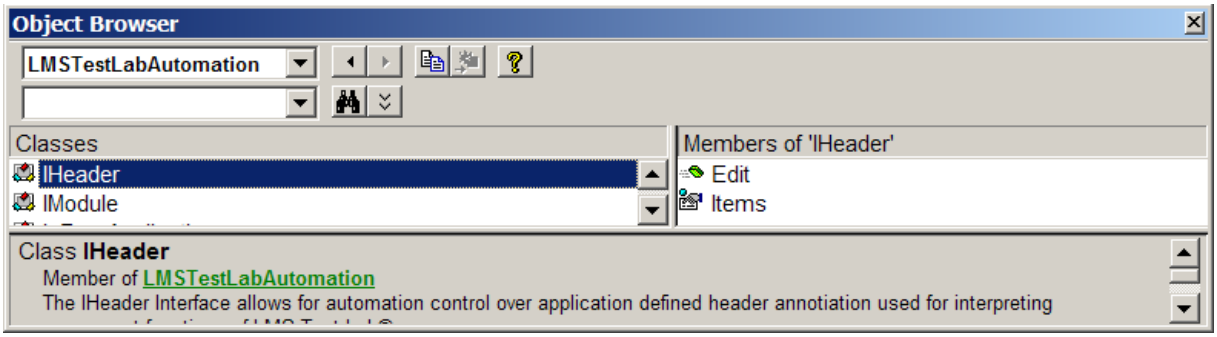
Description:

Creates a new IBlock2 object, based on the original one, with only the user attributes replaced with the passed NewUserAttributes.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2

Dim ModifAttrib as LMSTestLabAutomation.IattributeMap

Set ModifAttrib = TlBlock.UserAttributes

ModifAttrib.Add("MyOwnAnnotation","VehicleType ABC")

Dim NewBlock as LMSTestLabAutomation.IBlock2

…

Set NewBlock = TlBlock. ReplaceUserAttributes (ModifAttrib)
```

3.15.3 IHeader



3.15.3.1 Method: Edit

Signature:

Edit(HeaderFieldname,NewValue)

Description:

The 'Edit' method enables the modification of an IHeader object. The function is comparable to 'Correct Header properties', which you can issue interactively in Test.Lab. The result of this operation is a new header object that can be used in subsequent modifications, or to be replaced in an IBlock.

Example:

```
Dim TlBlock as LMSTestLabAutomation.IBlock2

…

Dim Existingheader As LMSTestLabAutomation.IHeader

Dim Modifiedheader As LMSTestLabAutomation.IHeader

Set Existingheader = TlBlock.Header

Set Modifiedheader = Existingheader.Edit( "HeaderFieldX",UpdatedvalueforX )
```

3.15.3.2 Property: Items

Type: Read-Only

Signature:

```
Property Items As Attributemap
```

Description:

This returns an attributemap object, containing all fields in the header. The value of these fields can be composites, i.e. not simple types such as interger, Boolean or strings. Most of these values are presented as simple types in properties list (cfr)

3.15.4 IPictureManager

The IPictureManager interface exposes part of the functionality of the Data Viewing minor of the Navigator sheet of Test.Lab™ Desktop. Through this interface one can manage pictures and displays for data viewing and data printing (**picture-based printing**) purposes.

Retrieving an IPictureManager interface can be done by accessing a port containing a picture manager. The LMS Test.Lab™ Desktop application for instance, holds such a port.

Example:

```
Dim Port As LMSTestLabAutomation.DataWatch

Dim PictureManager As LMSTestLabAutomation.IPictureManager

Set Port = TestLab.ActiveBook.FindDataWatch("Navigator//DataViewing//PictureManager")

Set PictureManager = Port.Data
```

Retrieves a IPictureManager interface from the Navigator sheet of the LMS Test.Lab™ Desktop application.

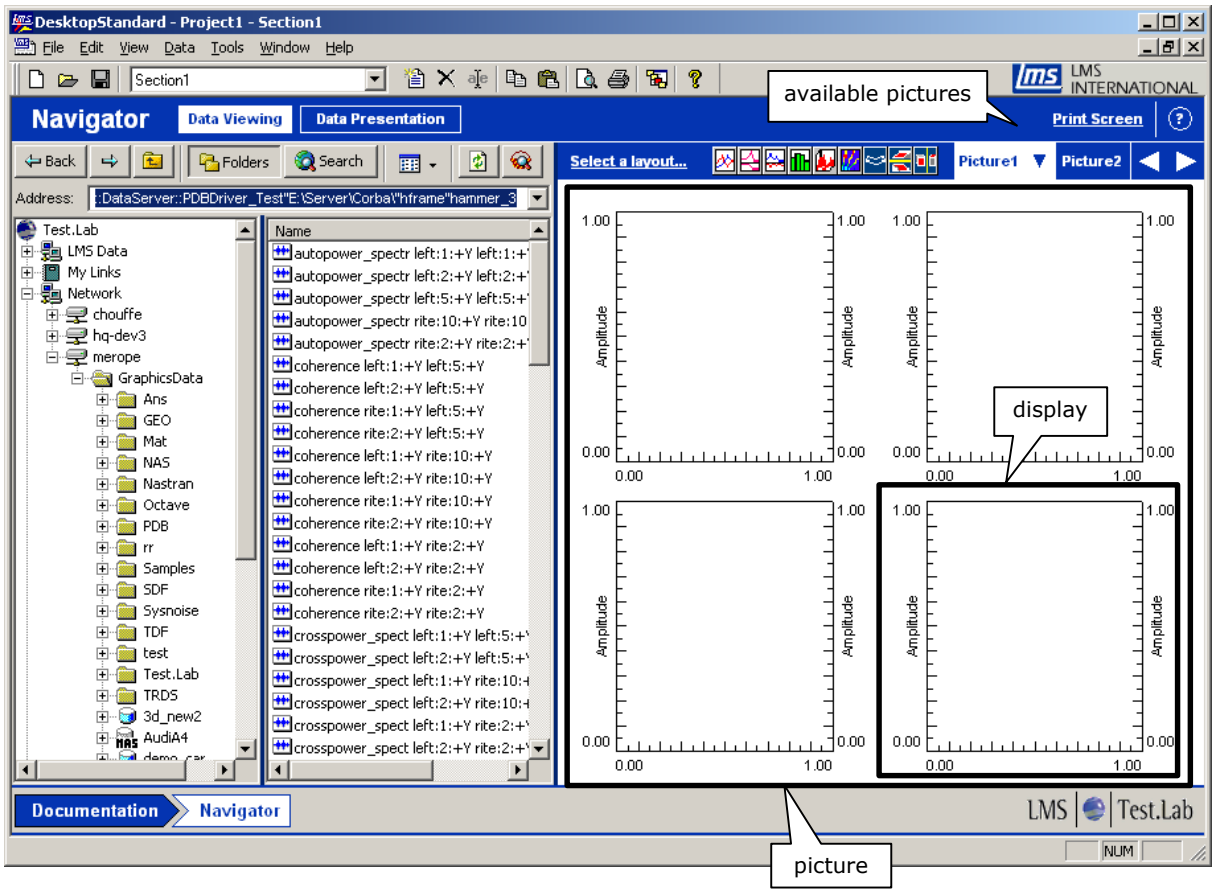


Figure 34

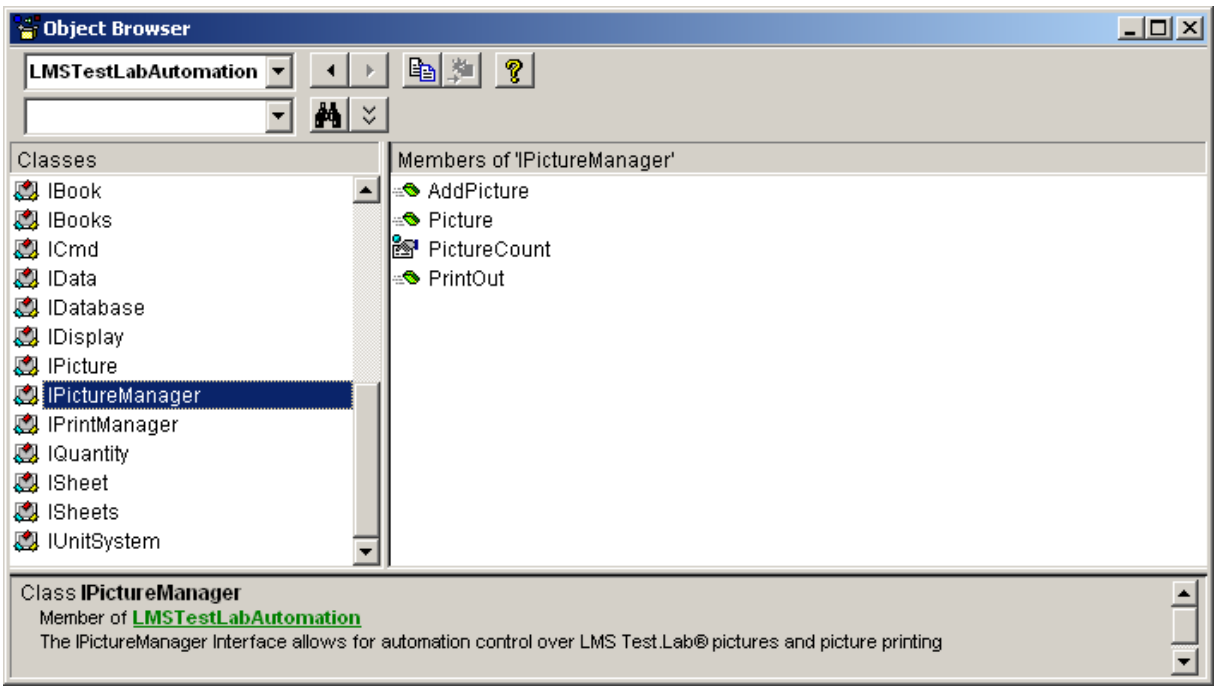


Figure 35

3.15.4.1 Method: AddPicture

Signature:

```
Function AddPicture(LayoutName As String) As IPicture
```

Description:

Adds a new picture to the picture manager based on the specified layout name and returns an interface to this new picture. Any layout name available in the Layout Management dialog (menu View →Layout Management…) can be used to create a picture.

Example:

```
Dim Picture As LMSTestLabAutomation.IPicture

Set Picture =  PictureManager.AddPicture( "FrontBack" )
```

Creates a picture based on the layout "FrontBack" and the created picture is assigned to the Picture variable

3.15.4.2 Property: Picture

Type: Read-Only

Signature:

```
Property Picture(Index As Long) As IPicture
```

Description:

Retrieves one picture from the picture manager using the specified index and returns an interface to this picture. The index used is zero-based (thus, use zero to retrieve the first picture).

3.15.4.3 Property: PictureCount

Type: Read-Only

Signature:

```
Property PictureCount As Long
```

Description:

Returns the total number of pictures available in the picture manager.

3.15.4.4 Subject: PrintOut

Signature:

```
Sub PrintOut(Index As Long, [PrintFormatName As String], [OutputFilename As String])
```

Description:

Prints a picture from the print manager using a print-format on the default printer or an electronic document. This procedure is referred to as picture-based printing. This method takes three parameters:

- *Index*
  The zero-based index specifies which picture is to be printed.
- *PrintFormatName* [optional]
  Specifies the name of the desired print-format to print with. The print-format should be created prior to picture printing and should be compatible with the picture which is about to be printed. The print-format name does not have to contain a full path to the print-format filename, it will be loaded from the

standard print-format folder. Note that the extension of the print-format (either .doc or .ppt) must be specified.
If no print-format is specified, the system will use the default print-format associated with the layout on which is picture is based.

- *OutputFilename* [optional]
  Specifies the **full pathname** of the output file (electronic document). If no output filename is specified, the default printer will be used as output device (paper document).

As a consequence, using the last two parameters one can overrule the options of the Printing Options dialog (menu File → Printing Options…).

Example:

```
PictureManager.PrintOut 0, “FrontBack.doc”, “C:\OUTPUT\Out1.doc”
```

Prints the first picture of the picture manager, using the FrontBack.doc print format, as an electronic document to C:\OUTPUT\Out1.doc

3.15.5 IPrintManager

The IPrintManager interface exposes part of the functionality of the Data Presentation minor of the Navigator sheet of Test.Lab™ Desktop. Through this interface one can manage pictures and displays for **format-based printing** purposes.

Retrieving an IPrintManager interface can be done by accessing a port containing a print manager. At this point, only the LMS Test.Lab™ Desktop application holds such a port.

Example:

```
Dim Port As LMSTestLabAutomation.DataWatch

Dim PrintManager As LMSTestLabAutomation.IPrintManager

Set Port = TestLab.ActiveBook.FindDataWatch("Navigator//DataPresentation//PrintManager")

Set PrintManager = Port.Data
```

Retrieves an IPrintManager interface from the Navigator sheet of the LMS Test.Lab™ Desktop application.

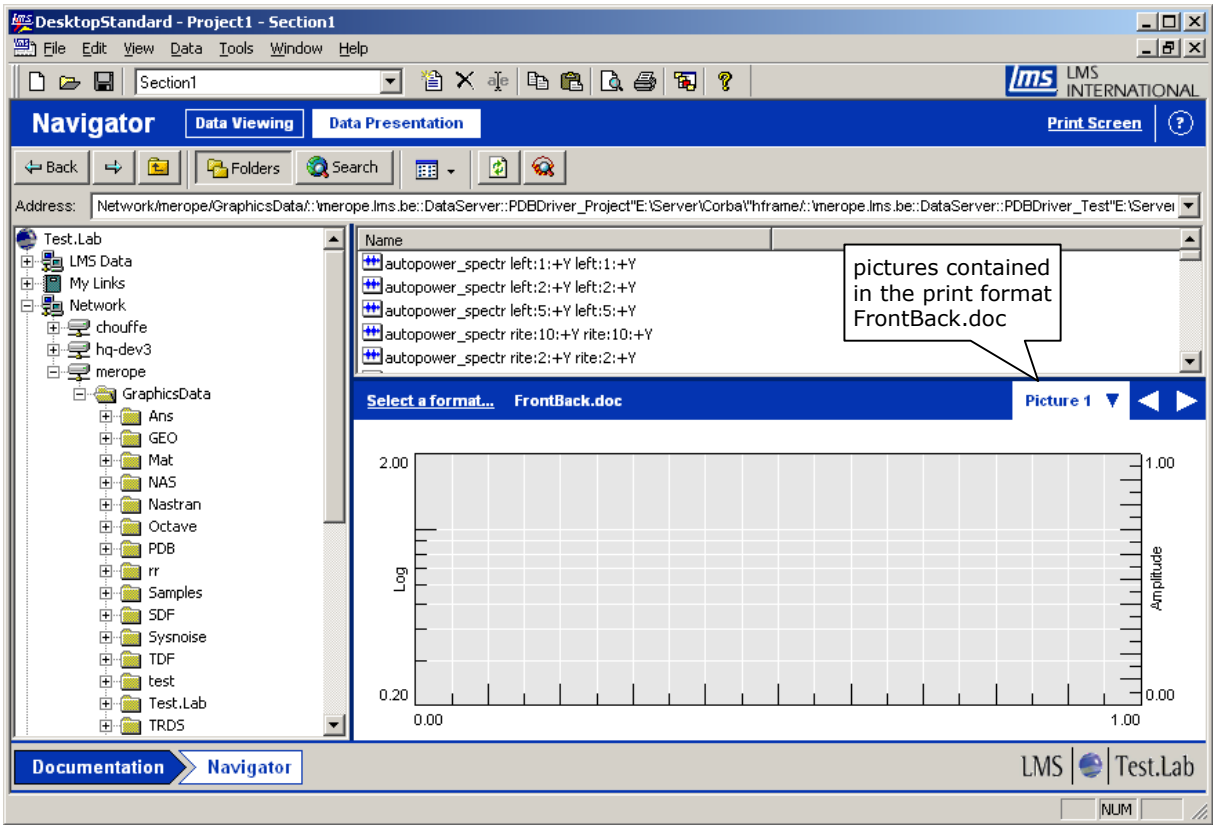


Figure 36

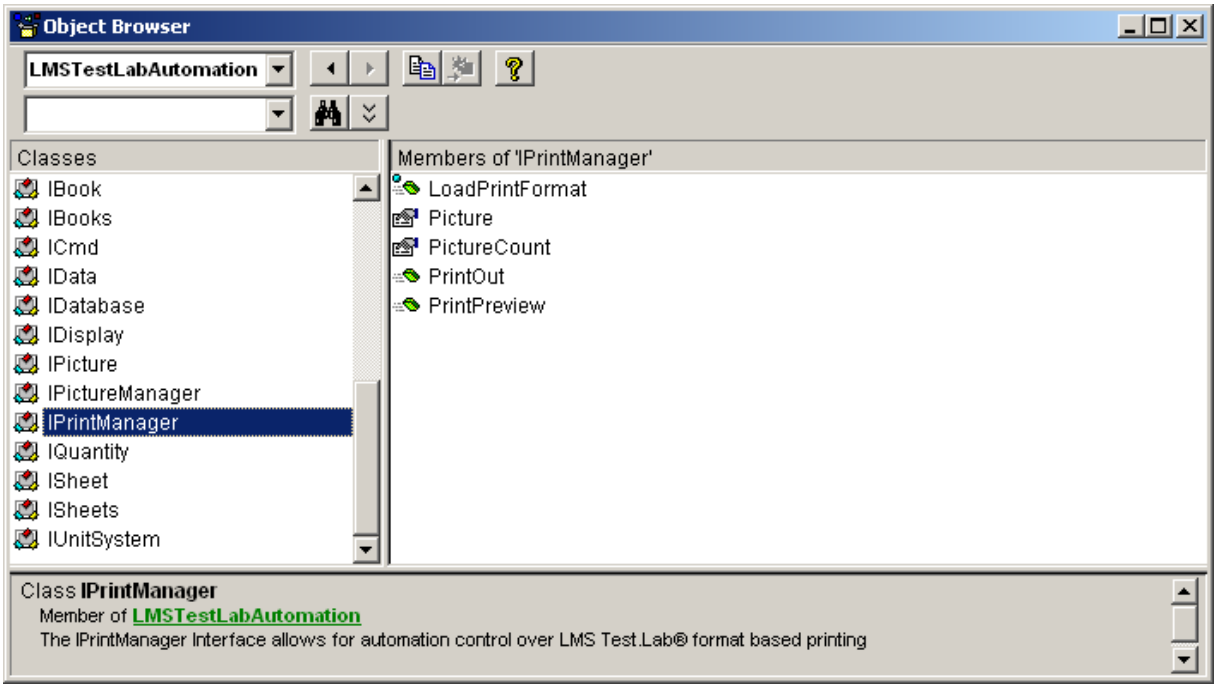


Figure 37

3.15.5.1 Method: LoadPrintFormat

Signature:

Sub LoadPrintFormat(PrintFormatName As String)

Description:

Loads a print format into the print manager. For each active picture in the print format a picture will be created in the print manager. The PrintFormatName parameter specifies the name of the desired print-format to load. The print-format name does not have to contain a full path to the print-format filename, it will be loaded from the standard print-format folder. Note that the extension of the print-format (either .doc or .ppt) must be specified.

3.15.5.2 Property: Picture

Type: Read-Only

Signature:

```
Property Picture(Index As Long) As IPicture
```

Description:

Retrieves one picture from the picture manager using the specified index and returns an interface to this picture. The index used is zero-based (thus, use zero to retrieve the first picture).

3.15.5.3 Property: PictureCount

Type: Read-Only

Signature:

```
Property PictureCount As Long
```

Description:

Returns the total number of pictures available in the print manager.

3.15.5.4 Method: PrintOut

Signature:

```
Sub PrintOut([OutputFilename As String])
```

Description:

Initiates the format-based print as an electronic document or a paper document. The optional *OutputFilename* parameter specifies the full pathname of the output file (electronic document). If no output filename is specified, the default printer will be used as output device (paper document).

3.15.5.5 Method: PrintPreview

Signature:

```
Sub PrintPreview()
```

Description:

Creates and shows a preview of the format-based print.

3.15.6 IPicture

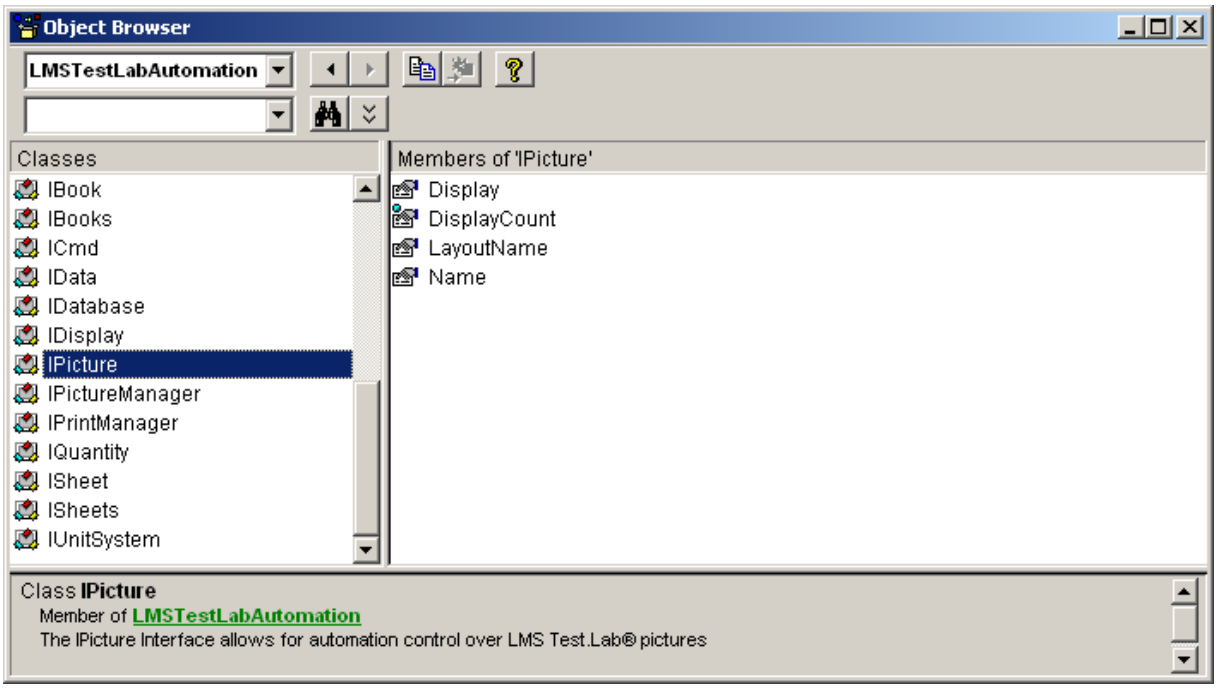


Figure 38

3.15.6.1 Method: Display

Signature:

Function Display(Index As Long) As IPicture

Description:

Retrieves one display from the picture.

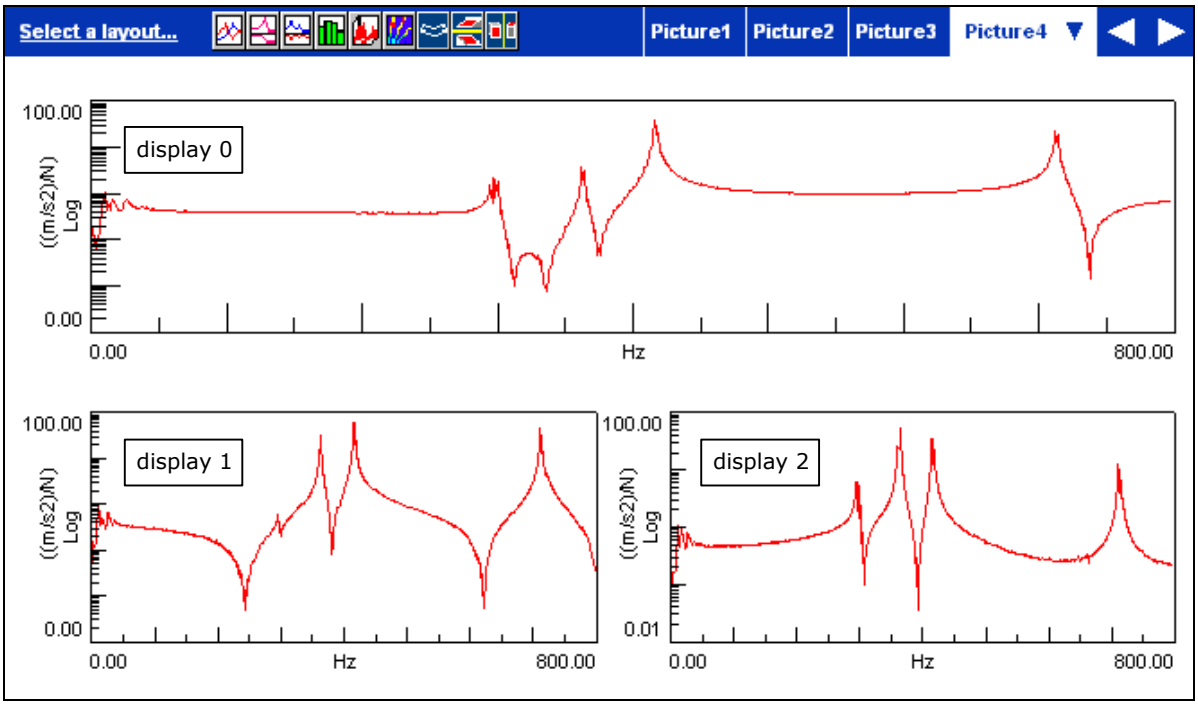


Figure 39

The index used is zero-based (thus, use zero to retrieve the first display). The order of the displays in the picture is left to right, top to bottom.

3.15.6.2 Property: DisplayCount

Type: Read-Only

Description:

Returns the total number of displays available in the picture.

3.15.6.3 Property: Name

Type: Read-Only

Description:

Returns the name of the picture.

3.15.6.4 Property: LayoutName

Type: Read-Only

Description:

Returns the layout name on which the picture is based on.

3.15.7 IDisplay

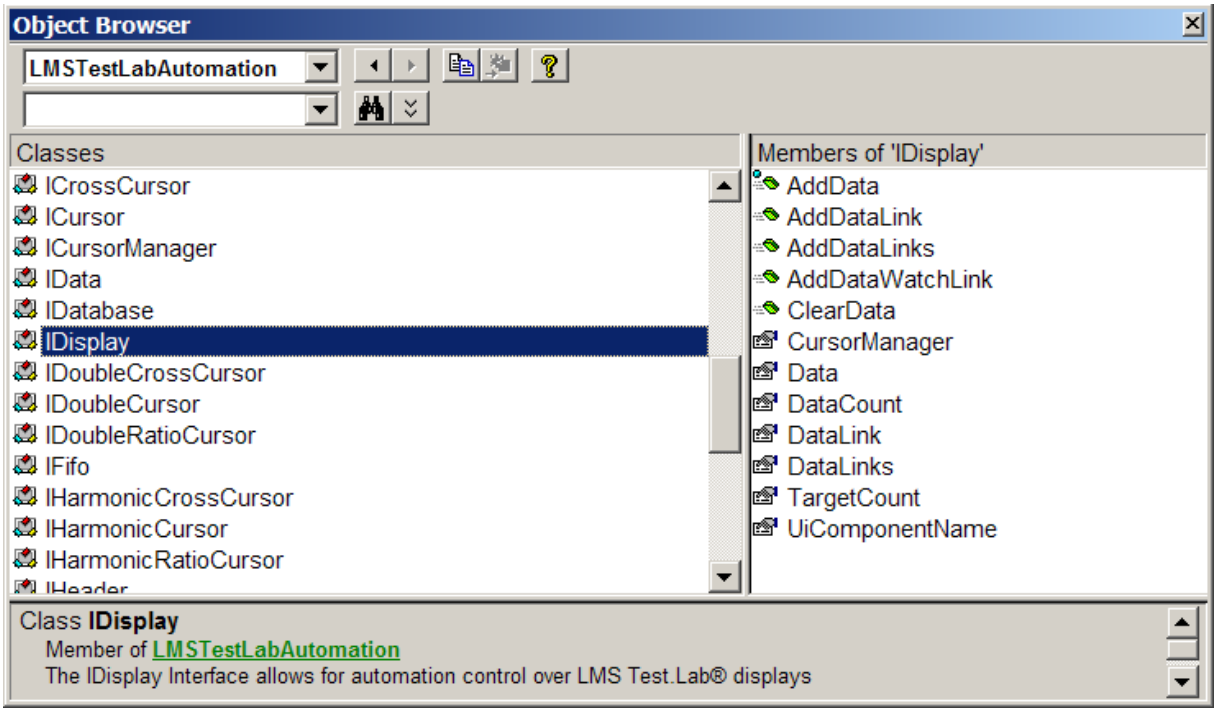


Figure 40

The IDisplay interface allows automation control over Test.Lab™ displays. At this point, only the following displays are supported through the IDisplay interface:

- Front/Back
- Bode

- Upper/Lower
- Octave

Most methods and properties of the IDisplay interface use the concept of target. Each display has a fixed number of targets depending on its axes configuration and is used to specify on which Y axis data must be added or retrieved.

Display	# Targets	Target 0	Target 1
Front/Back	2	Front Y axis	Back Y axis
Bode	1	Y axis	N.A.
Upper/Lower	2	Upper Y axis	Lower Y axis
Octave	1	Y axis	N.A.

Table 1

3.15.7.1 Method: AddData

Signature:

```
Sub AddData(Target As Long, Data As IData)
```

Description:

Adds data to a specific *Target*. In case of a Front/Back, Bode, Upper/Lower and an Octave display, the *Data* parameter should be of type IBlock (see 3.15.1 on page 53).

3.15.7.2 Method: AddDataLink

Signature:

```
Sub AddDataLink(Target As Long, DataLink As String)
```

Description:

Adds a data link to a specific *Target*. The *DataLink* parameter must contain the full path to some data which the display can visualize.

Example:

Display.AddDataLink(0,“My Computer/D:/Lms-NT/Local/Data/CompareRunsRef3.lms/Section1/Run_0/Fixed sampling/Runup/Sections/Frequencies/(0)Frequency 25.00 Hz Point1”)

adds a link to a block of a LMS Test.Lab project to the display.

Display.AddDataLink(0,“Network/hq-dev3/CorbaData/::\hq-dev3.lms.be::DataServer::FSDriver_Directory"C:\Program Files\LMS\CorbaData"Cada-x/::\hq-dev3.lms.be::DataServer::PDBDriver_Project"C:\Program Files\LMS\CorbaData\Cada-x\"hframe/::\hq-dev3.lms.be::DataServer::PDBDriver_Test"C:\Program Files\LMS\CorbaData\Cada-x\"hframe"hammer_3/::\hq-dev3.lms.be::DataServer::PDBDriver_Block"C:\Program Files\LMS\CorbaData\Cada-x\"hframe"hammer_3"28”)

adds a link to a block on the network to the display.

Display.AddDataLink(0,“{Project}/Section1/Run 1/Fixed sampling/Runup/Tacho/Time Tacho1”)

adds a link to a block of the current project to the display

3.15.7.3 Method: AddDataLinks

Signature:

```
Sub AddDataLinks(Target As Long, DataLinks() As String)
```

Description:

Adds one or more data links at once to a specific *Target*. The *DataLinks()* parameter specifies the list of links to be added, in other words, this is the “array version” of the AddDataLink method (see above). Adding multiple data links to the display at once (using the AddDataLinks method) is significantly faster than adding the data links one at a time (using the AddDataLink method).

3.15.7.4 Method: AddDataWatchLink

Signature:

```
Sub AddDataWatchLink(Target As Long, DataWatch As DataWatch)
```

Description:

Adds a link to a DataWatch (see 3.6 on page 30) to a specific *Target*. The specified *DataWatch* must hold data which the display can display.

Signature:

```
Dim MyDataWatch as LMSTestLabAutomation

Set MyDataWatch = TestLab.ActiveBook.CreateData
```

3.15.7.5 Method: ClearData

Signature:

```
Sub ClearData(Target As Long)
```

Description:

Clears all data on the specified *Target*.

3.15.7.6 Property: Data

Type: Read-Only

Signature:

```
Property Data(Target As Long, Index As Long) As IData
```

Description:

Retrieves one data entry from the specified *Target*. The *Index* parameter specifies which entry to retrieve and is zero-based. In case of the Front/Back, Upper/Lower, Bode and Octave display, the property will in fact return an IBlock.

3.15.7.7 Property: DataCount

Type: Read-Only

Signature:

```
Property DataCount(Target As Long) As Long
```

Description:

Returns the total number of data entries available in a specific *Target*.

3.15.7.8 Property: DataLink

Type: Read-Only

Signature:

```
Property DataLink(Target As Long, Index As Long) As String
```

Description:

Retrieves one data link from the specified *Target*. The *Index* parameter specifies which link to retrieve and is zero-based.

3.15.7.9 Property: DataLinks

Type: Read-Only

Signature:

```
Property DataLinks(Target As Long) As String()
```

Description:

Returns an array of data links for a specific *Target*.

3.15.7.10 Property: TargetCount

Type: Read-Only

Signature:

```
Property TargetCount As Long
```

Description:

Returns the total number of targets available in the display (see Table 1 on page 74).

3.15.7.11 Property: UIComponentName

Type: Read-Only

Signature:

```
Property UiComponentName As String
```

Description:

Returns the type name of the display.

Example:

```
Set MyPicture = PictureManager.AddPicture( “FrontBack” )
Set MyDisplay = MyPicture.Display( 0 )
MsgBox ( MyDisplay. UiComponentName )
```

A message box containing the message “LmsHq::FunctionalVC::FuncCompFDisplay::CFrontBackLegendToggleUI” will pop-up.

3.15.7.12 Property: CursorManager

Type: Read-Only

Signature:

```
Property CursorManager As ICursorManager
```

Description:

Returns the CursorManager.

Example:

```
Set MyPicture = PictureManager.AddPicture( "FrontBack" )

Set MyDisplay = MyPicture.Display( 0 )

MsgBox ( MyDisplay. UiComponentName )
```

A message box containing the message "LmsHq::FunctionalVC::FuncCompFDisplay::CFrontBackLegendToggleUI" will pop-up.

3.15.8 ICursorManager

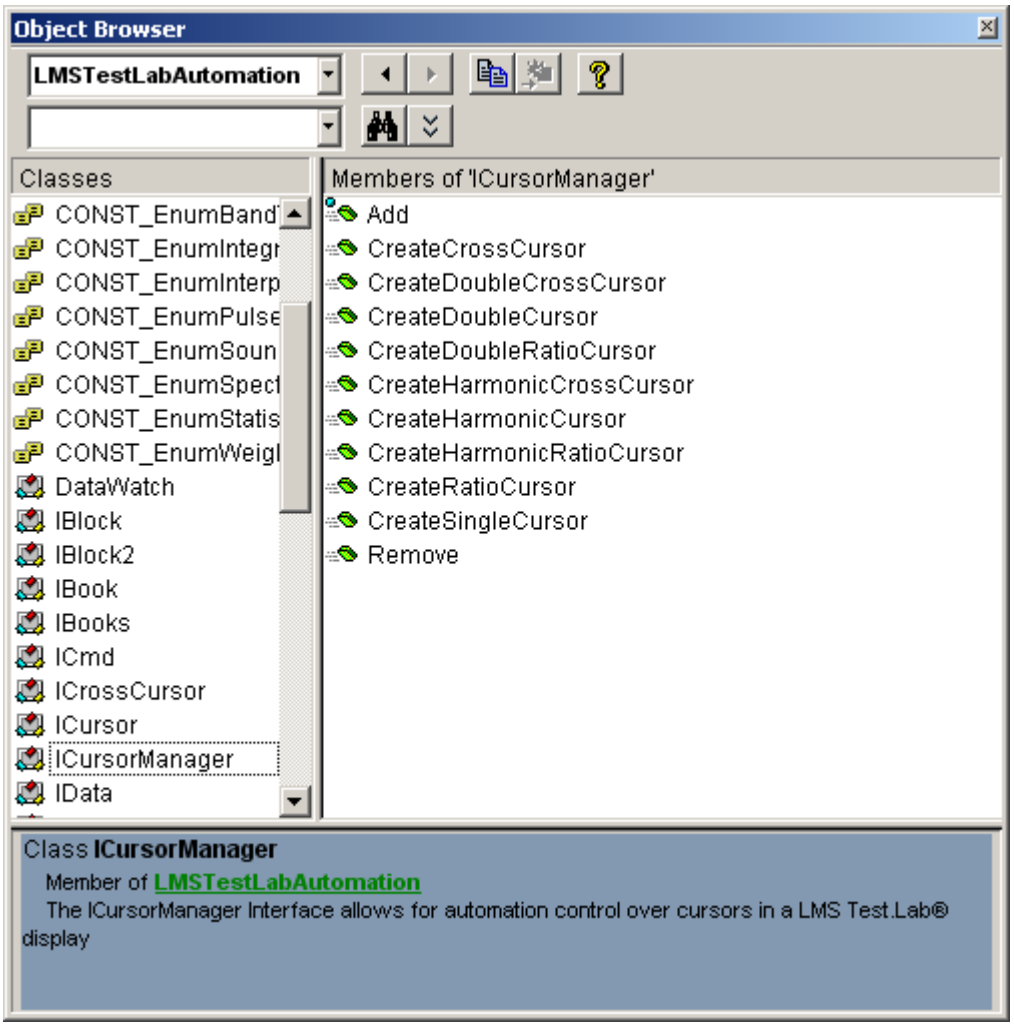


Figure 8

The cursorManager allows for automation control of cursors in Test.Lab Displays.

The most important functions on this interface are the Add and Remove functions.

The other functions are convenience functions, these functions don't change the state of the display itself, but they merely generate, in a single step, complex cursor objects that can be added to the display.

3.15.8.1 About Positions & Axes

Single cursors only have one position.

Double cursors have a master and a slave position. The master cursor is the left hand cursor, the slave the right hand cursor in a default double cursor. (Dragging the master cursor in the display moves both cursors, dragging the slave only moves the slave.)

Harmonic cursors only have one position, this is the position of the specified harmonic. the default harmonic is the first.

Cross cursors have double the amount of positions, once for the first and once for the second axis.

Ratio cursors have a Ratio: this specifies the ratio ot the values on the denominator and the nominator axis.

When creating cursor objects, a position can be specified where to place the cursor. Later on, after creation, the cursor objects can be asked to return the position the cursor is on. Both these positions are expressed in MKS units of the current user unit of the axis the cursor is on.

E.g. a double cursor on the Back Y axis will return 2 positions, both expressed in MKS units of the current user unit of the Y Back axis.

A double cross cursor on the X axis and the Back Y axis has 4 positions: the positions on X and Y axes of the master position, and the positions on X and Y axes of the slave position, all expressed in MKS units of the current user unit of the axis they are on.

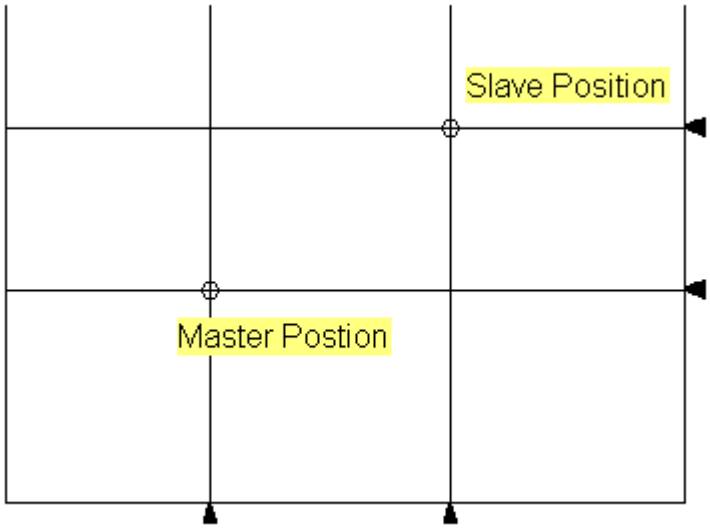
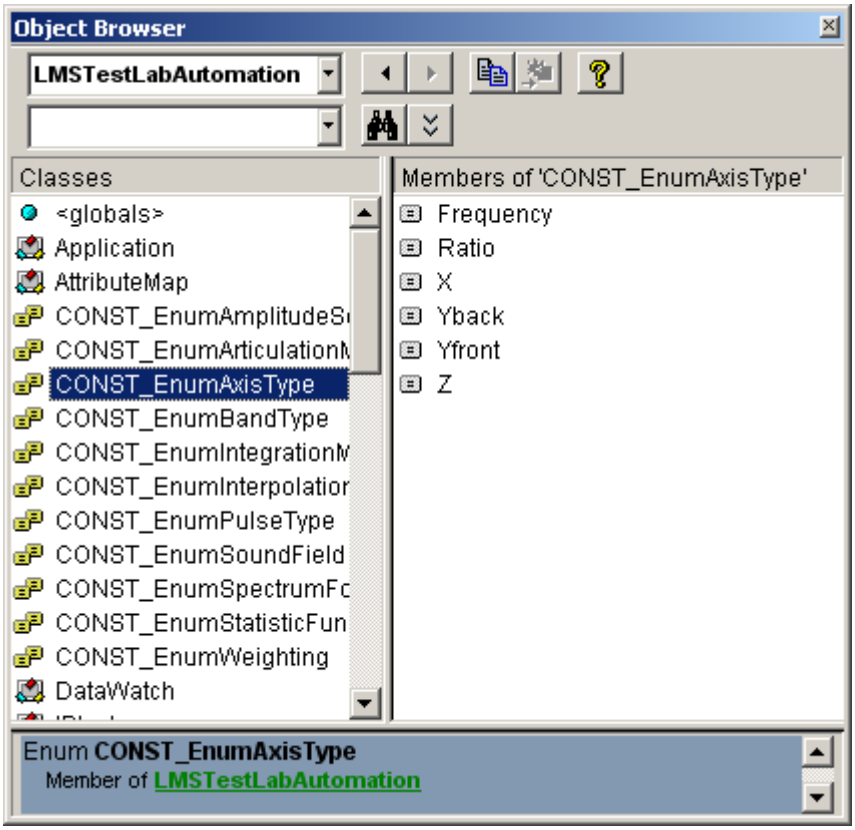


Figure 9

the ratio returned by ratio cursors is a ratio and therefore has no unit

Axes are represented by an axis type.



This enumerate describes the axis the cursor is on.

3.15.8.2 Method: Add

Signature:

```
Function Add (ID As String, Cursor As ICursor) As String
```

Description:

Default member of LMSTestLabAutomation.ICursorManager

Adds a Cursor object to the display & returns a DataPath to the added cursor.

This Data path can be used to create a DataWatch.

The ID must be unique. If you provide an existing ID, you get the error “Cursor with the specified ID already exists.”

(For now the returned data path string is the same string as the provided ID, but this may change in later releases.)

3.15.8.3 Method: Remove

Signature:

```
Sub Remove (ID As String)
```

Description:

Removes the Cursor object from the display.

The ID is the same ID as the one you provided when you added the cursor to the display.

The need for an ID to remove cursors from a display implies that you can only remove cursors you added yourself. You cannot remove cursors that were not added with automation calls.

No error is generated when you provide a wrong ID.

3.15.8.4 Method: CreateSingleCursor

Signature:

Function CreateSingleCursor (Axis As CONST_EnumAxisType, Position As Double, [CursorProperty As IData]) As ISingleCursor

Description:

This function does not change the display; it is just a convenience function on the CursorManager to create an object of type ISingleCursor.

Parameters:

Axis: the axis you want to put the cursor on.

Position: the position you want the cursor to be on. Default is 0.

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.8.5 Method: CreateCrossCursor

Signature:

Function CreateCrossCursor(Axis1 As CONST_EnumAxisType, Axis2 As CONST_EnumAxisType, Position1 As Double, Position2 As Double, [CursorProperty As IData]) As ICrossCursor

Description:

This function does not change the display, it is just a convenience function on the CursorManager to create an object of type ICrossCursor.

Parameters:

Axis1: the first axis you want to put the cursor on.

Axis2: the second axis you want to put the cursor on.

Position1: the position of the cursor on the first axis. Default is 0.

Position2: the position of the cursor on the second axis. Default is 0.

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.8.6 Method: CreateRatioCursor

Signature:

Function CreateRatioCursor(DenominatorAxis As CONST_EnumAxisType, NominatorAxis As CONST_EnumAxisType, Ratio As Double, [CursorProperty As IData]) As IRatioCursor

Description:

This function does not change the display, it is just a convenience function on the CursorManager to create an object of type IRatioCursor.

Parameters:

DenominatorAxis: the denominator axis of the cursor.

NominatorAxis: the nominator axis of the cursor.

Ratio: The Ratio of the cursor.

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.8.7 Method: CreateDoubleCursor

Signature:

Function CreateDoubleCursor(Axis As CONST_EnumAxisType, Position1 As Double, Position2 As Double, [CursorProperty As IData]) As IDoubleCursor

Description:

This function does not change the display, it is just a convenience function on the CursorManager to create an object of type IDoubleCursor.

Parameters:

Axis: the axis you want to put the cursor on.

Position1: the position of the first cursor. Default is 0.

Position2: the position of the second cursor. Default is 0.

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.8.8 Method: CreateDoubleCrossCursor

Signature:

Function CreateDoubleCrossCursor(Axis1 As CONST_EnumAxisType, Axis2 As CONST_EnumAxisType,

Position1 As Double, Position2 As Double, Position3 As Double, Position4 As Double, [CursorProperty As IData]) As IDoubleCrossCursor

Description:

This function does not change the display, it is just a convenience function on the CursorManager to create an object of type IDoubleCrossCursor.

Parameters:

Axis1: the first axis you want to put the cursor on.

Axis2: the second axis you want to put the cursor on.

Position1: the position of the first cursor on the first axis. Default is 0.

Position2: the position of the first cursor on the second axis. Default is 0.

Position3: the position of the second cursor on the first axis. Default is 0.

Position4: the position of the second cursor on the second axis. Default is 0.

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.8.9 Method: CreateDoubleRatioCursor

Signature:

Function CreateDoubleRatioCursor(DenominatorAxis As CONST_EnumAxisType, NominatorAxis As CONST_EnumAxisType, Ratio1 As Double, Ratio2 As Double, [CursorProperty As IData]) As IDoubleRatioCursor

Description:

This function does not change the display, it is just a convenience function on the CursorManager to create an object of type IDoubleRatioCursor.

Parameters:

DenominatorAxis: the denominator axis of the cursor.

NominatorAxis: the nominator axis of the cursor.

Ratio1: The ratio of the first cursor.

Ratio2: The ratio of the second cursor.

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.8.10 Method: CreateHarmonicCursor

Signature:

Function CreateHarmonicCursor(Axis As CONST_EnumAxisType, Position As Double, [Harmonic As Long], [CursorProperty As IData]) As IHarmonicCursor

Description:

This function does not change the display, it is just a convenience function on the CursorManager to create an object of type IHarmonicCursor.

Parameters:

Axis: the axis you want to put the cursor on.

Position: the position you want the cursor to be on. Default is 0.

Harmonic: number of the Harmonic the position is specified for. Default is the first Harmonic. (optional)

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.8.11 Method: CreateHarmonicCrossCursor

Signature:

Function CreateHarmonicCrossCursor(Axis1 As CONST_EnumAxisType, Axis2 As CONST_EnumAxisType, Position1 As Double, Position2 As Double, [Harmonic As Long], [CursorProperty As IData]) As IHarmonicCrossCursor

Description:

This function does not change the display, it is just a convenience function on the CursorManager to create an object of type IHarmonicCrossCursor.

Parameters:

Axis1: the first axis you want to put the cursor on.

Axis2: the second axis you want to put the cursor on.

Position1: the position of the cursor on the first axis. Default is 0.

Position2: the position of the cursor on the second axis. Default is 0.

Harmonic: number of the Harmonic the positions are specified for. Default is the first Harmonic. (optional)

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.8.12 Method: CreateHarmonicRatioCursor

Signature:

Function CreateHarmonicRatioCursor(DenominatorAxis As CONST_EnumAxisType, NominatorAxis As CONST_EnumAxisType,

Ratio As Double, [Harmonic As Long], [CursorProperty As IData]) As IHarmonicRatioCursor

Description:

This function does not change the display, it is just a convenience function on the CursorManager to create an object of type IHarmonicRatioCursor.

Parameters:

DenominatorAxis: the denominator axis of the cursor.

NominatorAxis: the nominator axis of the cursor.

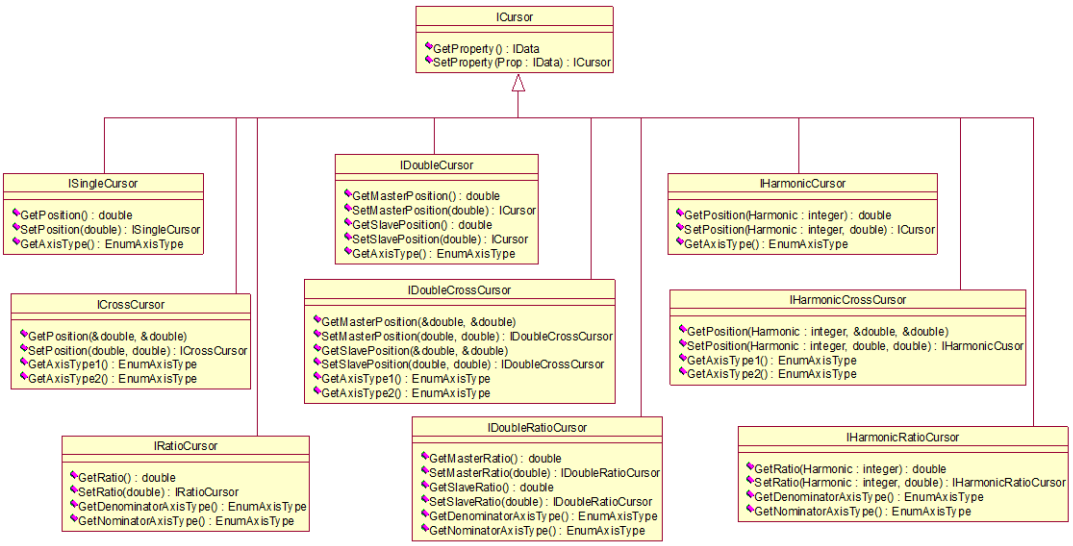
Ratio: The Ratio of the cursor.

Harmonic: number of the Harmonic the positions are specified for. Default is the first Harmonic. (optional)

CursorProperty: an IData Object containing additional Cursorproperties: e.g. Line Style,… (optional)

3.15.9 The Cursor interfaces

There is a hierarchy in the interfaces for cursors.



ICursor contains all the information that is common for all cursors

The other ICursor interfaces contain Get and Set functions for properties that are relevant for each specific type of cursor. These interfaces also support the ICursor interface, therefore the common information is also available on these interfaces.

To interact with cursors, you can listen on the DataWatch returned by the Add function on the CursorManager for changes(e.g. a user dragged the cursor to another position). To update a cursor in a display via automation, you can retrieve the cursor object from the port, call a Set function (e.g. Position = 20) and put the cursor object that the Set function returned back on the same port.

3.15.10 ICursor

3.15.10.1 Property: CursorProperty

Signature:

Property CursorProperty As IData

Description:

This property returns the property object of the cursor. This object is a Test.Lab object of type LmsHq/DataModelVI/PropertyFDisp/ICursorProperty. For further information about using and constructing Test.Lab objects cfr. the automation documentation tool.

3.15.10.2 Function: SetCursorProperty

Signature:

Function SetCursorProperty(CursorProperty As IData) As ICursor

Description:

This function returns a new instance of the cursor, with the provided Cursor properties applied.

Parameters:

CursorProperty: a Test.Lab datamodel object that is supports LmsHq/DataModelVI/PropertyFDisp/ICursorProperty. The actual Test.Lab types corresponding can be retrieved with the automation documentation tool.

3.15.11 ISingleCursor

3.15.11.1 Property: AxisType

Signature:

```
Property AxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the axis this cursor is on.

3.15.11.2 Property: Position

Signature:

```
Property Position As Double
```

Description:

This property returns the position this cursor is on.

3.15.11.3 Function: SetPosition

Signature:

```
Function SetPosition(Position As Double) As ISingleCursor
```

Description:

This function returns a new instance of the Cursor placed at the provided position.

Parameters:

Position: the new position of the cursor.

3.15.12 ICrossCursor

3.15.12.1 Property: AxisType1

Signature:

```
Property AxisType1 As CONST_EnumAxisType
```

Description:

This property returns the type of the first axis this cursor is on.

3.15.12.2 Property: AxisType2

Signature:

```
Property AxisType2 As CONST_EnumAxisType
```

Description:

This property returns the type of the second axis this cursor is on.

3.15.12.3 Function: GetPosition

Signature:

```
Sub GetPosition(Position1 As Double, Position2 As Double)
```

Description:

This function returns nothing. It sets the passed parameters to the 2 positions of the cursor.

Parameters:

Position1: this variable receives the position of the cursor on the first axis.

Position2: this variable receives the position of the cursor on the second axis.

3.15.12.4 Function: SetPosition

Signature:

```
Function SetPosition(Position1 As Double, Position2 As Double) As ICrossCursor
```

Description:

This function returns a new instance of the Cursor placed at the provided positions.

Parameters:

Position1: the new position of the cursor on the first axis.

Position2: the new position of the cursor on the second axis.

3.15.13 IRatioCursor

3.15.13.1 Property: DenominatorAxisType

Signature:

```
Property DenominatorAxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the denominator axis this cursor is on.

3.15.13.2 Property: NominatorAxisType

Signature:

```
Property NominatorAxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the nominator axis this cursor is on.

3.15.13.3 Property: Ratio

Signature:

```
Property Ratio As Double
```

Description:

This property returns the ratio of this cursor.

3.15.13.4 Function: SetRatio

Signature:

```
Function SetRatio(Ratio As Double) As IRatioCursor
```

Description:

This function returns a new instance of the cursor with the new ratio.

Parameters:

Ratio: the new ratio of the cursor.

3.15.14 IDoubleCursor

3.15.14.1 Property: AxisType

Signature:

```
Property AxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the axis this cursor is on.

3.15.14.2 Property: MasterPosition

Signature:

```
Property MasterPosition As Double
```

Description:

This property returns the position of the first (master) cursor of this double cursor.

3.15.14.3 Property: SlavePosition

Signature:

```
Property SlavePosition As Double
```

Description:

This property returns the position of the second (slave) cursor of this double cursor.

3.15.14.4 Function: SetMasterPosition

Signature:

```
Function SetMasterPosition(MasterPosition As Double) As IDoubleCursor
```

Description:

This function returns a new instance of the double cursor with the first (master) cursor placed at the provided position.

Parameters:

MasterPosition: the new position of the first (master) cursor.

3.15.14.5 Function: SetSlavePosition

Signature:

```
Function SetSlavePosition(SlavePosition As Double) As IDoubleCursor
```

Description:

This function returns a new instance of the double cursor with the second (slave) cursor placed at the provided position.

Parameters:

SlavePosition: the new position of the second (slave) cursor.

3.15.15 IDoubleCrossCursor

3.15.15.1 Property: AxisType1

Signature:

```
Property AxisType1 As CONST_EnumAxisType
```

Description:

This property returns the type of the first axis this cursor is on.

3.15.15.2 Property: AxisType2

Signature:

```
Property AxisType2 As CONST_EnumAxisType
```

Description:

This property returns the type of the second axis this cursor is on.

3.15.15.3 Function: GetMasterPosition

Signature:

```
Sub GetMasterPosition(MasterPosition1 As Double, MasterPosition2 As Double)
```

Description:

This function returns nothing. It sets the passed parameters to the 2 positions of the first (master) cursor.

Parameters:

Position1: this variable receives the position of the first (master) cursor on the first axis.

Position2: this variable receives the position of the first (master) cursor on the second axis.

3.15.15.4 Function: GetSlavePosition

Signature:

```
Sub GetSlavePosition(SlavePosition1 As Double, SlavePosition2 As Double)
```

Description:

This function returns nothing. It sets the passed parameters to the 2 positions of the second (slave) cursor.

Parameters:

Position1: this variable receives the position of the second (slave) cursor on the first axis.

Position2: this variable receives the position of the second (slave) cursor on the second axis.

3.15.15.5 Function: SetMasterPosition

Signature:

```
Function SetMasterPosition(MasterPosition1 As Double, MasterPosition2 As Double) As IDoubleCrossCursor
```

Description:

This function returns a new instance of the double cross cursor with the first (master) cursor placed at the provided positions.

Parameters:

MasterPosition1: the new position of first (master) cursor on the first axis.

MasterPosition2: the new position of first (master) cursor on the second axis.

3.15.15.6 Function: SetSlavePosition

Signature:

```
Function SetSlavePosition(SlavePosition1 As Double, SlavePosition2 As Double) As IDoubleCrossCursor
```

Description:

This function returns a new instance of the double cross cursor with the second (slave) cursor placed at the provided positions.

Parameters:

SlavePosition1: the new position of second (slave) cursor on the first axis.

SlavePosition2: the new position of second (slave) cursor on the second axis.

3.15.16 IDoubleRatioCursor

3.15.16.1 Property: DenominatorAxisType

Signature:

```
Property DenominatorAxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the denominator axis this cursor is on.

3.15.16.2 Property: NominatorAxisType

Signature:

```
Property NominatorAxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the nominator axis this cursor is on.

3.15.16.3 Property: MasterRatio

Signature:

```
Property MasterRatio As Double
```

Description:

This property returns the ratio of the first (master) cursor of this cursor.

3.15.16.4 Property: SlaveRatio

Signature:

```
Property SlaveRatio As Double
```

Description:

This property returns the ratio of the second (slave) cursor of this cursor.

3.15.16.5 Function: SetMasterRatio

Signature:

```
Function SetMasterRatio(MasterRatio As Double) As IDoubleRatioCursor
```

Description:

This function returns a new instance of the double ratio cursor with the first (master) cursor set to the new ratio.

Parameters:

MasterRatio: the new ratio of the first (master) cursor.

3.15.16.6 Function: SetSlaveRatio

Signature:

```
Function SetSlaveRatio(SlaveRatio As Double) As IDoubleRatioCursor
```

Description:

This function returns a new instance of the double ratio cursor with the second (slave) cursor set to the new ratio.

Parameters:

SlaveRatio: the new ratio of the second (slave) cursor.

3.15.17 IHarmonicCursor

3.15.17.1 Property: AxisType

Signature:

```
Property AxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the axis this cursor is on.

Parameters:

/

3.15.17.2 Property: Position

Signature:

```
Property Position([Harmonic As Long]) As Double
```

Description:

This property returns the position of the specified harmonic of the cursor.

Parameters:

Harmonic: number of the Harmonic the position is returned for. Default is the first Harmonic. (optional)

3.15.17.3 Function: SetPosition

Signature:

```
Function SetPosition([Harmonic As Long], Position As Double) As IHarmonicCursor
```

Description:

This function returns a new instance of the harmonic cursor placed at the provided position.

Parameters:

Harmonic: number of the Harmonic the position is specified for. Default is the first Harmonic. (optional)

Position: the new position of the cursor.

3.15.18 IHarmonicCrossCursor

3.15.18.1 Property: AxisType1

Signature:

```
Property AxisType1 As CONST_EnumAxisType
```

Description:

This property returns the type of the first axis this cursor is on.

3.15.18.2 Property: AxisType2

Signature:

```
Property AxisType2 As CONST_EnumAxisType
```

Description:

This property returns the type of the second axis this cursor is on.

3.15.18.3 Function: GetPosition

Signature:

```
Sub GetPosition([Harmonic As Long], Position1 As Double, Position2 As Double)
```

Description:

This function returns nothing. It sets the passed parameters to the 2 positions of the specified harmonic of the cursor.

Parameters:

Harmonic: number of the Harmonic the position is returned for. Default is the first Harmonic. (optional)

Position1: this variable receives the position of the specified harmonic of the cursor on the first axis.

Position2: this variable receives the position of the specified harmonic of the cursor on the second axis.

3.15.18.4 Function: SetPosition

Signature:

```
Function SetPosition([Harmonic As Long], Position1 As Double, Position2 As Double) As IHarmonicCrossCursor
```

Description:

This function returns a new instance of the harmonic cross cursor placed at the provided positions.

Parameters:

Harmonic: number of the Harmonic the position is specified for. Default is the first Harmonic. (optional)

Position1: the new position of the specified harmonic of the cursor on the first axis.

Position2: the new position of the specified harmonic of the cursor on the second axis.

3.15.19 IHarmonicRatioCursor

3.15.19.1 Property: DenominatorAxisType

Signature:

```
Property DenominatorAxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the denominator axis this cursor is on.

3.15.19.2 Property: NominatorAxisType

Signature:

```
Property NominatorAxisType As CONST_EnumAxisType
```

Description:

This property returns the type of the nominator axis this cursor is on.

3.15.19.3 Property: Ratio

Signature:

```
Property Ratio([Harmonic As Long]) As Double
```

Description:

This property returns the ratio of this cursor.

Parameters:

Harmonic: number of the Harmonic the ratio is returned for. Default is the first Harmonic. (optional)

3.15.19.4 Function: SetRatio

Signature:

```
Function SetRatio([Harmonic As Long], Ratio As Double) As IHarmonicRatioCursor
```

Description:

This function returns a new instance of the harmonic ratio cursor with the new ratio.

Parameters:

Harmonic: number of the Harmonic the ratio is specified for. Default is the first Harmonic. (optional)

Ratio: the new ratio of the cursor.

4. TEST.LAB™ ACTIVEX MODULE LOADER.

4.1 **Definition.**

As explained in 'Test.Lab™ Workbook loader automation interfaces.' the user can launch the main executable that allows them to load an application description. These descriptions are stored in worbooks and specify the list of modules necessary to make up the whole application. The modularity of these components is such that modules are not bound to one single context but they can be reused in several combinations.

The ActiveX Module loader is an ActiveX control that allows loading and running a single Test.Lab module. This virtually allows any Test.lab module to be loaded and run inside an application that can host ActiveX controls. The only difference between an ActiveX control and a regular executable is that it needs a host application to run in. So it runs in-process with its host and displays its GUI into the GUI of the host. Internet Explorer, VB 6.0, C++, .NET development environments are only a couple of examples in which you can use ActiveX controls.

Test.Lab currently only supports tree modules.

- Default
- PictureManager
- Picture

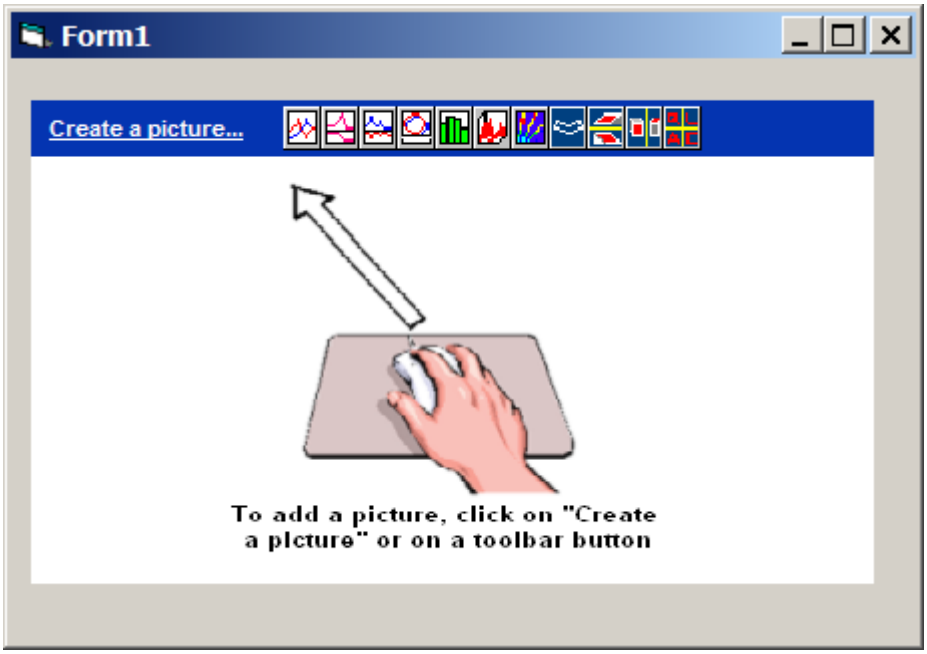
4.2 **Default.**

This module is the module that is loaded by default. It is shown here on the GUI of an external application.

This merely shows a bitmap with the Test.lab logo on it.

4.3 **PictureManager.**

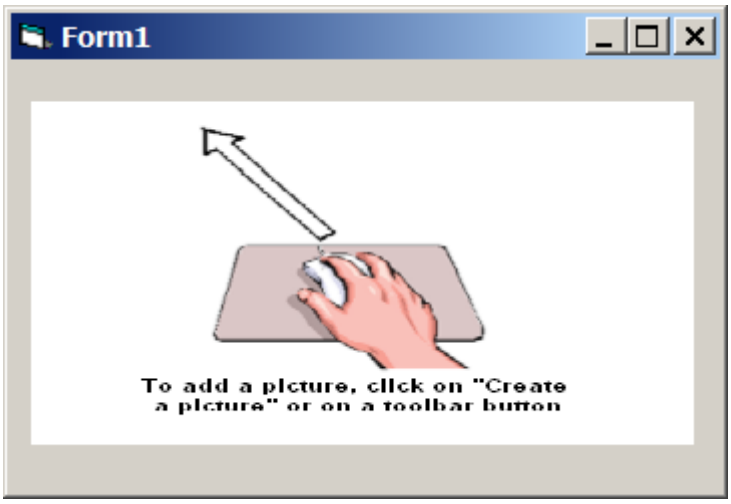
This module contains the picture manager functionality as it is presented in the 'Navigator' of 'Measure' sheet for instance.



This module can be used in the same way it can be used in Test.lab itself.

4.4 Picture.

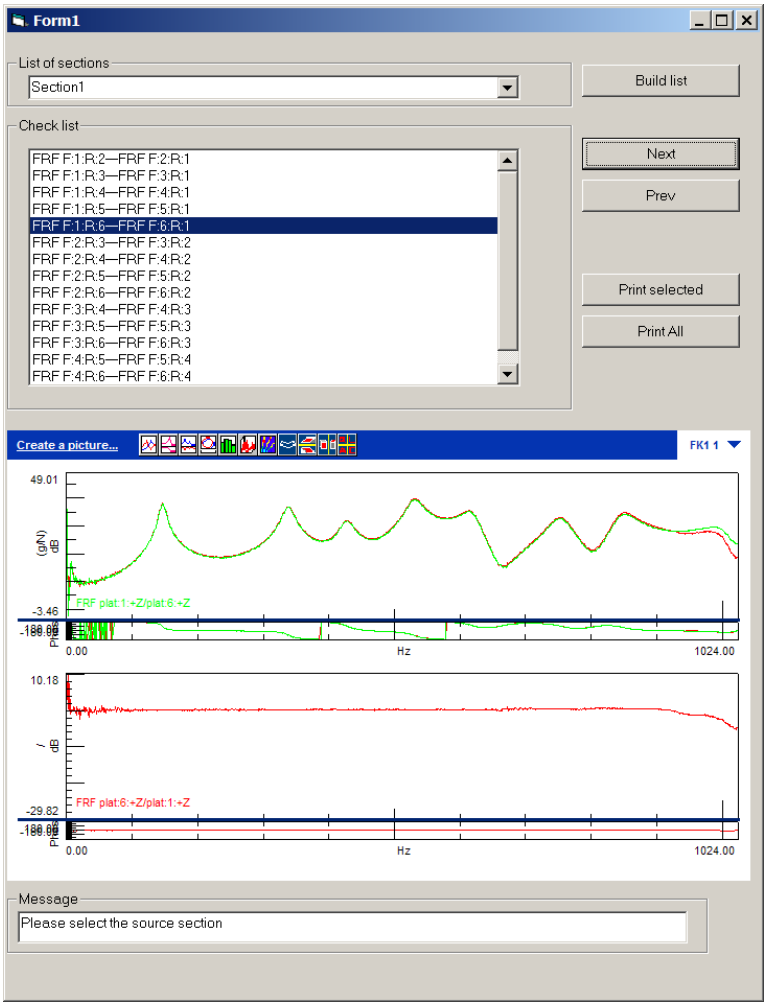
The 'Picture' module is a more restricted version of the 'Picturemanager' module in such that the users has no interactive access to the bar on the top.



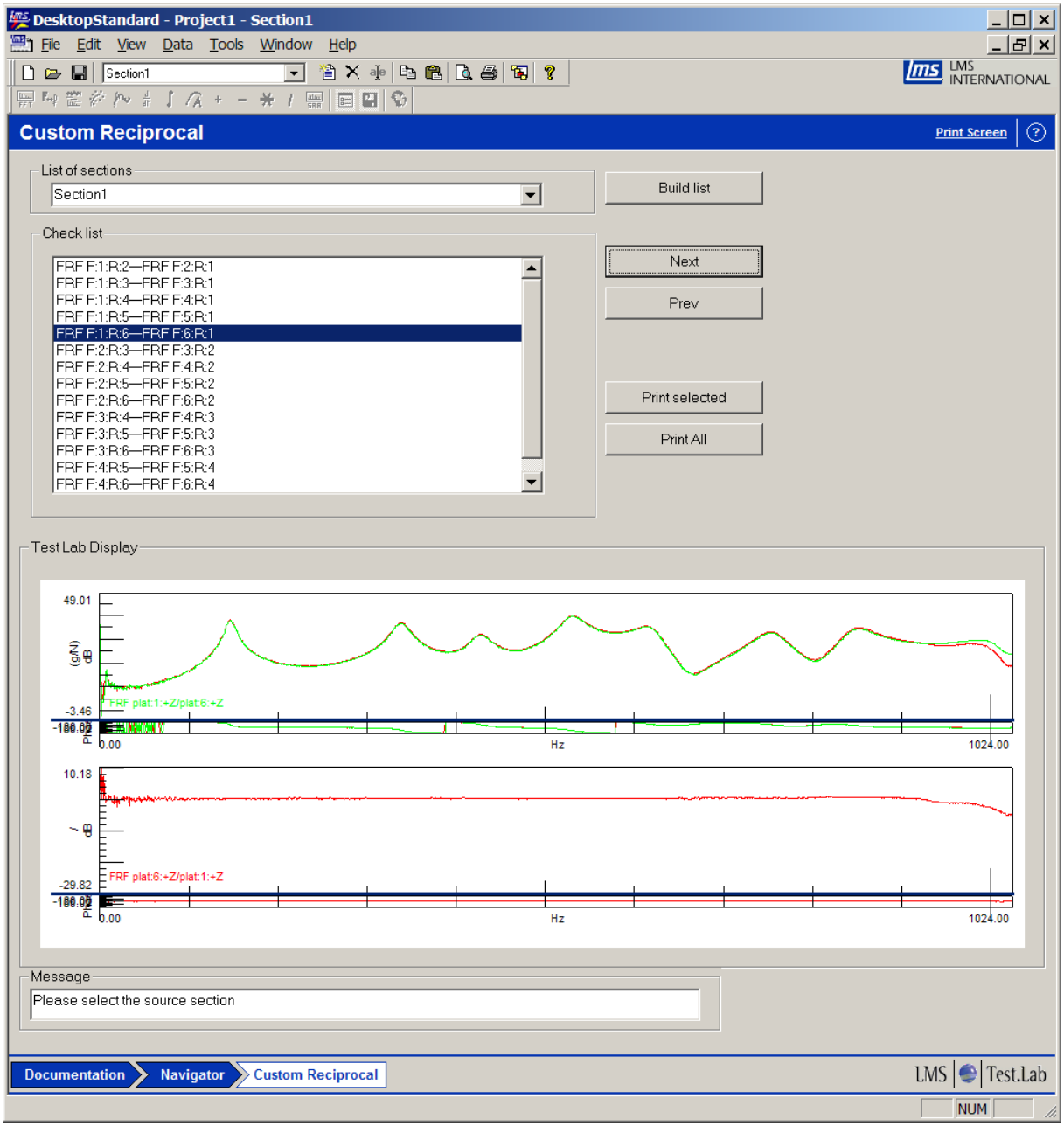
This module is more suited in situations where the developer wants to restrict access to the different manipulations offered by the picturemanager.

4.5 **Example.**

Following example shows an external application that hosts the picture manager and uses the functionality to present some analysis results.

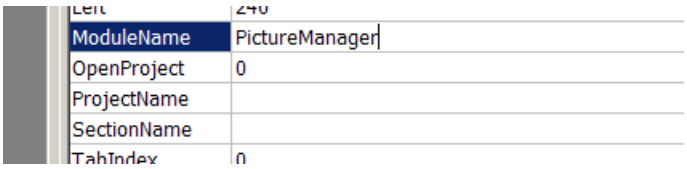


In stead of compiling an executable into an standalone window application, it can be compiled again into an ActiveX control itself, and hosted in one of the User add-in sheets in Test.Lab. An example of this is presented in the next figure.



4.6 ActiveX Module Loader design properties.

There are a couple of properties you can specify to configure the behaviour upon loading a module. First of all there ‘s the module name, which can be changed from ‘Default’ to any other supported module.



Depending on the requirements of the module to load, the user can require to open a project. To do so, the 'OpenProject' flag has to be changed to non-zero. If, in that case, the 'ProjectName' is left blank, a default project will be opened, just as with 'New project' in the Test.Lab workbook loader. Otherwise the specified project will be opened. Specifying a Section name will use the settings in that specific section, otherwise it will revert to the default section.

Left	240
ModuleName	PictureManager
OpenProject	-1
ProjectName	d:/local/Data/MyProject.lms
SectionName	Section2
TabIndex	0

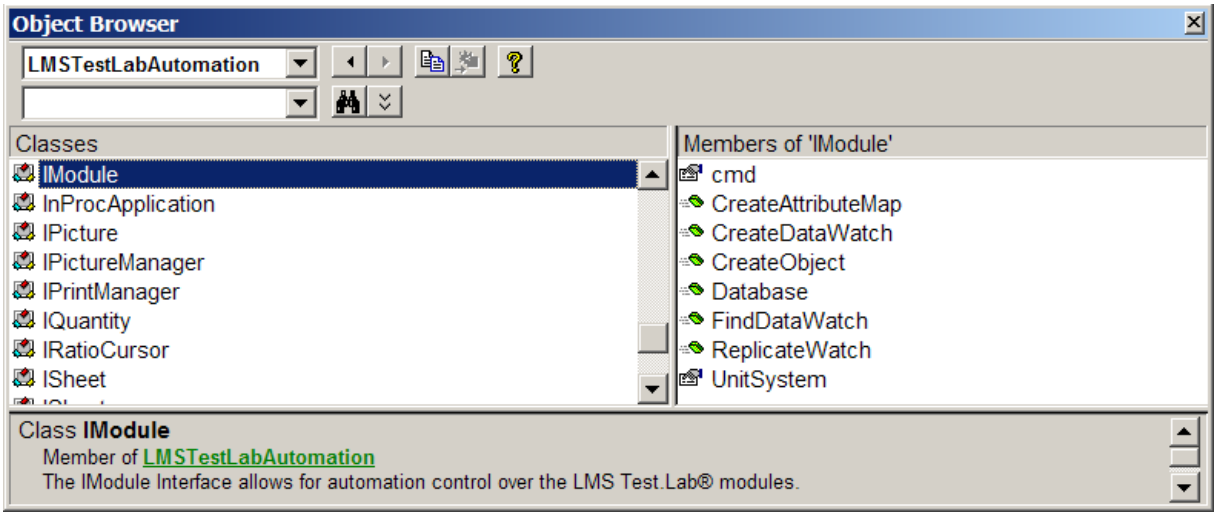
4.7 IModule interface.

Being able to host a module inside an external application is neat, but its use would only be limited if you wouldn't be able to interact with it. In the case of 'PictureModule' this would allow the user to load a picture interactively, but it is of little functional use if the client application can't put any data in it.

The 'Module' property on the control allows access to the ActiveXModule loader and the actually loaded module in the same way the Application automation object allows access to the Test.Lab workbook loader.

Following is a piece of sample code to indicate how to retrieve such an IModule object from the control, and use it to e.g. access a Test.Lab object.

```
Dim tl_module As LMSTestLabAutomation.IModule

Set tl_module = ActiveXModuleLoader1.Module

Dim tl_db As LMSTestLabAutomation.IDatabase

Set tl_db = tl_module.Database("c:/temp/full_plate6x6.lms")
```



All the members of the IModule interface have the same type of functioning and meaning as those mentioned in the discussion of the interfaces for Test.Lab workbook loader automation.

- Cmd (cfr 3.1.16)

- CreateAttributemap (cfr 3.1.14)
- CreateDataWatch (cfr 3.3.11)
- CreateObject (cfr3.1.11)
- Database This method differs slightly with the Database method discussed earlier .

  Signature:

  Database([ProjectPathName : String]) : IDataBase

  Description:

  When ‘ProjectPathname’ is specified the filename will be opened and returned as a IDatabase object. This is equivalent to 3.1.12.

  When no parameter is specified, the active project, opened by means of the design property, will be returned. This is equivalent to 3.3.12.
- FindDataWatch (cfr 3.3.10)
- ReplicateWatch : Currently unsupported.
- UnitSystem (cfr 3.1.4)